

VISUAL QUICKSTART GUIDE

Get up and running in no time!



**Register your book for
free video training!**
www.peachpit.com/register

Downloadable examples from the companion Web site!

HTML5 and CSS3

Seventh Edition

ELIZABETH CASTRO • BRUCE HYSLOP

© LEARN THE QUICK AND EASY WAY!

GUIA VISUAL DE INÍCIO RÁPIDO

HTML5 e CSS3

Sétima Edição

Elizabeth Castro • Bruce Hyslop



Imprensa de pêssego

HTML5 e CSS3, sétima edição: Guia visual de início rápido

Elizabeth Castro e Bruce Hyslop

Imprensa de pêssego
Rua Oitava, 1249
Berkeley, CA 94710
510/524-2178
510/524-2221 (fax)

Encontre-nos na Web em www.peachpit.com.

Para relatar erros, envie uma nota para errata@peachpit.com.
Peachpit Press é uma divisão da Pearson Education.

Copyright © 2012 por Elizabeth Castro e Bruce Hyslop

Editor: Clifford Colby
Editor de desenvolvimento: Robyn G. Thomas
Editor de produção: Cory Borman
Compositor: David Van Ness
Editor de texto: Scout Festa
Revisor: Nolan Hester
Editores técnicos: Michael Bester e Chris Casciano
Indexadora: Valerie Haynes Perry
Design da capa: RHDG/Riezebos Holzbaur Design Group, Peachpit Press
Design de interiores: Peachpit Press
Design do logotipo: MINE™ www.minesf.com

Aviso de Direitos

Todos os direitos reservados. Nenhuma parte deste livro pode ser reproduzida ou transmitida de qualquer forma, por qualquer meio, eletrônico, mecânico, fotocópia, gravação ou outro, sem a permissão prévia por escrito do editor. Para obter informações sobre como obter permissão para reimpressões e trechos, entre em contato com permissions@peachpit.com.

Capturas de tela de bart.gov cortesia do Distrito de Trânsito Rápido da Área da Baía de São Francisco (BART).

Capturas de tela de css3generator.com cortesia de Randy Jensen.

Capturas de tela de dribbble.com cortesia de Dan Cederholm.

Capturas de tela de fontquirrel.com cortesia de Ethan Dunham.

Foodense.is capturas de tela cortesia de Julie Lamba.

capturas de tela de modernizr.com cortesia de Faruk Ates.

Capturas de tela de namecheap.com cortesia de Namecheap.

Aviso de responsabilidade

As informações contidas neste livro são distribuídas "como estão", sem garantia. Embora todas as precauções tenham sido tomadas na preparação do livro, nem os autores nem a Peachpit terão qualquer responsabilidade perante qualquer pessoa ou entidade com relação a qualquer perda ou dano causado ou supostamente causado direta ou indiretamente pelas instruções contidas neste livro, ou pelos produtos de software e hardware de computador nele descritos.

Marcas registradas

Muitas das designações utilizadas pelos fabricantes e vendedores para distinguir os seus produtos são reivindicadas como marcas comerciais. Quando essas designações aparecem neste livro, e a Peachpit estava ciente de uma reivindicação de marca registrada, as designações aparecem conforme solicitado pelo proprietário da marca registrada. Todos os outros nomes de produtos e serviços identificados ao longo deste livro são usados apenas de forma editorial e para o benefício de tais empresas, sem intenção de violação da marca registrada. Nenhum tal uso, ou o uso de qualquer nome comercial, tem a intenção de transmitir endosso ou outra afiliação a este livro.

ISBN-13: 978-0-321-71961-4

ISBN-10: 0-321-71961-1

9 8 7 6 5 4 3 2 1

Impresso e encadernado nos Estados Unidos da América

Para a família.

Agradecimentos

Escrever os agradecimentos é um dos desafios mais assustadores de trabalhar em um livro, porque você quer ter certeza de transmitir seu apreço por todos de maneira adequada. Este livro é fruto do apoio, do trabalho incansável e do bom humor de muita gente. Espero fazer justiça a todos eles e espero que vocês me concedam um pouco enquanto lhes agradeço.

Um agradecimento muito sincero vai para:

Nancy Aldrich-Ruenzel e Nancy Davis, por me confiarem esta edição de um livro que tem sido importante para a Peachpit há muitos anos.

Cliff Colby, por me recomendar e tornar isso possível; pela sua confiança em mim e pela sua paciência, flexibilidade e orientação; e para inúmeras conversas e muitas

de risadas.

Robyn Thomas, por seu tremendo esforço em nos manter no caminho certo, discutindo incontáveis documentos, fazendo edições e sugestões cuidadosas e fornecendo palavras regulares de encorajamento, que sempre foram um incentivo.

Michael Bester, por todos os comentários e sugestões pontuais, detectando erros técnicos e omissões e nos ajudando a transmitir a mensagem certa aos leitores. Foi um verdadeiro prazer trabalhar com ele em outro livro.

Chris Casciano, na mesma linha, por todo o seu conhecimento técnico, sugestões e feedback crucial. Gostei muito de você ter se juntado a nós nas últimas semanas; tivemos sorte de ter você.

Cory Borman, por supervisionar habilmente a produção do livro e criar diagramas rapidamente, e por seu bom humor.

Scout Festa, por corrigir cuidadosamente a gramática e a pontuação, aperfeiçoar a linguagem, garantir a precisão das referências de figuras e capítulos e, acima de tudo, fornecer um nível de polimento muito importante.

David Van Ness, por seu grande cuidado no layout das páginas e por sua proficiência e atenção aos detalhes.

Nolan Hester, por emprestar sua experiência ao esforço de revisar as páginas dispostas.

Valerie Haynes Perry, por lidar com a tarefa crítica de criar um índice eficaz no qual os leitores confiarão continuamente.

Aos numerosos profissionais de marketing, vendas e outras pessoas da Peachpit por trabalharem nos bastidores para tornar o livro um sucesso.

À minha família e amigos, por acompanharem meu progresso e proporcionarem pausas ocasionais e bem-vindas na escrita. Agradeço principalmente àqueles amigos que provavelmente se cansaram de me ouvir dizer muitas vezes que eu não poderia me reunir, mas que continuaram perguntando mesmo assim.

Robert Reinhardt, como sempre, por me iniciar na escrita de livros e por sua orientação enquanto eu iniciava este.

A comunidade da Web, por suas inovações e por compartilhar seu conhecimento para que outros possam se beneficiar (citei muitos de vocês ao longo do livro).

A vocês, leitores, pelo interesse em aprender sobre HTML e CSS e por selecionarem este livro; Eu sei que você tem muitos outros para escolher. Espero que o livro lhe sirva bem.

Muito obrigado aos seguintes autores colaboradores. Os leitores têm um livro mais valioso devido aos seus esforços, pelos quais sou grato. Também gostaria de pedir desculpas a Erik Vorhes por não termos conseguido encaixar os Apêndices A e B no livro.

Leitores que os veem no site do livro certamente apreciará seu trabalho.

Em ordem alfabética de sobrenome, os autores colaboradores são:

Scott Boms (Capítulo 14)

Scott é um premiado designer, escritor e palestrante que fez parceria com organizações como PayPal, HSBC, Hyundai, DHL, XM Radio, revista Toronto Life e Masterfile durante seus mais de 15 anos de trabalho na Web. Quando ele está longe do computador, você pode encontrá-lo tirando fotos Polaroids; tocando bateria com sua banda, George; ou aproveitando o tempo com sua maravilhosa esposa e dois filhos. Ele é

@scottboms no Twitter.

Ian Devlin (Capítulo 17)

Ian Devlin é um desenvolvedor Web, blogueiro e autor irlandês que gosta de programar e escrever sobre tecnologias emergentes da Web, como HTML5 e CSS3. Além de desenvolvimento front-end, Ian também constrói soluções com tecnologias back-end como .NET e PHP. Ele escreveu recentemente um livro, HTML5 Multimedia: Develop and Design (Peachpit Press, 2011).

Seth Lemoine (capítulos 5 e 16)

Seth Lemoine é desenvolvedor de software e professor em Atlanta. Por mais de dez anos, ele trabalhou em projetos desafiadores para ver o que é possível, com tecnologias que vão desde HTML, JavaScript e CSS até Objective-C e Ruby. Seja encontrando maneiras inovadoras de ensinar HTML5 e CSS para seus alunos ou aperfeiçoando uma receita Schezuan em seu wok ao ar livre, ser criativo é sua paixão.

Erik Vorhes (Apêndices A e B, disponíveis no site do livro)

Erik Vorhes cria coisas para a Web com VSA Partners e é editor-chefe da Typedia (<http://typedia.com/>). Ele mora e trabalha em Chicago.

Brian Warren (Capítulo 13)

Brian Warren é designer sênior da Happy Cog na Filadélfia. Quando não está escrevendo ou desenhando, ele passa o tempo brincando com sua linda família, ouvindo música e fazendo cerveja. Ele bloga, de forma intermitente, em <http://begoodnotbad.com>.

E, por fim, gostaria de deixar um agradecimento especial a Elizabeth Castro. Ela criou a primeira edição deste livro há mais de 15 anos atrás e nutriu seu público a cada edição que se seguiu. Seu estilo de ensino ressoou em literalmente centenas de milhares de leitores ao longo dos anos. Sou extremamente grato pela oportunidade de fazer parte deste livro e tive o cuidado de fazer o que é certo tanto para ele quanto para os leitores enquanto trabalhava nesta edição.

—Bruce

Resumo do conteúdo

agradecimentos	4
introdução	xv
Capítulo 1 Blocos de construção de páginas da Web	1
Capítulo 2 Trabalhando com arquivos de páginas da Web	25
Capítulo 3 Estrutura HTML básica	41
Texto do capítulo 4	99
Imagens do capítulo 5	147
Links do capítulo 6	165
Capítulo 7 Blocos de construção CSS	179
Capítulo 8 Trabalhando com folhas de estilo	197
Capítulo 9 Definindo seletores	213
Capítulo 10 Formatando texto com estilos	241
Capítulo 11 Layout com estilos	275
Capítulo 12 Folhas de estilo de Mobile para Desktop	327
Capítulo 13 Trabalhando com fontes da Web	353
Capítulo 14 Melhorias com CSS3	371
Listas do Capítulo 15	397
Capítulo 16 Formulários	417
Capítulo 17 Vídeo, áudio e outros recursos multimídia	449
Capítulo 18 tabelas	489
Capítulo 19 Trabalhando com scripts	497
Capítulo 20 Teste e depuração de páginas da Web	505
Capítulo 21 publicando suas páginas na Web	521
índice	529

Índice

Agradecimentos	4
Introdução	xv
HTML e CSS em resumo.	xvi
Aprimoramento progressivo: uma prática recomendada	XVIII
Esse livro é para você?	xx
Como este livro funciona.	.XXII
Site complementar.	XXIV
 Capítulo 1 Blocos de construção de páginas da Web	1
Uma página HTML básica.	3
HTML semântico: marcação com significado.	6
Marcação: elementos, atributos e valores	.13
Conteúdo de texto de uma página da Web.	16
Links, imagens e outros conteúdos não textuais	.17
Nomes de arquivos.	19
URLs	20
Principais conclusões.	24
 Capítulo 2 Trabalhando com arquivos de páginas da Web	25
Planejando seu site.	26
Criando uma nova página da Web.	28
Salvando sua página da web.	30
Especificando uma página padrão ou página inicial.	33
Editando páginas da Web	35
Organizando Arquivos	36
Visualizando sua página em um navegador.	37
A inspiração de outros.	39
 Capítulo 3 Estrutura HTML Básica	41
Iniciando sua página da Web.	43
Criando um título.	46
Criando títulos	48
Compreendendo o esboço do documento HTML5.	50
Agrupando títulos.	58
Construções de páginas comuns.	60

Criando um cabeçalho.	61
Marcação de navegação	64
Criando um Artigo.	68
Definindo uma seção.	72
Especificando um aparte.	75
Criando um rodapé	80
Criando contêineres genéricos.	84
Melhorando a acessibilidade com	
elementos de nomenclatura ARIA com classe ou ID.	88
Adicionando o atributo Title aos elementos.	92
Adicionando comentários	95
	96

Capítulo 4 Texto 99

Iniciando um novo parágrafo.	0,100
Adicionando informações de contato do autor.	102
Criando uma Figura	104
Especificando hora.	106
Marcando texto importante e enfatizado.	110
Indicando uma Citação ou Referência.	112
Citando Texto.	113
Destacando texto.	116
Explicando abreviaturas	118
Definindo um Termo	0,120
Criação de sobrescritos e subscritos.	121
Observando edições e textos imprecisos.	124
Código de marcação.	128
Usando texto pré-formatado.	0,130
Especificação de letras miúdas	132
Criando uma quebra de linha	133
Criando períodos	134
Outros Elementos.	136

Capítulo 5 Imagens 147

Sobre Imagens para a Web	148
Obtendo imagens	152
Escolhendo um Editor de Imagens.	153
Salvando suas imagens	154
Inserindo imagens em uma página	0,156
Oferecendo texto alternativo.	157
Especificando o tamanho da imagem	0,158
Dimensionando imagens com o navegador	0,160

Dimensionando imagens com um editor de imagens.	161
Adicionando ícones ao seu site.	162
Capítulo 6 Links	165
A anatomia de um link	0,166
Criando um link para outra página da web.	167
Criando Âncoras	172
Vinculando a uma âncora específica.	174
Criando outros tipos de links.	175
Capítulo 7 Blocos de Construção CSS	179
Construindo uma regra de estilo.	181
Adicionando comentários às regras de estilo.	182
A cascata: quando as regras colidem.	184
O valor de uma propriedade.	188
Capítulo 8 Trabalhando com folhas de estilo	197
Criando uma folha de estilo externa	0,198
Vinculando a folhas de estilo externas	200
Criando uma folha de estilo incorporada.	202
Aplicando estilos embutidos	204
A importância da localização.	206
Usando folhas de estilo específicas de mídia	208
Oferecendo folhas de estilo alternativas.	210
A inspiração de outros: CSS.	212
Capítulo 9 Definindo Seletores	213
Construindo Seletores	214
Selecionando Elementos por Nome	216
Selecionando Elementos por Classe ou ID.	218
Selecionando Elementos por Contexto	221
Selecionando Parte de um Elemento	227
Selecionando links com base em seu estado.	230
Selecionando Elementos Baseados em Atributos.	232
Especificando grupos de elementos	236
Combinando Seletores	238
Recapitulação dos Seletores.	240

Capítulo 10 Formatando texto com estilos 241

Escolhendo uma família de fontes	243
Especificando fontes alternativas	244
Criando Itálico	246
Aplicando formatação em negrito	248
Configurando o tamanho da fonte	250
Definir a altura da linha	255
Configurando todos os valores de fonte de uma só vez.	256
Configurando a cor	258
Alterando o plano de fundo do texto.	260
Controlando o espaçamento	264
Adicionando recuos	265
Configurando Propriedades de Espaço em Branco	266
Alinhando Texto.	268
Alterando a caixa do texto	270
Usando Versaletes	271
Decorando Texto.	272

Capítulo 11 Layout com estilos 275

Considerações ao iniciar um layout.	276
Estruturando suas páginas.	279
Estilizando elementos HTML5 em navegadores mais antigos.	286
Redefinindo ou normalizando estilos padrão	290
O modelo da caixa.	292
Alterando o plano de fundo	294
Configurando a altura ou largura de um elemento.	298
Configurando as margens ao redor de um elemento.	302
Adicionando preenchimento ao redor de um elemento.	304
Fazendo os Elementos Flutuarem	306
Controlando onde os elementos flutuam.	308
Definindo a fronteira.	311
Elementos de compensação no fluxo natural.	314
Posicionando Elementos Absolutamente.	316
Posicionando Elementos em 3D.	318
Determinando como tratar o transbordamento.	320
Alinhando Elementos Verticalmente	322
Alterando o Cursor	323
Exibindo e ocultando elementos	324

Capítulo 12	Folhas de estilo de dispositivos móveis para desktop	327
	Estratégias e considerações móveis.	328
	Compreendendo e implementando consultas de mídia.	333
	Construindo uma página que se adapta às consultas de mídia.	340
Capítulo 13	Trabalhando com fontes da Web	353
	O que é uma fonte da Web?	354
	Onde encontrar fontes da web.	356
	Baixando sua primeira fonte da web.	358
	Trabalhando com @font-face	360
	Estilizando fontes da Web e gerenciando o tamanho do arquivo.	365
Capítulo 14	Melhorias com CSS3	371
	Noções básicas sobre prefixos de fornecedores	373
	Uma rápida olhada na compatibilidade do navegador.	375
	Usando Polyfills para aprimoramento progressivo.	376
	Arredondando os cantos dos elementos.	378
	Adicionando sombras projetadas ao texto.	382
	Adicionando sombras projetadas a outros elementos	384
	Aplicando vários fundos.	388
	Usando fundos gradientes.	390
	Configurando a opacidade dos elementos.	394
Capítulo 15	Listas	397
	Criando listas ordenadas e não ordenadas.	398
	Escolhendo seus marcadores.	401
	Escolhendo por onde começar a numeração da lista.	403
	Usando marcadores personalizados	404
	Controlando onde os marcadores ficam pendurados	406
	Configurando todas as propriedades de estilo de lista de uma só vez.	407
	Estilizando listas aninhadas.	408
	Criando Listas de Descrição	412
Capítulo 16	Formulários	417
	Criando Formulários	419
	Processando Formulários	421
	Envio de dados de formulário por e-mail.	424
	Organizando os elementos do formulário.	426
	Criando caixas de texto.	428

Criando caixas de senha	431
Criação de caixas de e-mail, telefone e URL	432
Rotulagem de peças de formulário	434
Criando botões de rádio	436
Criando caixas de seleção	438
Criando caixas de seleção	440
Criando áreas de texto	441
Permitindo que visitantes façam upload de arquivos.	442
Criando campos ocultos	443
Criando um botão Enviar.	444
Usando uma imagem para enviar um formulário.	446
Desativando Elementos de Formulário	0,447
Novos recursos HTML5 e suporte ao navegador.	448
 Capítulo 17 Vídeo, Áudio e Outros Multimídias	 449
Plug-ins de terceiros e tornar-se nativo.	451
Formatos de arquivo de vídeo.	452
Adicionando um único vídeo à sua página da web.	453
Explorando atributos de vídeo.	454
Adicionando controles e reprodução automática ao seu vídeo.	455
Loop de um vídeo e especificação de uma imagem de pôster	457
Impedindo o pré-carregamento de um vídeo	458
Usando vídeo com múltiplas fontes.	459
Várias fontes de mídia e o elemento de origem.	460
Adicionando vídeo com substitutos de hyperlink.	461
Adicionando Vídeo com Flash Fallbacks	463
Fornecendo acessibilidade.	467
Adicionando formatos de arquivo de áudio	468
Adicionando um único arquivo de áudio à sua página da Web.	469
Adicionando um único arquivo de áudio com controles à sua página da Web.	470
Explorando atributos de áudio.	471
Adicionando controles e reprodução automática ao áudio em um loop.	472
Pré-carregando um arquivo de áudio	0,473
Fornecendo múltiplas fontes de áudio.	0,474
Adicionando áudio com substitutos de hyperlink.	0,475
Adicionando Áudio com Flash Fallbacks	0,476
Adicionando áudio com Flash e um substituto de hyperlink.	0,478
Obtendo arquivos multimídia.	480
Considerando o gerenciamento de direitos digitais (DRM)	0,481
Incorporação de animação Flash.	482

Incorporando vídeo do YouTube	484
Usando vídeo com Canvas	485
Acoplamento de vídeo com SVG	486
Recursos adicionais	487
Capítulo 18 Tabelas	489
Estruturando Tabelas	490
Abrangendo Colunas e Linhas	494
Capítulo 19 Trabalhando com Scripts	497
Carregando um script externo	499
Adicionando um script incorporado	502
Eventos JavaScript	503
Capítulo 20 Teste e depuração de páginas da Web	505
Tentando algumas técnicas de depuração	506
Verificando o que é fácil: Geral Verificando	508
o que é fácil: HTML Verificando o que	510
é fácil: CSS Validando seu código	512
Testando sua página	514
quando as imagens	516
ainda não aparecem travadas?	519
	520
Capítulo 21 Publicando suas páginas na web	521
Obtendo seu próprio nome de domínio	522
Encontrando um host para seu site	523
Transferindo arquivos para o servidor	525
Índice	529
Os capítulos bônus mencionados neste e-book estão disponíveis após o índice.	
Apêndice A	A1
Apêndice B	B1

Esta página foi intencionalmente deixada em branco

Introdução

Se você está apenas começando sua aventura na construção de sites ou já construiu alguns antes, mas deseja garantir que seu conhecimento esteja atualizado, você chegou em um momento muito interessante no setor.

Como codificamos e estilizamos as páginas, os navegadores nos quais visualizamos as páginas e os dispositivos nos quais visualizamos os navegadores todos avançaram substancialmente nos últimos anos. Antes limitados a navegar na Web a partir de nossos computadores desktop ou laptops, agora podemos levar a Web conosco em qualquer número de dispositivos: telefones, tablets e, sim, laptops e desktops, e muito mais.

E é como deveria ser, porque o A promessa da Web sempre foi a dissolução das fronteiras – o poder de compartilhar e acessar informações livremente de qualquer metrópole, comunidade rural,

ou em qualquer lugar intermediário, a partir de qualquer dispositivo habilitado para Web. Em suma, a promessa da Web reside na sua universalidade. E o alcance da Web continua a expandir-se à medida que a tecnologia chega a comunidades que antes estavam excluídas.

Somando-se à grandeza da Web está o fato de que qualquer pessoa é livre para criar e lançar um site. Este livro mostra como. É ideal para iniciantes sem conhecimento de HTML ou CSS que desejam começar a criar páginas da Web. Você encontrará instruções claras e fáceis de seguir que o guiarão passo a passo pelo processo de criação de páginas.

Por último, o livro é um guia útil para manter à mão. Você pode procurar tópicos no índice ou índice e consultar apenas aqueles assuntos sobre os quais precisa de mais informações.

HTML e CSS em resumo

Na raiz do sucesso da Web está uma linguagem de marcação simples, baseada em texto, fácil de aprender e que qualquer dispositivo com um navegador da Web básico pode ler: HTML. Cada página da Web requer pelo menos algum HTML; não seria uma página da Web sem ele.

Como você aprenderá com mais detalhes à medida que avança neste livro, o HTML é usado para definir o significado do seu conteúdo, e o CSS é usado para definir a aparência do seu conteúdo e da página da Web. Tanto as páginas HTML quanto os arquivos CSS (folhas de estilo) são arquivos de texto, o que os torna fáceis de editar. Você pode ver trechos de HTML e CSS em “Como este livro funciona”, próximo ao final desta introdução.

Você mergulhará no aprendizado de uma página HTML básica logo de cara no Capítulo 1, e começará a aprender como estilizar suas páginas com CSS no Capítulo 7. Consulte “O que este livro lhe ensinará” para uma visão geral de tudo. os capítulos e um resumo dos temas abordados.

O que é HTML5?

Ajuda saber alguns princípios básicos sobre as origens do HTML para entender o HTML5. HTML começou no início da década de 1990 como um pequeno documento que detalhava um punhado de elementos usados para construir páginas da Web. Muitos desses elementos serviam para descrever o conteúdo de páginas da Web, como títulos, parágrafos e listas. O número da versão do HTML aumentou à medida que a linguagem evoluiu com a introdução de outros elementos e ajustes em suas regras. A versão mais atual é HTML5.

HTML5 é uma evolução natural do anterior versões de HTML e se esforça para refletir as necessidades da Web atual e futura sites. Ele herda a grande maioria dos recursos de seus antecessores, o que significa que se você codificou o HTML antes do HTML5 chegar ao mercado

cena, você já conhece muito HTML5. Isso também significa que grande parte do HTML5

funciona em navegadores antigos e novos; ser compatível com versões anteriores é um princípio fundamental de design do HTML5 (consulte www.w3.org/TR/principios-de-design-html/).

HTML5 também adiciona uma série de novos recursos.

Muitos são simples, como elementos adicionais (**artigo, seção, figura** e muitos mais) que são usados para descrever o conteúdo. Outros são bastante complexos e auxiliam na criação de aplicativos Web poderosos

Você precisará ter um bom domínio da criação de páginas da Web antes de passar para os recursos mais complicados que o HTML5 oferece. HTML5 também introduz reprodução nativa de áudio e vídeo em suas páginas da Web, que o livro também aborda.

O que é CSS3?

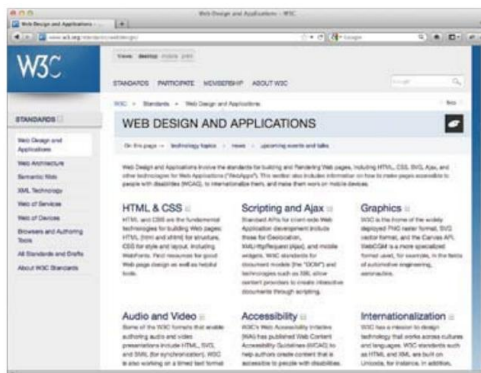
A primeira versão do CSS só existiu depois que o HTML já existia há alguns anos, tornando-se oficial em 1996. Assim como o HTML5 e sua relação com versões anteriores do HTML, o CSS3 é uma extensão natural das versões do CSS que o precederam. isto.

CSS3 é mais poderoso do que versões anteriores de CSS e introduz vários efeitos visuais, como sombras projetadas, sombras de texto, cantos arredondados e gradientes. (Consulte “O que este livro irá lhe ensinar” para obter detalhes sobre o que é abordado.)

Padrões e especificações da Web

Você deve estar se perguntando quem criou HTML e CSS em primeiro lugar e quem continua a desenvolvê-los. O mundo inteiro

O Web Consortium (W3C) – dirigido pelo inventor da Web e do HTML, Tim Berners-Lee – é a organização responsável por orientar o desenvolvimento de padrões da Web. Especificações (ou especificações, abreviadamente) são documentos que definem os parâmetros



R O site W3C é a principal fonte de especificações de padrões da Web do setor.

de linguagens como HTML e CSS. Em outras palavras, as especificações padronizam as regras. Acompanhe a atividade do W3C em www.w3.org A.

Por vários motivos, outra organização – o Web Hypertext Application Technology Working Group (WHATWG, encontrado em www.whatwg.org) – está desenvolvendo a especificação HTML5. O W3C incorpora o trabalho do WHATWG em sua versão oficial das especificações em andamento.

Com os padrões em vigor, podemos construir nossas páginas a partir do conjunto de regras acordado, e navegadores – como Chrome, Firefox, Internet Explorer (IE), Opera e Safari – podem ser criados para exibir nossas páginas com essas regras. em mente. (No geral, os navegadores implementam bem os padrões. Versões mais antigas do

O IE, especialmente o IE6, tem alguns problemas.

As especificações passam por vários estágios de desenvolvimento antes de serem consideradas finais, momento em que são apelidadas de Recomendação (www.w3.org/2005/10/Processo-20051014/tr).

Partes das especificações HTML5 e CSS3 ainda estão sendo finalizadas, mas isso não significa que você não possa usá-las. Leva apenas tempo (literalmente anos) para que o processo de padronização siga seu curso. Os navegadores começam a implementar os recursos de uma especificação muito antes de ela se tornar uma recomendação, porque isso informa o próprio processo de desenvolvimento da especificação. Portanto, os navegadores já incluem uma grande variedade de recursos em HTML5 e CSS3, embora ainda não sejam recomendações.

No geral, os recursos abordados neste livro estão bem arraigados em suas respectivas especificações, portanto o risco de serem alterados antes de se tornarem uma recomendação é mínimo. Os desenvolvedores já usam muitos recursos HTML5 e CSS3 há algum tempo. Então você pode.

Progressivo Aprimoramento: Uma melhor prática

Comecei a introdução falando da universalidade da Web – a noção de que a informação na Web deve ser acessível.

apto para todos. O aprimoramento progressivo ajuda você a criar sites com a universalidade em mente. Não é uma linguagem, mas sim uma abordagem para a construção de sites que Steve Champeon criou em 2003 (http://en.wikipedia.org/wiki/Aprimoramento_progressivo).

A ideia é simples, mas poderosa: comece seu site com conteúdo e comportamento HTML que é acessível a todos os visitantes A. Para o mesmo página, adicione seu design com CSS B e adicione comportamento adicional com JavaScript, normalmente carregando-os de arquivos externos (você aprenderá como fazer isso).

O resultado é que dispositivos e navegadores capaz de acessar páginas básicas terá a experiência padrão simplificada; dispositivos e navegadores capazes de visualizar sites mais robustos verão a versão aprimorada.

A experiência no seu site não precisa ser a mesma para todos, desde que o seu conteúdo seja acessível. Em essência, a ideia por trás do aprimoramento progressivo é que todos ganham.



A Uma página HTML básica sem CSS personalizado aplicado a ela. Esta página pode não parecer ótima, mas as informações são acessíveis – e é isso que importa. Mesmo navegadores próximos ao início da Web, há mais de 20 anos, podem exibir esta página; o mesmo acontece com os telefones celulares mais antigos com navegadores da Web. E os leitores de tela, softwares que leem páginas da Web em voz alta para visitantes com deficiência visual, poderão navegar facilmente.



B A mesma página visualizada em um navegador que suporta CSS. São as mesmas informações, apenas apresentadas de forma diferente. Os usuários com dispositivos e navegadores mais capazes obtêm uma experiência aprimorada ao visitar a página.

Este livro ensina como construir sites progressivamente aprimorados, mesmo que nem sempre indique isso explicitamente ao fazê-lo. É um resultado natural das melhores práticas transmitidas ao longo do livro.

Contudo, os Capítulos 12 e 14 abordam de frente o aprimoramento progressivo. Dê uma olhada neles se estiver interessado em ver como o princípio do aprimoramento progressivo ajuda a construir um site que adapta seu layout com base no tamanho da tela do dispositivo e nos recursos do navegador, ou como navegadores mais antigos exibirão designs simplificados enquanto modernos os navegadores exibirão aqueles aprimorados com efeitos CSS3.

O aprimoramento progressivo é uma prática recomendada fundamental que está no centro dos canteiros de obras para todos.

Esse livro é para você?

Este livro não pressupõe nenhum conhecimento prévio de construção de sites. Então, nesse sentido, é para o iniciante absoluto. Você aprenderá HTML e CSS desde o início. Ao fazer isso, você também aprenderá sobre os recursos novos em HTML5 e CSS3, com ênfase naqueles que designers e desenvolvedores estão usando hoje em seu trabalho diário.

Mas mesmo se você estiver familiarizado com HTML e CSS, ainda terá que aprender com este livro, especialmente se quiser se atualizar sobre muitas das novidades em HTML5, CSS3 e práticas recomendadas.

O que este livro vai te ensinar

Adicionamos aproximadamente 125 páginas a este livro desde a edição anterior para trazer a você o máximo de material possível. (A primeira edição do livro, publicada em 1996, tinha um total de 176 páginas.)

Também fizemos atualizações substanciais (ou reescritas completas) em quase todas as páginas anteriores. Em suma, esta Sétima Edição representa uma revisão importante.

Os capítulos estão organizados assim:

n Os capítulos 1 a 6 e 15 a 18 cobrem os princípios de criação de páginas HTML e a variedade de elementos HTML à sua disposição, demonstrando claramente quando e como usar cada um deles.

n Os capítulos 7 a 14 se aprofundam no CSS, desde a criação de sua primeira regra de estilo até a aplicação de efeitos visuais aprimorados com CSS3.

O Capítulo 19 mostra como adicionar JavaScript pré-escrito às suas páginas.

n O Capítulo 20 explica como testar e depure suas páginas antes de colocá-las na Web.

n O Capítulo 21 explica como proteger seu próprio nome de domínio e depois publicar seu site na Web para que todos possam ver.

Expandindo isso, alguns dos tópicos incluem:

n Criar, salvar e editar HTML e Arquivos CSS.

n O que significa escrever HTML semântico e por que é importante.

n Como separar o conteúdo da sua página (ou seja, seu HTML) de sua apresentação (ou seja, seu CSS) – um aspecto fundamental do aprimoramento progressivo.

n Estruture seu conteúdo de maneira significativa usando elementos HTML que já existem há anos e que são novos no HTML5.

n Melhorar a acessibilidade do seu site com funções de referência ARIA e outras boas práticas de codificação.

n Adicionar imagens às suas páginas e otimizá-las para a Web.

n Vincular de uma página da Web para outra página ou de uma parte de uma página para outra parte.

n Estilizar o texto (tamanho, cor, negrito, itálico, e mais); adicionar cores e imagens de fundo; e implementar um layout fluido com várias colunas que pode encolher e expandir para acomodar diferentes tamanhos de tela.

- n Aproveitando novos seletores em CSS3
 - que permitem que você direcione seus estilos de uma forma mais ampla do que era possível anteriormente.
- n Aprender suas opções para abordar visitantes em dispositivos móveis.
- n Construindo um único site para todos os usuários—estejam eles usando um telefone celular, tablet, laptop, computador desktop ou outro dispositivo habilitado para Web - com base em muitos dos princípios do web design responsivo, alguns dos quais aproveitam as consultas de mídia CSS3.
- n Adicionando fontes da Web personalizadas às suas páginas com **@font-face**.
- n Usando efeitos CSS3, como opacidade, transparência alfa de fundo, gradientes, cantos arredondados, sombras projetadas, sombras dentro de elementos, sombras de texto e múltiplas imagens de fundo.
- n Construir formulários para solicitar informações de seus visitantes, incluindo o uso de alguns dos novos tipos de entrada de formulário em HTML5.
- n Incluir mídia em suas páginas com elementos **de áudio e vídeo** HTML5 .

E mais.

Esses tópicos são complementados por dezenas de exemplos de código que mostram como implementar os recursos com base nas práticas recomendadas do setor.

O que este livro não vai ensinar

Infelizmente, mesmo depois de adicionar tantas páginas desde a edição anterior, há muito o que falar quando se trata de HTML e

CSS que tivemos que deixar de fora alguns tópicos.

Com algumas exceções, limitamo-nos a omitir itens que você teria menos ocasiões de usar, que ainda estão sujeitos a alterações, que não possuem amplo suporte ao navegador, que exigem conhecimento de JavaScript ou que são assuntos avançados.

Alguns dos tópicos não abordados incluem:

n Os detalhes HTML5 , **resumo, menu, comando** e elementos **keygen** .

n O elemento **canvas** HTML5 , que permite desenhar gráficos (e até criar jogos) com JavaScript.

n As APIs HTML5 e outros recursos avançados recursos que exigem conhecimento de JavaScript ou que não estão diretamente relacionados ao novo HTML5 semântico elementos.

sprites CSS . Essa técnica envolve combinar mais de uma imagem em uma única imagem, o que é muito útil para minimizar o número de ativos que suas páginas precisam carregar. Consulte www.bruceontheloose.com/sprites/ para obter mais informações.

n Substituição de imagem CSS. Essas técnicas costumam ser combinadas com sprites CSS. Consulte www.bruceontheloose.com/ir/ para obter mais informações.

n Transformações CSS3, animações e transições.

n Novos módulos de layout do CSS3.

Como este livro funciona

Quase todas as seções do livro contêm exemplos práticos de código que demonstram o uso no mundo real (**A** e **B**). Normalmente, eles são acompanhados de capturas de tela que mostram os resultados do código quando você visualiza a página da Web em um navegador C.

A maioria das capturas de tela são da versão mais recente do Firefox disponível na época. No entanto, isso não implica uma recomendação do Firefox em detrimento de qualquer outro navegador. Os exemplos de código serão muito semelhantes em qualquer uma das versões mais recentes do Chrome, Internet Explorer, Opera ou Safari. Como você aprenderá no Capítulo 20, você deve testar suas páginas em uma ampla variedade de navegadores antes de colocá-las na Web,

porque não há como saber quais navegadores seus visitantes usarão.

O código e as capturas de tela são acompanhados por descrições dos elementos HTML ou propriedades CSS em questão, tanto para dar contexto aos exemplos quanto para aumentar sua compreensão deles.

Em muitos casos, você pode descobrir que as descrições e exemplos de código são suficientes para começar a usar os recursos HTML e CSS. Mas se você precisar de orientação explícita sobre como usá-los, instruções passo a passo serão sempre fornecidas.

Finalmente, a maioria das seções contém dicas que transmitem informações adicionais de uso, práticas recomendadas, referências a partes relacionadas do livro, links para recursos relevantes e muito mais.

R Você encontrará um trecho de código HTML em muitas páginas, com as seções pertinentes destacadas. Uma reticência (...) representa código ou conteúdo adicional que foi omitido por questões de brevidade. Frequentemente, a parte omitida é mostrada em uma figura de código diferente.

```
...
<corpo>
<header role="banner">
  ...
    <nav role="navegação">
      <ul class="nav">
        <li><a href="/" class="current">página inicial</a></li>
        <li><a href="/about/">sobre</a></li>
        <li><a href="/resources/">recursos</a></li>
        <li><a href="/archives/">arquivos</a></li>
      </ul>
    </nenhum>
  ...
</header>

...

</body>
</html>
```

B Se o código CSS for relevante para o exemplo, ele será mostrado em uma caixa própria, com as seções pertinentes destacadas.

```
/* Navegação no site */
.nome em {
    flutuar: esquerda;
    tamanho da fonte: .75em; /* torna
    os marcadores ¶ menores */
}

.nav li a {
    tamanho da fonte: 1,5em;
}

.nav li:primeiro filho {
    estilo de lista: nenhum;
    preenchimento à esquerda: 0;
}
```



C Capturas de tela de um ou mais navegadores demonstram como o código afeta a página.

Convenções utilizadas neste livro

O livro usa as seguintes convenções:

n A palavra HTML é abrangente, representando a linguagem em geral.
HTML5 é usado quando se refere a essa versão específica do HTML, como ao discutir um recurso que é novo no HTML5 e não existe nas versões anteriores do HTML. A mesma abordagem se aplica ao uso dos termos CSS (geral) e CSS3 (específico para CSS3).

n O texto ou código que é um espaço reservado para um valor que você mesmo criaria está em itálico. A maioria dos espaços reservados aparece nas instruções passo a passo. Por exemplo, “Ou digite **#rrggbb**, onde **rrggbb** é a representação hexadecimal da cor”.

n O código que você realmente deve digitar ou que representa código HTML ou CSS aparece nesta **fonte**.

n Uma seta (¶) em uma figura de código indica uma continuação da linha anterior — a linha foi quebrada para caber na coluna B do livro. A seta não faz parte do código em si, portanto não é algo que você digitaria. Em vez disso, digite a linha continuamente, como se ela não tivesse passado para outra linha.

n A primeira ocorrência de uma palavra é itálico definido quando é definido.

n IE é frequentemente usado como uma abreviatura popular de Internet Explorer. Por exemplo, IE9 é sinônimo de Internet Explorer 9.

n Sempre que um sinal de mais (+) segue um número da versão do navegador, significa a versão listada mais as versões subsequentes. Por exemplo, Firefox 8+ refere-se ao Firefox 8.0 e todas as versões posteriores.

Site complementar

O site do livro, em www.bruceontheloose.com/htmlcss/, contém o índice, todos os exemplos de código completos apresentados no livro (além de alguns adicionais que não caberiam), links para recursos citados no livro (também como adicionais), informações sobre referências usadas durante a escrita, uma lista de erratas e muito mais.

O site também inclui seções de referência (Apêndices A e B) que não tivemos espaço para incluir no livro. Estes são

útil para procurar rapidamente elementos e atributos HTML ou propriedades e valores CSS. (Eles também contêm algumas informações não abordadas no livro.)

Você pode encontrar os exemplos de código em www.bruceontheloose.com/htmlcss/examples/. Você pode navegar por eles a partir daí ou abaixo carregue-os em seu computador - todos os arquivos HTML e CSS estão à sua disposição.

Em alguns casos, incluí comentários adicionais no código para explicar mais sobre o que ele faz ou como usá-lo. A

Alguns exemplos de código do livro estão truncados por questões de espaço, mas as versões completas estão no site do livro. Sinta-se à vontade para usar o código como desejar, modificando-o conforme necessário para seus próprios projetos.

Os URLs de algumas das principais páginas do site do livro são os seguintes:

Página inicial : www.bruceontheloose.com/htmlcss/

n Exemplos de código: www.bruceontheloose.com/htmlcss/examples/

n Apêndice A: Referência HTML:
www.bruceontheloose.com/ref/html/

n Apêndice B: Propriedades e valores CSS:
www.bruceontheloose.com/ref/css/

Espero que você ache o site útil.

Treinamento em vídeo

Os guias visuais de início rápido agora estão uniformes mais visual: Com base no sucesso dos livros Visual QuickStart Guide mais vendidos, o Peachpit agora oferece Vídeo QuickStarts. Como complemento deste livro, Peachpit oferece mais de uma hora de vídeos curtos baseados em tarefas que ajudarão você a dominar os principais recursos e técnicas do HTML5; em vez de apenas ler sobre como usar HTML5, você pode observá-lo em ação. É uma ótima maneira de aprender todos os conceitos básicos e alguns dos mais novos ou

recursos mais complexos do HTML5. Faça login no site do Peachpit em www.peachpit.com/ registre-se para registrar seu livro e você encontrará uma amostra gratuita de streaming; adquirir o restante do material é rápido e fácil.

Construção de página da web

Blocos

Embora as páginas Web se tenham tornado cada vez mais complexas, a sua estrutura subjacente permanece notavelmente simples. A primeira coisa que você deve saber é que é impossível criar uma página Web sem HTML. Como você aprenderá, o HTML hospeda seu conteúdo e descreve seu significado. Por sua vez, os navegadores da Web renderizam o conteúdo em HTML para os usuários.

Uma página da Web é composta principalmente de três componentes:

- n Conteúdo de texto: O texto simples que aparece na página para informar os visitantes sobre sua empresa, férias em família, produtos ou qualquer que seja o foco de sua página.

- n Referências a outros arquivos: Estes carregam itens como imagens, áudio, vídeo e arquivos SVG, além de links para outras páginas e recursos HTML, bem como para folhas de estilo (que controlam o layout da sua página) e arquivos JavaScript (que adicionam comportamento à sua página).

- n Marcação: Os elementos HTML que descreva o conteúdo do seu texto e faça as referências funcionarem. (O m em HTML significa marcação.)

Neste capítulo

Uma página HTML básica	3
HTML semântico: marcação com significado	6
Marcação: Elementos, Atributos e Valores	13
Conteúdo de texto de uma página da web	16
Links, imagens e outros itens não textuais	17
Conteúdo	17
Nomes de arquivos	19
URLs	20
Principais conclusões	24

É importante observar que cada um desses componentes de uma página Web é composto exclusivamente de texto. Isso significa que as páginas são salvas em formato somente texto e podem ser visualizadas em praticamente qualquer navegador e em qualquer plataforma, seja desktop, celular, tablet ou outro. Garante a universalidade da Web. Uma página pode parecer diferente quando visualizada em um dispositivo versus

outro, mas tudo bem. O importante como primeiro passo é tornar o conteúdo acessível a todos os usuários, e o HTML permite isso.

Além dos três componentes dos quais uma página da Web é composta principalmente, uma página também inclui HTML que fornece informações sobre a própria página, a maioria das quais seus usuários não veem explicitamente e que se destina principalmente a navegadores e pesquisas. motores. Isto pode incluir informações

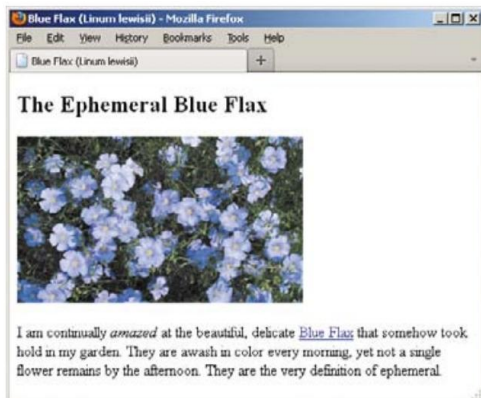
sobre o idioma principal do conteúdo (inglês, francês e assim por diante), codificação de caracteres (normalmente UTF-8) e muito mais.

Este capítulo guiará você por uma página HTML básica, discutirá algumas práticas recomendadas e explicará cada um dos três componentes importantes.

Nota: Conforme mencionado na introdução, utilizo HTML para me referir à linguagem em geral. Nos casos em que estou destacando características especiais exclusivas de uma versão do idioma, usarei o nome individual. Por exemplo, "HTML5

introduz vários elementos novos e redefine ou elimina outros que existiam anteriormente no HTML 4 e XHTML 1.0."

Para mais detalhes, consulte "Como este livro funciona" na introdução.



A Uma renderização padrão típica da página.

Embora isso mostre a página no Firefox, a página é exibida de forma semelhante em outros navegadores.

Uma página HTML básica

Vamos dar uma olhada em uma página HTML básica para contextualizar o que vem a seguir neste capítulo e além. A Figura **A** ilustra como um navegador de desktop normalmente renderiza o código HTML em B. Você aprenderá algumas noções básicas sobre o código B, mas não se preocupe se não entender tudo agora. Isso é apenas para lhe dar uma ideia do HTML. Você tem o resto do livro para

aprenda mais sobre isso.

Você provavelmente pode adivinhar um pouco do que está acontecendo no código, especialmente na seção **body**. Primeiro vamos dar uma olhada na parte anterior ao **corpo**.

B Aqui está o código para uma página HTML básica. Destaquei as partes HTML para que você possa distingui-las do conteúdo de texto da página. Conforme demonstrado em A, o HTML que envolve o conteúdo do texto não aparece quando você visualiza a página em um navegador. Mas, como você aprenderá, a marcação é essencial porque descreve o significado do conteúdo. Observe também que cada linha é separada por um retorno de carro. Isso não é obrigatório e não afeta a renderização da página.

```
<!DOCTYPEhtml>
<html lang="pt">
<cabeca>
  <meta charset="utf-8" />
  <title> Linho Azul (Linum lewisii)</title>
</head>
<corpo>
  <artigo>
    <h1>O Efêmero Linho Azul</h1>

    <p>Fico continuamente <em>surpreso</em> com a beleza e a delicadeza <a href="http://
    en.wikipedia.org/wiki/Linum_lewisii" rel="external" title="Saiba mais sobre o Linho
    Azul"> Linho Azul</a> que de alguma forma tomou conta do meu jardim. Eles ficam cheios
    de cores todas as manhãs, no entanto, nem uma única flor permanece à tarde. Elas são a
    própria definição de efêmero.</p>
  </artigo>
</body>
</html>
```

Tudo acima da tag inicial **<body>** são as informações de instrução para navegadores e motores de busca mencionados anteriormente C. Cada página começa com a declaração DOCTYPE, que informa ao navegador a versão HTML da página.

Você deve sempre usar o DOCTYPE do HTML5, que é **<!DOCTYPE html>**. A caixa do texto não importa, mas é mais comum usar DOCTYPE em letras maiúsculas.

Independentemente disso, inclua sempre o DOCTYPE em suas páginas. (Veja a barra lateral "DOCTYPE aprimorado do HTML5" no Capítulo 3 para obter mais informações.)

Os bits que começam em **<!DOCTYPE html>** e continuam até **</head>** são invisíveis para os usuários com uma exceção: o texto entre **<title>** e **</title>**—**Blue Flax** (Linum lewisii)—aparece como o título no final. bem no topo da janela do navegador e em um navegador

guia B. Além disso, normalmente é o padrão nome de um marcador ou favorito do navegador e é uma informação valiosa para pesquisa motores. O Capítulo 3 explica o que fazem as outras partes do segmento superior de uma página.

C O texto do elemento **título** é a única parte da área superior de um documento HTML que o usuário vê. O resto são informações sobre a página para navegadores e mecanismos de busca.

```
<!DOCTYPEhtml>
<html lang="pt">
  <cabeca>
    <meta charset="utf-8" />
    <title>O Efêmero Linho Azul ÿ
      (Linum lewisii)</title>
  </head>
```

D O conteúdo de uma página existe entre as tags inicial e final do elemento **body**. O documento termina em **</html>**.

```
<!DOCTYPEhtml>
<html lang="pt">
... [cabeçalho do documento] ...
<corpo>
  <artigo>
    <h1>O Efêmero Linho Azul</h1>

    <p>Fico continuamente
    <em>maravilha</em> &#xA0; com a
    beleza e a delicadeza &#xA0; <a href="http://
    en.wikipedia.org/ &#xA0; wiki/Linum_lewisii"
    rel="external" &#xA0; title=" Saiba mais sobre
    o Linho Azul"> &#xA0; Linho Azul</a> que
    de alguma forma tomou conta do meu
    jardim. Elas ficam inundadas de &#xA0;
    cores todas as manhãs, mas nem
    uma &#xA0; única flor permanece à &#xA0;
    tarde. Eles são a própria &#xA0; definição de efêmero.</p>
  </artigo>
</body>
</html>
```

Enquanto isso, o conteúdo da sua página – ou seja, o que é visível para os usuários – fica entre **<body>** e **</body>**. Finalmente, o **</html>** A tag end sinaliza o final da página D.

O recuo do código não tem absolutamente nenhuma influência sobre se o código é HTML válido. Também não afeta a forma como o conteúdo é exibida no navegador (o elemento **pre**, sobre o qual você aprenderá no Capítulo 4, é a única exceção). No entanto, é costume recuar o código aninhado em um elemento pai para facilitar a compreensão da hierarquia dos elementos à medida que você lê o código. Você aprenderá mais sobre pais e filhos posteriormente neste capítulo. Você também aprenderá mais detalhadamente sobre a renderização padrão do navegador.

Primeiro, vamos discutir o que significa escrever HTML semântico e por que ele é a base de um site eficaz.

HTML semântico:

Marcação com significado

HTML é um sistema inteligente de inclusão de informações sobre o conteúdo de um texto documento. Essa informação, chamada de marcação, descreve o significado do conteúdo, ou seja, a semântica. Você já viu alguns exemplos em nossa página HTML básica, como o elemento **p** que marca o conteúdo do parágrafo.

HTML não define como o conteúdo deverá aparecer em um navegador; esse é o papel do CSS (Cascading Style Sheets). O HTML5 enfatiza essa distinção mais do que qualquer versão anterior do HTML. Está no cerne da linguagem.

Você deve estar se perguntando por que, se for esse o caso, algum texto na página HTML básica parece maior que outro texto ou está em negrito ou itálico B.

Ótima pergunta. A razão é que todo navegador da Web possui um arquivo CSS integrado (uma folha de estilo) que determina como cada elemento HTML é exibido por padrão, a menos que você crie o seu próprio para sobrescrevê-lo. A apresentação padrão varia um pouco de navegador para navegador, mas no geral é bastante consistente. Mais importante ainda, a estrutura subjacente e o significado do conteúdo, conforme definido pelo seu HTML, permanecem os mesmos.

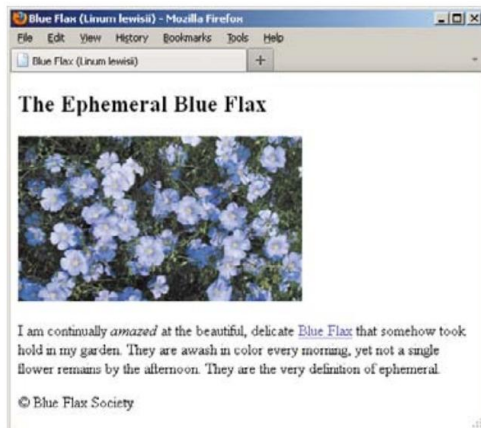
A O conteúdo da nossa página básica mais um segundo parágrafo adicionado no final. Os elementos HTML não determinam como o conteúdo deve aparecer, apenas o que significam. Em vez disso, a folha de estilo integrada de cada navegador determina como o conteúdo é exibido por padrãoB.

```
...
<corpo>
  <artigo>
    <h1>O Efêmero Linho Azul</h1>

    <p>Fico continuamente <em>maravilha</
em> y com a beleza e a delicadeza
y <a href="http://en.wikipedia.org/ y wiki/
Linum_lewisii" rel="external" y title=" Saiba
mais sobre o Linho Azul"> y Linho Azul</a>
que de alguma forma tomou conta do
meu jardim. Elas ficam inundadas de y cores
todas as manhãs, mas nem uma y única flor
permanece à y tarde. Eles são a
própria y definição de efêmero.</p>

    <p><small>&copy; Sociedade do Linho
Azul. y </small></p>
  </artigo>
</body>
</html>
```



■ A folha de estilo padrão de um navegador renderiza títulos (elementos h1–h6) de maneira diferente do texto normal, coloca o texto **em** itálico e colore e sublinha os links. Além disso, alguns elementos começam em suas próprias linhas (**h1** e **p**, por exemplo) e outros são exibidos no conteúdo circundante (como **a** e **em**). Este exemplo inclui um segundo parágrafo (o aviso de direitos autorais) para deixar claro que cada parágrafo ocupa sua própria linha. É simples substituir qualquer uma ou todas essas regras de apresentação por suas próprias folhas de estilo.

Nível de bloco, inline e HTML5

Como você pode ver, alguns elementos HTML (por exemplo, o **artigo**, **h1** e **p**) são exibidos em sua própria linha, como um parágrafo em um livro, enquanto outros (por exemplo, o **a** e **em**) são renderizados na mesma linha que outro conteúdo. Novamente, isso é uma função das regras de estilo padrão do navegador, não dos próprios elementos HTML. Permita-me elaborar avaliar. Antes do HTML5, a maioria dos elementos eram categorizados como nível de bloco (aqueles exibidos em sua própria linha) ou inline (aqueles exibidos dentro de uma linha de texto). O HTML5 elimina esses termos porque eles associam elementos à apresentação, o que você aprendeu não é a função do HTML.

Em vez disso, de modo geral, os elementos que foram anteriormente apelidados de in-line são categorizados em HTML5 como conteúdo de fraseado. isto é, elementos e o texto contido neles que aparecem principalmente em um parágrafo. (O Capítulo 4 concentra-se quase exclusivamente na formulação de conteúdo. Veja a lista completa em <http://dev.w3.org/html5/spec-author-view/content-models.html#phrasing-content-0>.)

Os antigos elementos de nível de bloco também caem agora em novas categorias HTML5 que focam em sua semântica. Muitos destes elementos constituem os principais blocos estruturais e títulos de seu conteúdo (aprofunde-se no Capítulo 3 para saber mais sobre como seccionar conteúdo e encaminhar elementos de conteúdo).

Com tudo isso dito, os navegadores não mudaram as regras de exibição padrão para esses elementos, nem deveriam. Afinal, você não gostaria, digamos, que os dois parágrafos (os elementos **p**) se chocassem, ou que o texto **em** ("espantado") quebrasse a frase aparecendo em sua própria linha (**em** é o elemento você usa para adicionar ênfase).

Geralmente, títulos, parágrafos e elementos estruturais, como **artigos**, são exibidos em sua própria linha e o conteúdo de frases é exibido na mesma linha do conteúdo circundante. E mesmo que o HTML5 não use mais os termos nível de bloco e inline, ajuda saber o que eles significam.

É comum que os tutoriais os utilizem desde eles estavam enraizados no vernáculo HTML antes do HTML5. Eu poderia usá-los ocasionalmente no livro para transmitir rapidamente se um elemento ocupa sua própria linha ou compartilha uma linha por padrão.

Abordaremos CSS em detalhes em capítulos posteriores, mas por enquanto saiba que uma folha de estilo, como uma página HTML, é apenas texto, então você pode criar uma com o mesmo editor de texto que seu HTML.

Foco do HTML5 na Semântica

O HTML5 enfatiza a semântica do HTML, deixando todo o estilo visual para o CSS. Esse nem sempre foi o caso das versões anteriores do HTML.

Não existia um meio adequado para estilizar páginas nos primeiros anos da Web; O HTML já tinha alguns anos quando o CSS1 foi formalmente introduzido em dezembro de 1996. Para preencher essa lacuna, entretanto, o HTML incluiu um punhado de elementos de apresentação cujo objetivo era permitir o estilo básico do texto, como torná-lo em negrito, em itálico ou em um tamanho diferente do texto ao redor.

Esses elementos serviram ao seu propósito na época, mas caíram em desuso à medida que as melhores práticas evoluíram para a Web.

desenvolvimento. Central para esse pensamento era – e ainda é – a noção de que HTML serve apenas para descrever o significado do conteúdo, não sua exibição.

Os elementos HTML de apresentação quebraram essa prática recomendada. Como tal, o HTML 4 rejeitou seu uso, recomendando que os autores usassem CSS para estilizar texto e outros elementos da página.

HTML5 vai além; elimina alguns elementos de apresentação e redefine outros para que tenham apenas valor semântico em vez de ditar a apresentação.

O **pequeno** elemento é um exemplo.

Inicialmente, pretendia-se tornar o texto menor que o texto normal. No entanto, em HTML5, **pequeno** representa letras miúdas, como uma isenção de responsabilidade legal. Você pode usar CSS para torná-lo o maior texto da página, se desejar, mas isso não mudará o significado do seu **pequeno** conteúdo.

Enquanto isso, a antiga contraparte **do pequeno**, o **grande** elemento, não existe no HTML5. Existem outros exemplos também, que você aprenderá à medida que avança no livro.

HTML5 também define novos elementos, como **cabeçalho**, **rodapé**, **navegação**, **artigo**, **seção** e muitos mais que enriquecem a semântica do seu conteúdo. Você aprenderá sobre isso mais tarde também.

No entanto, quer você use um elemento HTML que existe desde o início do

linguagem ou algo novo em HTML5, seu objetivo deve ser o mesmo: escolher os elementos que melhor descrevem o significado do seu conteúdo, independentemente de sua apresentação.

C O **corpo** da nossa página básica, que contém o **artigo**, **h1**, **img**, **p**, **em** e **elementos** a para descrever o significado do conteúdo. Todo o conteúdo está aninhado no **artigo**.

```
<corpo>
  <artigo>

    <h1>O Efêmero Linho Azul</h1>

    <p>Fico continuamente <em>maravilha</em>
y com a beleza e a delicadeza y <a
href="http://en.wikipedia.org/ y wiki/
Linum _lewisii" rel="external" y title=" Saiba mais
sobre o Linho Azul"> y Linho Azul</a> que de
alguma forma tomou conta do meu jardim.

Elas ficam inundadas de y cores todas as
manhãs, mas y nem uma única flor
permanece y à tarde. Eles são a própria y
definição de efêmero.</p>

  </artigo>
</body>
```

D Os títulos são elementos críticos na definição do esboço de uma página. Eles tornam uma página mais acessível aos usuários de leitores de tela e os mecanismos de pesquisa os utilizam para determinar o foco de uma página.

```
<h1>O Efêmero Linho Azul</h1>
```

E É fácil adicionar uma imagem a uma página com **img**.
Conforme definido pelo atributo **alt** , "Blue Flax (Linum lewisii)" é exibido se nossa imagem não for.

```

```

A Semântica do Nosso

Página HTML básica

Agora que você conhece a função do HTML, vamos examinar um pouco mais profundamente o processo de pensamento por trás da marcação do conteúdo de amostra. Como você verá, não há mágica em escrever HTML semântico. É principalmente bom senso quando você estiver familiarizado com os elementos à sua disposição. Vamos revisitar o **corpo** da nossa página básica para conhecer alguns dos elementos HTML C mais usados.

Todo o conteúdo está contido em um **artigo** elemento C. Resumindo, **o artigo** define um conteúdo distinto. O elemento **article** é a escolha apropriada para cercar o conteúdo da nossa página básica, mas não necessariamente para todas as páginas que você escreverá. Você aprenderá mais sobre quando usar **o artigo** no Capítulo 3.

O próximo é o título D. O HTML fornece seis níveis de título, **h1–h6**, sendo **h1** o mais importante. Um **h2** é um subtítulo de um **h1**, um **h3** é um subtítulo de um **h2** e assim por diante, assim como quando você digita um documento com vários títulos em um processador de texto.

Toda página HTML deveria ter um **h1** (ou mais, dependendo do seu conteúdo), então marcar nosso título com **h1** foi a escolha óbvia. Os elementos de título **h1–h6** são abordados mais detalhadamente no Capítulo 3.

Em seguida, você tem uma imagem E. O elemento **img** é a escolha principal para exibir uma imagem, portanto, novamente, não houve debate sobre qual elemento era apropriado. O atributo **alt** fornece texto que será exibido se a imagem não carregar ou se a página for visualizada em um navegador somente texto. Você aprenderá mais sobre imagens no Capítulo 5.

O parágrafo é marcado com – surpresa – o elemento **p**. Assim como nos materiais impressos, um parágrafo pode conter uma única frase ou várias frases.

Se nossa página precisasse de outro parágrafo, você simplesmente adicionaria outro elemento **p** após o primeiro.

Existem dois elementos aninhados em nosso parágrafo que define o significado de pedaços de texto: **em** e **F**. Esses são exemplos dos numerosos elementos de conteúdo de frases que o HTML5 fornece, a maioria dos quais melhora a semântica do texto do parágrafo.

Conforme mencionado, esses, juntamente com **p**, são discutidos no Capítulo 4.

O elemento **em** significa “ênfase no estresse”.

No caso da nossa página, destaca-se o espanto que as flores provocaram em F.

Lembre-se disso porque o HTML descreve

o significado do conteúdo, **em** dita a ênfase semântica, e não visual, embora seja comum renderizar o texto em itálico.

Finalmente, a página básica define um link para outra página com o elemento **a** (“âncora”), que é o elemento mais poderoso em todo o HTML porque torna o

Web, a Web: vincula uma página a outra página ou recurso e vincula uma parte de uma página a outra parte de uma página (a mesma página ou uma diferente). No exemplo, significa que o texto “Blue Flax” é um link para uma página da Wikipedia G.

F O elemento **p** pode conter outros elementos que definem a semântica das frases dentro de um parágrafo. Os elementos **em** e **a** são dois exemplos.

```
<p> Fico continuamente <em>surpreso</em> com &#x201c;a linda e delicada <a href="http://en.wikipedia.org/wiki/Linum_lewisii" rel="external" title="Saiba mais sobre Blue Flax">Blue Flax</a> que de alguma forma &#x201c;tomou conta do meu jardim. Eles est&#x201c;o inundados de &#x201c;cores em todas as cores manh&#x201c;a, mas nem uma &#x201c;flor permanece &#x201c;à tarde. Elas s&#x201c;o a pr&#x201c;pria defini&#x201c;o de ef&#x201c;mero.</p>
```

G Este elemento define um link para a página da Wikipedia sobre Blue Flax. O atributo opcional **rel** aumenta a semântica, indicando que o link aponta para outro site. O link funciona sem ele, no entanto. O atributo opcional **title** aprimora a semântica de **a**, fornecendo informações sobre a página vinculada. Ele aparece no navegador quando um usuário passa o mouse sobre o link.

```
<a href="http://en.wikipedia.org/wiki/Linum_lewisii" rel="external" title="Saiba mais sobre o Linho Azul"> Linho Azul</a>
```

Muito fácil, certo? Depois de aprender mais sobre os elementos HTML disponíveis

para você, escolher os corretos para o seu conteúdo geralmente é uma tarefa simples.

Ocasionalmente, você encontrará um conteúdo que poderia ser razoavelmente marcado de mais de uma maneira, e tudo bem.

Nem sempre existe uma maneira certa e errada, apenas na maioria das vezes.

Por último, o HTML5 não tenta fornecer um elemento para cada tipo de conteúdo imaginável, porque a linguagem se tornaria desajeitada. Em vez disso, adota uma postura prática e real, definindo elementos que cobrem a grande maioria dos casos.

Parte da beleza do HTML é que é simples para qualquer pessoa aprender o básico, construir algumas páginas e aumentar seu conhecimento a partir daí. Portanto, embora existam aproximadamente 100 elementos HTML, não se deixe assustar por esse número. Há alguns itens básicos que você usará repetidamente, enquanto os restantes são reservados para casos menos comuns. Você já aprendeu o básico de vários elementos comuns, então está no caminho certo.

Por que a semântica é importante

Agora que você conhece a importância do HTML semântico e o viu em ação, precisa saber os motivos pelos quais ele é importante.

Aqui estão alguns dos motivos mais importantes (esta não é uma lista exaustiva), alguns dos quais já mencionamos:

n Melhor acessibilidade e interoperabilidade

habilidade (o conteúdo está disponível para tecnologias assistivas para visitantes com deficiência e para navegadores em desktops, celulares, tablets e outros dispositivos)

n Otimização aprimorada do mecanismo de pesquisa (ESSE)

n (normalmente) código mais leve e páginas mais rápidas

n Manutenção e estilo de código mais fáceis

Se você não está familiarizado com acessibilidade, é prática disponibilizar seu conteúdo para todos os usuários, independentemente de suas capacidades (consulte www.w3.org/standards/webdesign/acessibilidade). Tim Berners-Lee, inventor da Web, disse a famosa frase: "O poder da Web está na sua universalidade.

O acesso de todos, independentemente da deficiência, é um aspecto essencial."

Qualquer dispositivo com navegador é capaz de exibir HTML, já que é apenas texto. Os meios pelos quais um usuário acessa o conteúdo podem variar, entretanto. Por exemplo, os utilizadores com visão visualizam o conteúdo, enquanto um utilizador com deficiência visual pode aumentar o tamanho da página ou da fonte ou utilizar um leitor de ecrã, software que lê o conteúdo em voz alta para eles (um exemplo de tecnologia de apoio). Em alguns casos, os leitores de tela anunciam o tipo de elemento HTML que envolve o conteúdo para fornecer ao usuário contexto sobre o que está por vir. Por exemplo, o usuário pode ser informado de que uma lista foi encontrada antes do

itens individuais da lista são lidos em voz alta. Da mesma forma, os usuários são informados quando um link é encontrado para que eles possam decidir se devem segui-lo.

Os usuários de leitores de tela podem navegar em uma página de várias maneiras, como pular de um título para outro por meio de um comando de teclado. Isso permite que eles reúnam os principais tópicos de uma página e ouçam com mais detalhes aqueles que lhes interessam, em vez de do que ter que ouvir a página inteira sequencialmente.

Então você pode ver por que uma boa semântica faz uma diferença marcante para usuários com deficiências.

O SEO – isto é, a classificação da sua página nos resultados dos mecanismos de pesquisa – pode melhorar, porque os mecanismos de pesquisa enfatizam as partes do seu conteúdo que são marcadas de uma maneira específica. Por exemplo, os títulos informam ao mecanismo de pesquisa os principais tópicos da sua página, ajudando o mecanismo de pesquisa a determinar como indexar o conteúdo da sua página.

À medida que avança no livro, você aprenderá por que uma boa semântica pode tornar seu código mais eficiente e mais fácil de manter e estilo.

Marcação: Elementos, Atributos e Valores

Agora que você já viu um pouco de HTML, vamos dar uma olhada mais de perto no que constitui marcação.

HTML tem três componentes principais de marcação: elementos, atributos e valores. Você viu exemplos de cada um em nossa página básica.

Elementos

Os elementos são como pequenos rótulos que descrevem as diferentes partes de uma página da Web: “Este é um título, aquela coisa ali é um parágrafo e aquele grupo de links é a navegação”.

Discutimos alguns elementos na seção anterior. Alguns elementos possuem um ou mais atributos, que descrevem melhor a finalidade e o conteúdo (se houver) do elemento.

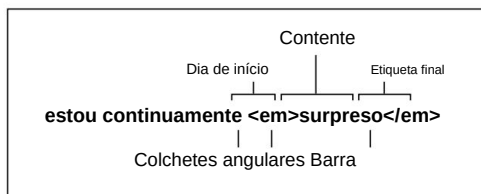
Os elementos podem conter texto e outros elementos, ou eles podem estar vazios. Um não-vazio

O elemento consiste em uma tag inicial (o nome do elemento e os atributos, se houver, entre sinais de menor que e maior que), o conteúdo e uma tag final (uma barra seguida pelo nome do elemento, novamente entre sinais de que e de maior que) A.

Um elemento vazio (também chamado de elemento void) se parece com uma combinação de tags de início e fim, com um sinal de menor que inicial, o nome do elemento seguido por quaisquer atributos que ele possa ter, um espaço opcional, uma barra opcional e o sinal maior que final, que é obrigatório B.

O espaço e a barra antes do final de um elemento vazio são opcionais em HTML5. Provavelmente é justo dizer que aqueles de nós que anteriormente codificaram em XHTML, que requer a barra para fechar um elemento vazio, tendem a usá-lo em HTML5 também, embora certamente outros o tenham abandonado.

Eu o incluí no meu código, mas se você optar por omiti-lo no seu, a página se comportará



R Aqui está um elemento HTML típico. A tag inicial e a tag final circundam o texto que o elemento descreve. Neste caso, a palavra “espantado” é enfatizada, graças ao elemento **em**. É comum digitar as tags dos elementos em letras minúsculas.

```

```

Um espaço e uma barra

B Elementos vazios, como **img** mostrado aqui, não circundam nenhum conteúdo de texto (o texto do atributo **alt** faz parte do elemento, não está rodeado por ele). Eles possuem uma única tag que serve tanto para abrir quanto para fechar o elemento. O espaço e a barra no final são opcionais no HTML5, mas é comum incluí-los. No entanto, o > que completa o elemento é obrigatório.

exatamente o mesmo. Seja qual for o caminho que você escolher, recomendo fazê-lo de forma consistente.

É comum digitar os nomes dos elementos em letras minúsculas, embora o HTML5 também não seja exigente aqui, permitindo letras maiúsculas. No entanto, hoje em dia é raro encontrar alguém que codifique em letras maiúsculas, então, a menos que o rebelde que existe em você não consiga resistir, eu não recomendo. É visto como uma prática datada.

Atributos e Valores

Os atributos contêm informações sobre o conteúdo do documento, em vez de ser o próprio conteúdo (C e D). Em HTML5, o valor de um atributo pode opcionalmente ser

entre aspas, mas é costume incluí-las, por isso recomendo que você sempre faça isso. E assim como acontece com os nomes dos elementos, recomendo que você digite os nomes dos atributos em letras minúsculas.

Embora você encontre detalhes sobre valores aceitáveis para a maioria dos atributos neste livro, deixe-me dar uma ideia dos tipos de valores que você encontrará à medida que avança.

Alguns atributos podem aceitar qualquer valor, outros são mais limitados. Talvez os mais comuns sejam aqueles que aceitam valores enumerados ou predefinidos. Em outras palavras, você deve selecionar um valor de uma lista padrão de

escolhas E. Certifique-se de escrever enumeradas valores em todas as letras minúsculas.

for é um atributo do **rótulo**

```
<label for="email">Endereço de e-mail</label>
```

O valor do atributo **for**

C Aqui está um elemento **rótulo** (que associa um rótulo de texto a um campo de formulário) com um par simples atributo-valor. Os atributos estão sempre localizados dentro da tag inicial de um elemento. É costume colocá-los entre aspas.

href é um atributo de **um**

Valor para **href**

rel também é um atributo de **um**

Valor para **rel**

```
<a href="http://en.wikipedia.org/wiki/Linum_lewisii" rel="external" title="Saiba mais sobre o Linho Azul">Linho Azul</a>
```

Valor para **título**

título é um atributo de **um**

D Alguns elementos, como **mostrado** aqui, podem receber um ou mais atributos, cada um com seu próprio valor. A ordem não é importante. Separe cada par atributo-valor do próximo com um espaço.

```
<link rel="folha de estilo" media=
```

Valor predefinido

E Alguns atributos aceitam apenas valores específicos. Por exemplo, o atributo **media** no elemento **link** pode ser definido como **all**, **screen** ou **print**, entre outros, mas você não pode simplesmente criar um valor para ele como faz com o Atributo do Título.

Muitos atributos exigem um número para seu valor, especialmente aqueles que descrevem tamanho e comprimento. Um valor numérico nunca inclui unidades, apenas o número. Quando as unidades são aplicáveis, como na largura e na altura de uma imagem ou vídeo, elas são entendidas como pixels.

Alguns atributos, como **href** e **src**, fazem referência a outros arquivos e, portanto, devem conter valores na forma de URL, ou Uniform Resource Locator, o endereço exclusivo de um arquivo na Web. Você aprenderá mais sobre URLs na seção "URLs" deste capítulo.

Pais e Filhos

Se um elemento contiver outro, ele será considerado o pai do elemento incluído ou filho. Quaisquer elementos contidos no elemento filho são considerados descendentes do pai externo.

elemento F. Na verdade, você pode criar uma árvore genealógica de uma página da Web que mostre os relacionamentos hierárquicos entre cada elemento da página e que identifique cada elemento exclusivamente.

Essa estrutura subjacente semelhante a uma árvore genealógica é um recurso importante do código HTML. Ele facilita tanto os elementos de estilo (sobre os quais você começará a aprender no Capítulo 7) quanto a aplicação do comportamento JavaScript a eles.

É importante observar que quando os elementos contêm outros elementos, cada elemento deve estar adequadamente aninhado, ou seja, totalmente contido em seu pai. Sempre que você usar uma tag final, ela deverá corresponder à última tag inicial não fechada. Em outras palavras, primeiro abra o elemento 1, depois abra o elemento 2, depois feche o elemento 2 e depois feche o elemento 1 G.

```
<artigo>
  <h1>O Efêmero Linho Azul</h1>
  
  <p>... continuamente <em>espantado</em> ... delicado <a ...> Linho Azul</a> ...</p>
</artigo>
```

F O elemento **article** é pai dos elementos **h1**, **img** e **p**. Por outro lado, os elementos **h1**, **img** e **p** são filhos (e descendentes) do **artigo**. O elemento **p** é pai dos elementos **em** e **a**. O **em** e **a** são filhos do **p** e também descendentes (mas não filhos) do **artigo**. Por sua vez, o **artigo** é seu ancestral.

Correto (sem linhas sobrepostas)

```
<p>... continuamente <em>espantado</em> ...</p>
<p>... continuamente <em>espantado...</p></em>
```

Incorreto (os conjuntos de tags se cruzam)

G Os elementos devem estar adequadamente aninhados. Se você abrir **p** e depois **em**, deverá fechá-los antes de fechar **p**.

Uma página da Web

Conteúdo de texto

O texto contido nos elementos talvez seja o ingrediente mais básico de uma página da Web. Se você já usou um processador de texto, você digitou algum texto. O texto em uma página HTML, entretanto, apresenta algumas diferenças importantes.

Primeiro, quando um navegador renderiza HTML, ele recolhe espaços ou tabulações extras em um único espaço e converte retornos e feeds de linha em um único espaço ou os ignora completamente (A e B).

Em seguida, o HTML costumava ser restrito a caracteres ASCII – basicamente as letras do idioma inglês, numerais e alguns dos símbolos mais comuns. Caracteres acentuados (comuns a muitas línguas da Europa Ocidental) e muitos símbolos cotidianos tiveram que ser criados com referências de caracteres especiais como **´** (para é) ou **©** (para ©). Veja a lista completa em www.elizabethcastro.com/html/extras/entities.html.

Unicode atenua muitos problemas com caracteres especiais. É prática padrão codificar páginas em UTF-8, como na página básica C, e salvar arquivos HTML com a mesma codificação (veja “Salvando sua página da Web” no Capítulo 2). Eu recomendo que você faça o mesmo.

Como o Unicode é um superconjunto de ASCII – é tudo o que ASCII é e muito mais – documentos codificados em Unicode são compatíveis com navegadores e editores existentes, exceto os mais antigos. Navegadores que não entendem Unicode irão

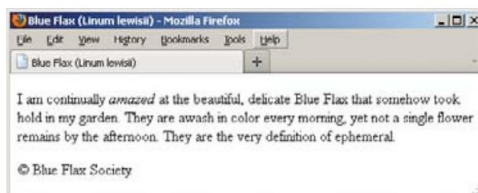
interpretar a parte ASCII do documento corretamente, enquanto os navegadores que entendem Unicode também exibirão a parte não-ASCII. Mesmo assim, às vezes ainda é comum usar referências de caracteres, como para o símbolo de copyright, pois é fácil lembrar e digitar **©**A.

O conteúdo de texto de uma página (destacado) é basicamente qualquer coisa além da marcação. Neste exemplo, observe que cada frase é separada por pelo menos um retorno de carro e algumas palavras são separadas por vários espaços (apenas para enfatizar o ponto sobre o colapso de retornos e espaços). Além disso, inclui uma referência de caractere especial (**©**) para o símbolo de copyright para garantir que ele seja exibido corretamente, independentemente da codificação na qual você salvou este documento.

```
<p> Fico continuamente <em>surpreso</em> com  
o &y lindo, &y que delicado Linho Azul que  
de alguma forma tomou conta do meu jardim.
```

```
Elas ficam inundadas de cores todas as &y  
manhãs, mas nem uma única flor &y  
permanece à tarde.
```

```
Eles são a própria definição de &y  
efêmero.</p>  
<p>&copy; Sociedade do Linho Azul.</p>
```



B Observe que quando você visualiza o documento com um navegador, os retornos e espaços extras são ignorados e a referência do caractere é substituída pelo símbolo correspondente (©).

C Especifique a codificação de caracteres do seu documento diretamente após a tag **head** start. O **conjunto de caracteres** atributo define o tipo de codificação.

```
<!DOCTYPEhtml>  
<html lang="pt">  
<cabeca>  
    <meta charset="utf-8" />  
    <title>Linho Azul (Linum lewisii)</title>  
</head>  
<corpo>  
    ...  
</body>  
</html>
```

R Em nosso documento HTML básico, há uma referência a um arquivo de imagem chamado **blueflax.jpg**, que o navegador solicitará, carregará e exibirá quando carregar o restante da página. A página também inclui um link para outra página sobre Blue Flax.

```
...
<artigo>
  <h1>O Efêmero Linho Azul</h1>

  <p>Fico continuamente <em>surpreso</
  em> com & y a linda e delicada <a href=
  & y "http://en.wikipedia.org/wiki/Linum_
  & y lewisii" rel="external" title="Saiba & y
  mais sobre o Linho Azul">Linho Azul & y </
  a> que de alguma forma tomou conta
  do meu & y jardim. Eles ficam cheios de
  cores todas as & y manhãs, mas nem
  uma única flor & y permanecem à tarde. Eles
  são a & y própria definição de efêmero.</p>
</artigo>
...
```



B Imagens e outros conteúdos não textuais são referenciados em uma página da Web e o navegador os exibe junto com o texto.

Links, imagens e Outro não-texto Conteúdo

É claro que parte do que torna a Web tão vibrante são os links de uma página para outra e as imagens, vídeos, músicas, animações e muito mais. Em vez de realmente incluir os arquivos externos, como vídeos, no arquivo HTML, esses arquivos são salvos de forma independente e são simplesmente referenciados na página A. Como a referência nada mais é do que texto, o arquivo HTML permanece quase universalmente acessível. .

Os navegadores podem lidar com links e imagens (exceto em navegadores somente de texto) sem perder o ritmo. No entanto, eles não podem necessariamente lidar com qualquer outro tipo de arquivo. Se você fizer referência a um arquivo que o navegador do visitante não entende, o navegador muitas vezes tentará encontrar um plug-in ou aplicativo auxiliar— algum programa apropriado no computador do visitante — que seja capaz de abrir esse tipo de arquivo.

Você também pode fornecer aos navegadores informações extras sobre como renderizar conteúdo com um plugin, se necessário, ou como fazer o download do plugin, se o visitante ainda não o tiver em seu computador.

Todo esse negócio de baixar e instalar plug-ins atrapalha a experiência do usuário em seu site, desde que ele permaneça por aqui. Os plug-ins também podem apresentar problemas de desempenho porque não são uma parte nativa do navegador.

O Flash, por exemplo, tem sido o plugin mais difundido há anos. Sem dúvida, você já assistiu a um vídeo on-line reproduzido em Flash em algum momento e sentiu o computador ficar lento ou travar ocasionalmente o navegador (ou ambos).

O HTML5 tenta mitigar muitos desses problemas introduzindo a reprodução de mídia nativa no navegador por meio do **áudio** e elementos **de vídeo**. Infelizmente, tem havido um debate entre os fornecedores de navegadores sobre quais formatos de mídia suportar, então você nem sempre pode acabar com os plug-ins ainda. Mas é um começo.

Você aprenderá mais sobre imagens no Capítulo 5 e abordará plug-ins, elementos de mídia do HTML5 e muito mais no Capítulo 17.

Nomes de arquivos

Como qualquer outro documento de texto, uma página da Web possui um nome de arquivo que se

identifica para você, para os visitantes e para os navegadores da Web dos visitantes. Há algumas dicas que você deve ter em mente ao atribuir nomes de arquivos às suas páginas da Web que ajudarão você a organizar seus arquivos, tornar mais fácil para os visitantes encontrarem e acessarem suas páginas, garantir que seus navegadores visualizem as páginas corretamente e melhorar o SEO. (A e B).

Use nomes de arquivos em letras minúsculas

Como o nome do arquivo que você escolhe para sua página da Web determina o que os visitantes terão que digitar para chegar à sua página, você pode salvá-los de erros de digitação inadvertidos (e dores de cabeça) usando apenas letras minúsculas nos nomes dos arquivos.

Também é uma grande ajuda quando você mesmo cria links entre suas páginas. Se todos os nomes de arquivos tiverem apenas letras minúsculas, será apenas uma coisa a menos com que você terá que se preocupar.

Separe palavras com um traço

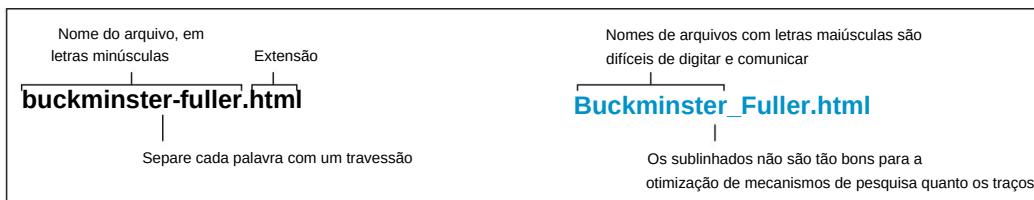
Nunca inclua espaços entre palavras nos nomes dos arquivos. Em vez disso, use um travessão, por exemplo, **company-history.html** e **my-favorite-movies.html**. Você encontrará sites ocasionais que usam sub-

pontuações ("_"), mas não são recomendadas, porque os travessões são preferidos pelos motores de busca.

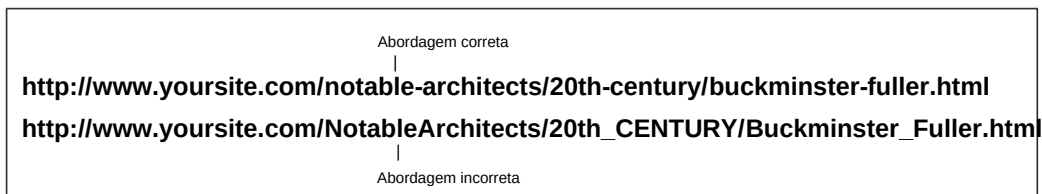
Use a extensão adequada

A principal forma de um navegador saber que deve ler um documento de texto como uma página da Web é observando sua extensão. Embora .htm também funciona, .html é o habitual, então recomendo que você use isso como extensão. Se a página tiver alguma outra extensão, como .txt, o navegador irá tratá-la como texto e mostrar todo o seu código bacana ao visitante.

TIP Esteja ciente de que nem o Mac OS nem o Windows sempre revelam a extensão real de um documento. Altere as opções de pasta, se necessário, para poder ver as extensões.



R Lembre-se de usar letras minúsculas nos nomes dos arquivos, separar as palavras com um travessão e adicionar a extensão .html. A mistura de letras maiúsculas e minúsculas torna mais difícil para os visitantes digitarem o endereço correto e encontrarem sua página.



B Use todas as letras minúsculas e travessões também para seus diretórios e pastas. A chave é a consistência. Se você não usar letras maiúsculas, seus visitantes (e você) não precisarão perder tempo se perguntando "Agora, era B maiúsculo ou minúsculo?"

URLs

Uniform Resource Locator, ou URL, é um nome sofisticado para endereço. Ele contém informações sobre onde um arquivo está e o que um navegador deve fazer com ele. Cada arquivo no

A Internet tem um URL exclusivo.

A primeira parte do URL é chamada de esquema. Diz ao navegador como lidar com o arquivo que está prestes a abrir. O esquema mais comum que você verá é HTTP ou Hypertext Transfer Protocol. É usado para acessar páginas da Web A.

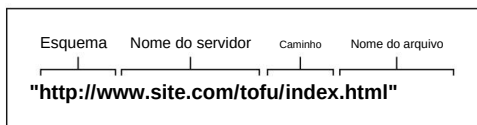
A segunda parte da URL é o nome do servidor onde o arquivo está localizado, seguido do caminho que leva ao arquivo e do próprio nome do arquivo. Às vezes, um URL omite um nome de arquivo e termina com um caminho, que pode ou não incluir uma barra B. Nesse caso, o URL se refere ao arquivo padrão no último diretório do caminho, normalmente chamado **de índice .html**.

Outros esquemas comuns são **https**, para páginas Web seguras; **ftp** (File Transfer Protocol), para download de arquivos C; **mailto**, para envio de e-mail D; e **arquivo**, para acessar arquivos em um disco rígido local ou em redes locais de compartilhamento de arquivos (você não terá a oportunidade de usar o esquema **de arquivos** com muita frequência, se é que o fará) E.

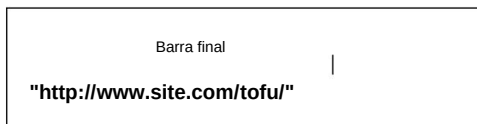
Um esquema geralmente é seguido por dois pontos e duas barras. **mailto** e **notícias** são exceções; estes levam apenas dois pontos.

Observe que o esquema **do arquivo** é seguido por dois pontos e três barras. Isso é porque o host, que em outros esquemas fica entre a segunda e a terceira barras, é considerado o computador local. Sempre digite esquemas em letras minúsculas.

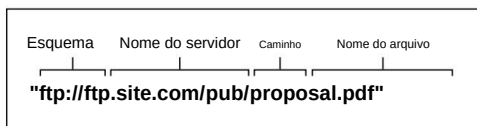
Desses esquemas, você usará **http** e **mailto** com mais frequência. Os demais são para casos especializados.



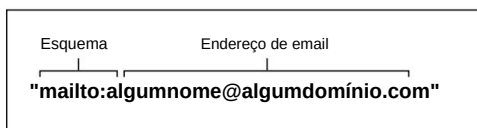
A Sua URL básica contém um esquema, nome do servidor, caminho e nome do arquivo.



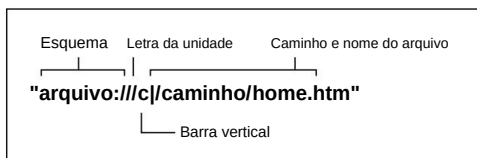
B Um URL com uma barra final e sem nome de arquivo aponta para o arquivo padrão no último diretório nomeado (neste caso, o diretório **tofu**). O nome de arquivo padrão mais comum é **index.html**. Portanto, esta URL e a do exemplo anterior apontam para a mesma página.



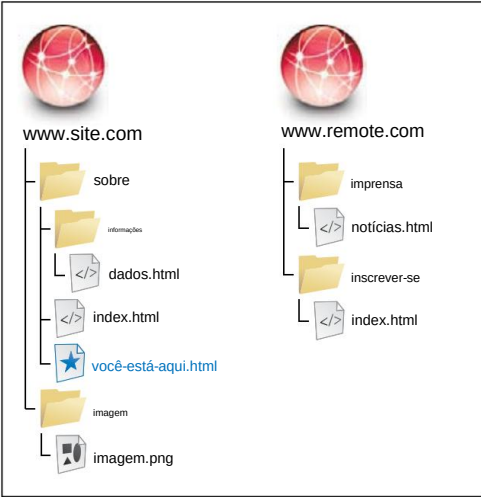
C Quando o usuário clicar neste URL, o navegador iniciará uma transferência FTP do arquivo **proposta.pdf**.



D Um URL para um endereço de e-mail inclui o esquema **mailto** seguido por dois pontos, mas sem barras, e depois o próprio endereço de e-mail.



E Para referenciar um arquivo em um Windows local máquina, use o esquema **de arquivo**. Para Macintosh, use **file:/// Harddisk/path/filename**. Nenhuma barra vertical é necessária. (Isso às vezes também funciona para Windows.)



O documento que contém os URLs (neste caso, **você está aqui.html**) é o ponto de referência para URLs relativos. Em outras palavras, URLs relativos são relativos à localização desse arquivo no servidor. URLs absolutos funcionarão independentemente de onde estejam localizados, pois sempre contém o URL completo de um recurso.

URLs absolutos

URLs podem ser absolutos ou relativos. Um URL absoluto mostra o caminho completo para o arquivo, incluindo o esquema, o nome do servidor, o caminho completo e o próprio nome do arquivo F. Um URL absoluto é análogo a um endereço completo, incluindo nome, rua e número, cidade, estado , código postal e país. Não importa de onde a carta seja enviada, os correios poderão encontrar o destinatário. Em termos de URLs, isso significa que a localização do próprio URL absoluto foi

não tem influência sobre a localização do arquivo real referenciado - seja em uma página da Web em seu servidor ou em outro servidor, um valor absoluto URL para um arquivo específico será exatamente igual mesmo.

Ao fazer referência a um arquivo do servidor de outra pessoa, você sempre usará um URL absoluto. Você também precisará usar

URLs absolutos para sites FTP ou, geralmente, qualquer tipo de URL que não use protocolo HTTP.

A **Tabela 1.1** descreve como você acessaria vários arquivos de **you-are-here.html** — tanto aqueles no mesmo site (site.com) da página quanto em outro site (remote.com) — como forma de ilustrar a diferença entre URLs relativos e absolutos.

TABELA 1.1 URLs absolutos versus URLs relativos

Nome do arquivo	URL absoluto (pode ser usado em qualquer lugar)	URL relativo (funciona apenas em you-are-here.html)
index.html	http://www.site.com/about/index.html	index.html
dados.html	http://www.site.com/about/info/data.html	/info/data.html
imagem.png	http://www.site.com/img/image.png	../img/image.png
noticias.html	http://www.remote.com/press/news.html	(nenhum: use absoluto)
index.html	http://www.remote.com/sign-up/index.html	(nenhum: use absoluto)

URLs relativos

Para lhe dar instruções sobre como chegar à casa da minha vizinha, em vez de fornecer o endereço completo, posso apenas dizer: “fica três portas abaixo, à direita”. Este é um endereço relativo – para onde ele aponta depende da origem da informação. Com a mesma informação em uma cidade diferente, você nunca encontraria meu vizinho.

Da mesma forma, um URL relativo descreve a localização do arquivo desejado com referência para o local do arquivo que contém a própria referência do URL. Portanto, você pode fazer com que o URL diga algo como “link para a página xyz que está no mesmo diretório desta página”.

O URL relativo de um arquivo que está no mesmo diretório da página atual (ou seja, aquele que contém o URL em questão) é simplesmente o nome do arquivo e a extensão. Você cria o URL para um arquivo em um subdiretório do diretório atual digitando o nome do subdiretório seguido por uma barra e, em seguida, o nome e a extensão de o arquivo desejado.

Para fazer referência a um arquivo em um diretório em um nível superior da hierarquia de arquivos, use dois pontos e uma barra. Você pode combinar e repetir os dois pontos e a barra para fazer referência a qualquer arquivo no mesmo servidor ou unidade que o arquivo atual.

Dentro da pasta atual, existe um arquivo chamado “index.html”...

"index.html"

G O URL relativo para vincular a um arquivo na mesma pasta (ver F). Apenas o nome e a extensão do arquivo são obrigatórios no URL, em vez de precedê-los com **http://www.site.com/about/** (a pasta na qual ambos os arquivos residem).

Dentro da pasta atual, há uma pasta chamada “info”...

"info/dados.html"

...Isso contém...

...um arquivo chamado “data.html”.

H Para referenciar um arquivo (**data.html**, neste exemplo) que está dentro de uma pasta dentro da pasta atual (veja F), adicione o nome da subpasta e um encaminhamento barra na frente do nome do arquivo.

A pasta que contém a pasta atual...

...Contém... uma pasta chamada “img”

"../img/image.png"

...Isso contém...

...um arquivo chamado “image.png”...

I Este arquivo, como você pode ver em F, está em uma pasta (img) que fica ao lado da pasta atual (about) no diretório raiz do site. Nesse caso, você usa dois pontos e uma barra para subir de nível e, em seguida, anota o subdiretório, seguido por uma barra, seguida pelo nome do arquivo. (Na prática normal, você escolheria um nome de arquivo de imagem mais descritivo do que **image.png**, que é deliberadamente genérico para o exemplo.)

Alternativamente, se seus arquivos estiverem em um servidor Web, você pode evitar caminhos de arquivo complicados, como `../img/family/vacation.jpg`, primeiro indo direto para a raiz do seu site e, em seguida, detalhando a partir daí até o arquivo de destino. Uma única barra no início consegue isso, então o URL relativo da raiz neste caso seria `/img/family/vacation.jpg` (assumindo a `img` fica na pasta raiz do site, o que é habitual).

Novamente, isso só funciona em um servidor Web, como no provedor de hospedagem que atende seu site ou em um que você esteja executando localmente em sua máquina (Apache é a escolha mais popular para isso).

Se você não estiver desenvolvendo seu site localmente em um servidor, geralmente você desejará usar URLs relativos (exceto quando apontar para arquivos no servidor de outra pessoa, é claro). Eles facilitarão a movimentação de suas páginas de um sistema local para um servidor. Contanto que a posição relativa de cada arquivo permaneça constante, você não precisará alterar nenhum dos caminhos, para que os links funcionem corretamente.

Principais conclusões

Os conceitos básicos de HTML e algumas práticas recomendadas importantes fornecem a base para a construção de sites eficazes. Vamos revisitar as principais conclusões:

n Uma página da Web é composta principalmente de três componentes: conteúdo de texto, referências a outros arquivos e marcação.

n A marcação HTML é composta de elementos, atributos e valores.

n É comum escrever seu HTML em letras minúsculas (DOCTYPE é uma exceção), colocar os valores dos atributos entre aspas e fechar os elementos vazios com um espaço e uma barra (/).

n Sempre comece seus documentos HTML com a declaração DOCTYPE:

<!DOCTYPEhtml>

n O conteúdo de uma página vai para o elemento **body** . As instruções destinadas principalmente ao navegador e aos mecanismos de pesquisa estão antes disso, principalmente na **cabeça**.

n Marque seu conteúdo com HTML semântico e sem se preocupar com como ele deve aparecer em um navegador.

n O HTML semântico melhora a acessibilidade e pode tornar seu site mais eficiente e mais fácil de manter e estilizar.

n CSS controla a apresentação do HTML contendo.

n A folha de estilo de cada navegador determina a apresentação padrão do HTML. Você pode substituir essas regras pelo seu próprio CSS.

n Crie nomes de arquivos e pastas em todos os casos e separe as palavras com um travessão em vez de um espaço ou sublinhado.

A seguir, você aprenderá como trabalhar com arquivos de páginas da Web.

2

Trabalhando com Web

Arquivos de página

Antes de começar a escrever elementos e atributos HTML, é importante saber como criar os arquivos nos quais usará esse código.

Neste capítulo, você aprenderá como criar, editar e salvar arquivos de páginas da Web.

Também abordarei algumas considerações de planejamento e organizacionais.

Se você não aguenta mais esperar e já sabe como criar os arquivos reais, vá para o Capítulo 3, onde começo a explicar o código HTML em si.

Neste capítulo

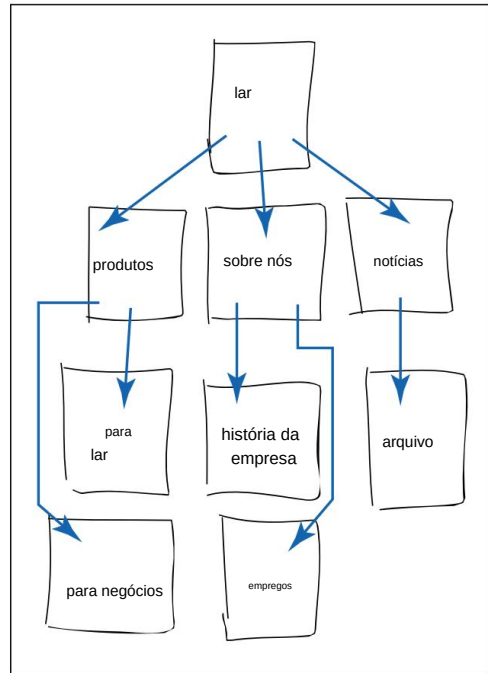
Planejando seu site	26
Criando uma nova página da web	28
Salvando sua página da web	30
Especificando uma página padrão ou página inicial	33
Editando páginas da web	35
Organizando arquivos	36
Visualizando sua página em um navegador	37
A inspiração de outros	39

Planejando seu site

Embora você possa começar a escrever páginas da Web imediatamente, é uma boa ideia primeiro pensar e planejar seu site. Dessa forma, você terá uma direção e precisará fazer menos reorganizações. mais tarde.

Para planejar seu site:

- n Descubra por que você está criando este site.
O que você quer transmitir?
- n Pense no seu público. Como pode
você adapta seu conteúdo para atrair esse público?
- n Quantas páginas serão necessárias? Que tipo de
estrutura você gostaria que tivesse?
Você deseja que os visitantes percorram seu site
em uma sequência específica ou deseja que
seja mais fácil para eles explorarem em qualquer
direção?
- n Esboce seu site no papel.
- n Elabore uma nomenclatura simples e consistente
convenção para suas páginas, imagens e outros
arquivos externos (consulte "Nomes de arquivos"
no Capítulo 1).



R Esboçar seu site e pensar no que ele pode conter pode ajudá-lo a decidir que tipo de estrutura ele precisa.

TIP Não exagere na fase de planejamento do seu site. Em algum momento, você terá que se aprofundar e começar a escrever conteúdo e código.

TIP Se você não está muito familiarizado com a Web, navegue primeiro para ter uma ideia das possibilidades. Você pode começar com os sites de alguns de seus concorrentes.

TIP É comum, mas não obrigatório, mapear a estrutura de pastas do seu site de acordo com a forma como ela está organizada no papel A. Consulte “Organização de arquivos”.

TIP Veja o artigo de Erin Kissane “Uma lista de verificação para Content Work” (www.alistapart.com/articles/a-checklist-for-content-work/) para ideias sobre como você pode abordar a elaboração do conteúdo do seu site. É uma amostra do livro dela, que aborda o tema estratégia de conteúdo.

TIP Os Princípios da Beleza, de Jason Beaird
ful Web Design (SitePoint, 2010) pode interessá-lo se
você não for um designer ou um designer novato e
estiver procurando orientação sobre como criar um
site atraente e eficaz.

Criando um novo

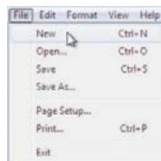
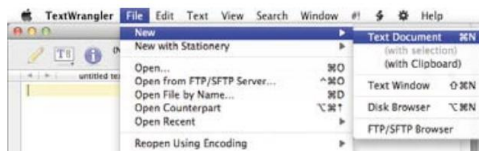
Página da Internet

Você não precisa de nenhuma ferramenta especial para criar uma página da Web. Você pode usar qualquer editor de texto, até mesmo o Notepad (A e C), que está incluído no Windows, ou o TextWrangler (A e B), que pode ser baixado gratuitamente para OS X (www.barebones.com/products/textwrangler) . (Macs incluem um editor chamado TextEdit, mas há um bug em algumas versões do OS X que torna difícil

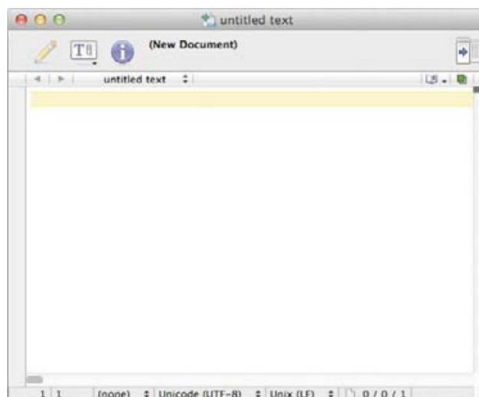
para trabalhar com arquivos HTML.)

Para criar uma nova página da Web:

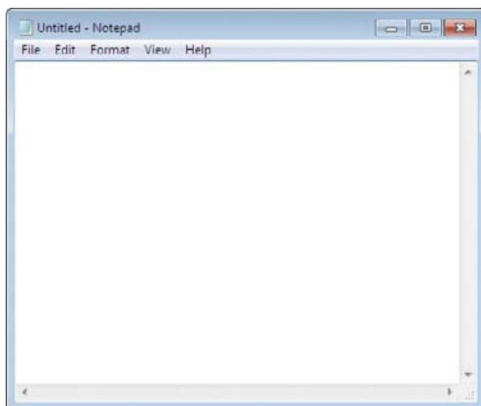
1. Abra qualquer editor de texto.
2. Escolha Arquivo > Novo para criar um novo, documento em branco A.
3. Crie o conteúdo HTML conforme explicado no restante deste livro, começando pelo Capítulo 3.
4. Certifique-se de salvar seu arquivo conforme indicado em "Salvar sua página da web".



A Abra seu editor de texto. Digite seu HTML no documento em branco que aparece ou escolha Arquivo > Novo. A opção exata do menu pode variar ligeiramente. Se você estiver usando TextWrangler (Mac), é Arquivo > Novo > Documento de Texto, conforme mostrado (parte superior). A outra imagem é o Bloco de Notas (Windows) (parte inferior).



B Em um Mac, você pode usar o TextWrangler para escrever o código HTML da sua página. Veja nas dicas uma lista de editores para Mac com recursos mais robustos para escrever código.



C Este é o Bloco de Notas, o programa mais básico que os usuários do Windows podem usar para criar páginas HTML. Vários outros estão disponíveis (veja as dicas).

TIP Existem vários editores de texto para OS X e Windows que são especificamente adaptados para codificação de HTML (e CSS). Eles têm recursos de dicas de código e conclusão de código para ajudá-lo a codificar com mais precisão e rapidez, realçam o código para facilitar a distinção entre os elementos HTML e o conteúdo de texto que você escreveu neles e possuem diversos outros recursos úteis; O bloco de notas não possui nenhum deles. Alguns editores de HTML gratuitos estão disponíveis, mas outros geralmente valem o investimento e geralmente incluem uma versão de teste gratuita que você pode testar antes de fazer uma compra.

TIP Alguns editores populares para OS X são BBEdit (www.barebones.com/products/bbedit/), Coda (www.panic.com/coda/), Espresso (<http://macrabbbit.com/espresso/>), Sublime Text (www.sublimetext.com) e TextMate (<http://macromates.com>). (TextWrangler é comumente conhecido como "BBEdit Lite".) TextMate é o mais popular deles, embora tenha visto a concorrência invadir sua base de usuários. O Sublime Text também está disponível no Windows, assim como o E Text Editor (www.e-texteditor.com), o Notepad++ (<http://notepad-plus-plus.org>) e muitos outros. Pesquise online por "editor HTML" para saber mais.

TIP Se você usar um editor como um mencionado na dica anterior, o processo é semelhante para a criação de uma nova página. E para editar uma página existente, basta escolher Arquivo > Abrir no editor de texto de sua preferência e abrir o arquivo (consulte "Editando páginas da Web"). Use o restante deste livro para adicionar seu próprio HTML e CSS para criar a página desejada.

TIP Não use processadores de texto, como o Microsoft Word, para codificar suas páginas HTML. Eles podem adicionar código desnecessário ou inválido aos seus arquivos.

Salvando sua página da web

Você cria páginas da Web com um editor de texto, mas elas devem ser visualizadas em vários navegadores e em diversas plataformas. Para ser acessível a todos esses diferentes programas, você salva páginas da Web em um formato universal “somente texto” – sem qualquer formatação proprietária que um processador de texto possa aplicar.

Para que os navegadores (e servidores) reconheçam as páginas da Web e saibam interpretar a marcação que elas contêm, os arquivos de páginas da Web têm a extensão .html ou .htm em seus arquivos.

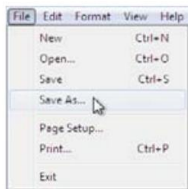
nomes de arquivos; isso também distingue esses arquivos de arquivos de texto simples que não são páginas da Web. Embora ambos funcionem, é comum usar a extensão .html, por isso recomendo que você a utilize para seus arquivos.

Por causa dessa extensão, o ícone de uma página da Web corresponde ao navegador padrão do sistema – não ao editor com o qual o arquivo foi escrito A. Na verdade, quando você clica duas vezes em um arquivo de página da Web, ele é aberto em um navegador, não em um editor de texto. Isso pode ser ótimo para testar uma página em um navegador, mas acrescenta uma etapa extra à edição de páginas da Web (consulte “Editando páginas da Web”).

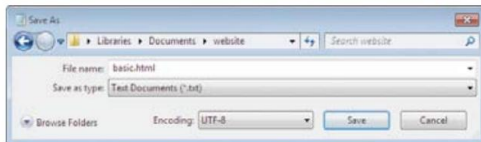
Para resumir, ao salvar sua página da Web, você deve salvá-la em formato somente texto com a extensão .html ou .htm.



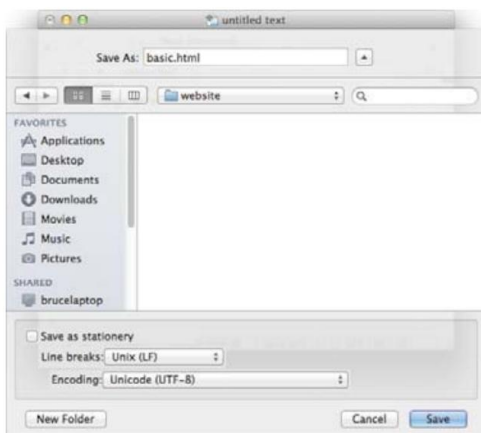
A Uma planilha do Excel possui a extensão .xlsx e é identificada com o ícone do Excel (parte superior). Se você clicar duas vezes nele, ele será exibido no Excel. Um arquivo de página da Web, independentemente do editor de texto com o qual você o criou, possui a extensão .html ou .htm e é identificado com o ícone do navegador padrão (Firefox, neste caso). Se você clicar duas vezes nele, ele será exibido no navegador padrão (não no editor de texto).



B Escolha Arquivo > Salvar como no seu editor de texto.



C No Bloco de notas, dê um nome ao seu arquivo com a extensão .html ou .htm, escolha Documentos de texto no menu suspenso "Salvar como tipo", certifique-se de que a codificação esteja definida como UTF-8 (veja a última dica) e clique em Salvar. As opções podem ser diferentes (mas semelhantes) em outro editor de texto.



D No TextWrangler, dê um nome ao seu arquivo e escolha um local para salvá-lo. O padrão do TextWrangler é UTF-8 (que é o que você deseja, exceto em casos especiais), mas você pode fazer uma escolha diferente no menu suspenso Codificação (veja a última dica). Clique em Salvar para salvar o arquivo.

Para salvar sua página da Web:

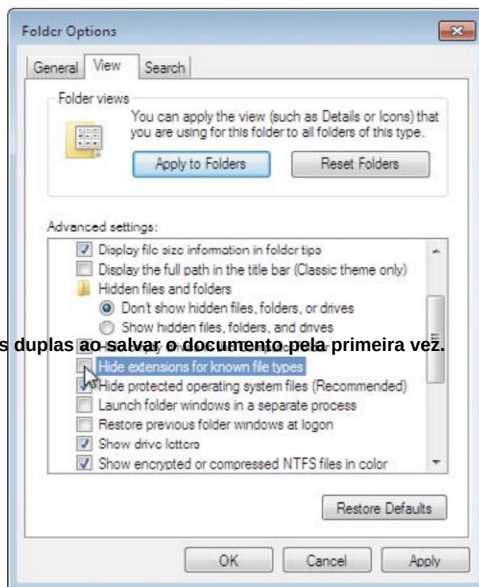
1. Depois de criar sua página da Web, escolha Arquivo > Salvar como em seu editor de texto B.
2. Na caixa de diálogo que aparece, escolha Texto Simples ou Documento de Texto (ou qualquer que seja a palavra do seu programa) para o formato.
3. Dê ao documento a extensão .html (de preferência) ou .htm. (Isto é muito importante!)
4. Escolha a pasta na qual deseja salvar o Página da Internet.
5. Clique em Salvar (**C** e D).

TIP Não importa se você usa .html ou .htm, embora .html seja recomendado porque é a escolha comum. Seja qual for o uso, seja consistente, pois usar a mesma extensão tornará mais fácil lembrar seus URLs posteriormente.

continua na próxima página

TIP Alguns editores de texto no Windows podem adicionar sua extensão padrão ao nome do arquivo, mesmo se você já tiver especificado .html ou .htm. (Observe que isso não deve ser um problema com a maioria dos editores projetados especificamente para editar páginas HTML.) Seu arquivo, agora denominado webpage.html.txt, não será visualizado corretamente em um navegador. Para piorar a situação, o Windows muitas vezes oculta extensões para que o problema não seja completamente óbvio, especialmente para os não iniciados. Existem duas soluções. A primeira é colocar o nome do arquivo entre aspas duplas ao salvar o documento pela primeira vez. Isso deve evitar que a extensão extra seja adicionada. Em seguida, você pode dizer ao Windows para exibir as extensões de arquivo E, para que você possa ver a extensão ofensiva e removê-la do nome do arquivo.

TIP Quando você escolhe um formato somente texto, seu arquivo geralmente é salvo com a codificação de caracteres padrão do sistema. Se você quiser criar páginas da Web em outra codificação (talvez para incluir símbolos especiais ou texto em outros idiomas), você terá que usar um editor de texto que permita escolher a codificação. Normalmente, UTF-8 é a melhor opção de codificação. Se o seu editor tiver a opção de salvar arquivos codificados como "UTF-8, sem BOM", "UTF-8, sem BOM" ou algo semelhante, escolha essa opção. Caso contrário, escolha UTF-8 F. Em alguns casos, o modo UTF-8 de um editor não inclui a lista técnica, mesmo que não anote explicitamente esse fato em seu menu de codificação. (Veja http://en.wikipedia.org/wiki/Byte_order_mark se você estiver curioso sobre o significado do BOM. Esteja preparado para se encantar!)



E No Windows Explorer, escolha Organizar > Opções de pasta e pesquisa ou Ferramentas > Opções de pasta (dependendo da sua versão do Windows) para visualizar esta caixa de diálogo. Pode parecer diferente dependendo da sua versão do Windows. Clique na guia Exibir e role para baixo até ver "Ocultar as extensões dos tipos de arquivo conhecidos". Certifique-se de que esteja desmarcado se quiser ver a extensão de um arquivo (como .html) na área de trabalho.



F Muitos editores de texto permitem escolher a codificação do seu arquivo, para que você possa salvar símbolos e caracteres de diferentes idiomas no mesmo documento. UTF-8 é a codificação recomendada na maioria dos casos. Escolha a opção UTF-8 sem BOM se estiver disponível em seu editor. Caso contrário, escolha UTF-8. Alguns editores (como o TextWrangler, mostrado aqui) usam-no como padrão.



A Salve o arquivo como **index.html** para designar o arquivo como a página padrão que deve ser aberta naquele diretório.



B Quando o visitante digita o caminho para o diretório, mas omite o próprio nome do arquivo, o arquivo com o nome padrão é usado. Eu digitei `http://bruceontheloose.com/htmlcss/examples/antoni-gaudi/` neste exemplo. Se eu tivesse digitado `http://bruceontheloose.com/htmlcss/examples/antoni-gaudi/index.html` em vez disso, a mesma página teria carregado.

Especificando um padrão Página ou página inicial

A maioria dos servidores Web possui um sistema para reconhecer uma página padrão em cada pasta, com base no nome do arquivo. Em quase todos os casos, **index.html** é reconhecido como a página padrão A, embora os servidores normalmente procure nomes de arquivos como **index.htm** e **default.htm** se **index.html** não existir. Se seus visitantes digitam uma URL com um diretório, mas não especificam um nome de arquivo, o arquivo padrão é usado B.

A página padrão (normalmente **index.html**) que você cria no nível superior do seu diretório da Web (geralmente chamada de raiz) é a página inicial do seu site. Esta é a página que aparecerá quando os visitantes digitarem seu domínio sem informações adicionais de caminho: `www.seudominio.com`.

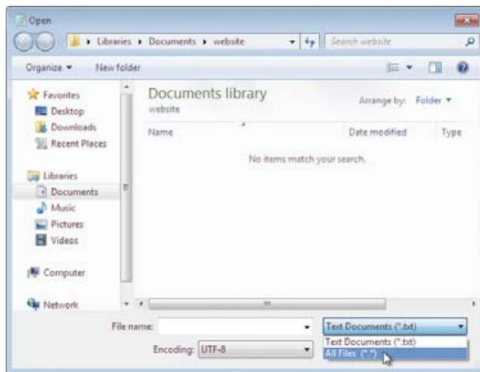
Da mesma forma, você pode criar uma página padrão para todo e qualquer diretório do seu site. Por exemplo, a página de destino (ou seja, a página principal) de **/produtos/** ou **/sobre-nós/** O diretório em seu site também seria chamado de **index.html**, mas existiria em sua pasta específica. (Um diretório é apenas uma pasta, como as que você vê na unidade do seu computador.) Os visitantes normalmente acessam essas seções do seu site a partir da sua página inicial ou por meio da navegação principal que existe em cada página.

continua na próxima página

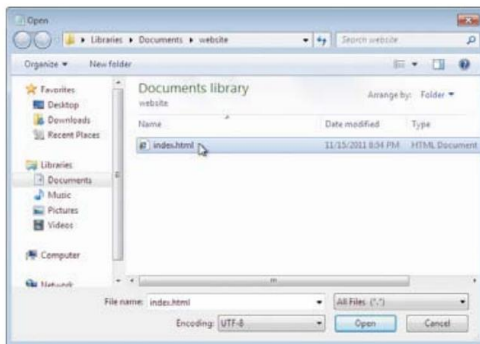
Para especificar uma página inicial para seu site ou uma página inicial para um diretório dentro dele:

Salve seu arquivo como **index.html** (veja “Salvando sua página da Web”) na pasta desejada. (Se **index.html** não funcionar como padrão página no servidor do seu site ao carregá-la conforme o Capítulo 21, consulte seu provedor de hospedagem na Web.)

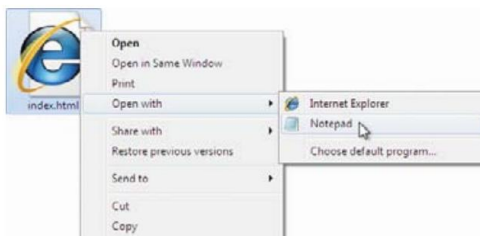
TIP Se você não tiver uma página padrão em cada diretório, alguns servidores poderão mostrar uma lista do conteúdo do diretório (que você pode ou não querer revelar aos visitantes). Para manter esses olhares curiosos afastados, crie uma página padrão para cada diretório do seu site que contenha páginas HTML. Alternativamente, você pode alterar a configuração do servidor para que a lista de arquivos fique oculta (você também pode mostrá-la se já estiver oculta). Ocultar a lista é aconselhável para pastas que contêm ativos, como imagens, arquivos de mídia, folhas de estilo e arquivos JavaScript. Peça instruções ao seu provedor de hospedagem na Web sobre como fazer isso.



R Alguns editores de texto no Windows, como o Bloco de Notas, não conseguem ver arquivos HTML automaticamente. Escolha Todos os arquivos (ou uma opção semelhante), se necessário, para visualizar arquivos com qualquer extensão.



B Assim que os arquivos com qualquer extensão forem exibidos, você pode escolher o arquivo HTML desejado e clicar em Abrir.



C No Windows, você também pode clicar com o botão direito no ícone do documento ou nome do arquivo e escolher Editar ou Abrir com no menu pop-up que aparece. Em um Mac, clique com o botão direito no ícone, selecione Abrir com no menu pop-up e escolha o editor de texto desejado.

Editando páginas da web

Como as páginas da Web são visualizadas com mais frequência com um navegador da Web, quando você clica duas vezes nelas na área de trabalho, o navegador padrão abre e as exibe alegremente. Se quiser editar a página da Web, você terá que abri-la manualmente em seu editor de texto.

Para editar páginas da Web:

1. Abra seu editor de texto.
2. Escolha Arquivo > Abrir.
3. Navegue até o diretório que contém o arquivo desejado.
4. Se você não vir seu arquivo listado, escolha a opção Todos os Arquivos (ou descrição semelhante) (A e B). O nome e a localização podem variar ligeiramente de programa para programa e de plataforma para plataforma.
5. Clique em Abrir. Seu arquivo está pronto para edição.

TIP Depois de fazer alterações em um já documento salvo, geralmente você pode simplesmente escolher Arquivo > Salvar para salvar as alterações, sem se preocupar com o formato (conforme descrito em “Salvar sua página da Web”).

Organizando arquivos

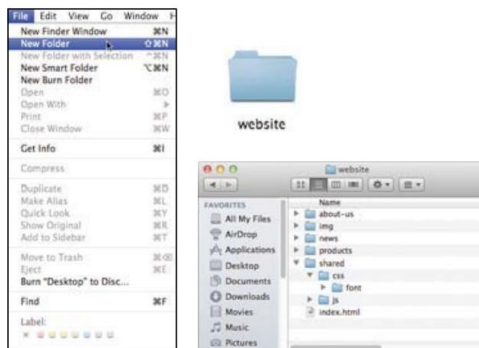
Antes de ter muitos arquivos, é uma boa ideia descobrir onde colocá-los. É habitual (mas não obrigatório) criar uma pasta para cada seção principal

dentro do seu site, permitindo agrupar páginas HTML relacionadas.

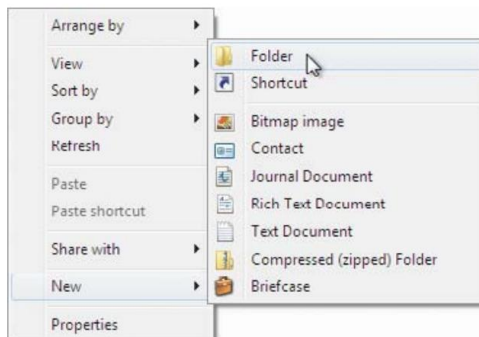
Para organizar seus arquivos:

1. Crie uma pasta ou diretório central para armazenar todo o material que estará disponível disponível em seu site. No Mac, escolha Arquivo > Nova pasta no Finder
 - A. No Windows, na área de trabalho (ou em uma pasta de sua escolha), clique com o botão direito e escolha Novo > Pasta B. Dê um nome para a pasta.
2. Crie subpastas de uma forma que reflita a organização do seu site (A e C). Por exemplo, você pode decidir criar uma pasta separada para cada seção do seu site, juntamente com subpastas individuais dentro delas, conforme necessário.
3. É comum criar uma pasta de nível superior para as imagens do seu site e, opcionalmente, adicionar subpastas para ajudá-lo a organizar suas imagens por seção ou outros critérios. Outra abordagem é criar uma pasta de nível superior chamada **Assets** (ou algo semelhante) e colocar sua pasta de imagens nela, junto com pastas para outros ativos, como vídeo, folhas de estilo e assim por diante. (Você aprenderá sobre folhas de estilo a partir do Capítulo 7.)

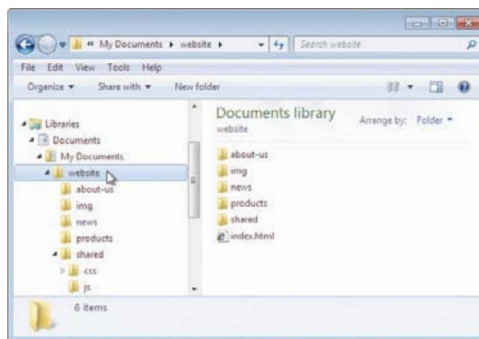
TIP Use nomes curtos e descritivos para seus arquivos e pastas, de preferência separando as palavras em um nome com um traço (não um espaço). Use todas as letras minúsculas para que seus URLs sejam mais fáceis de digitar e, assim, suas páginas sejam mais fáceis de acessar. Para obter mais detalhes sobre como criar bons nomes de arquivos, consulte “Nomes de Arquivos” no Capítulo 1.



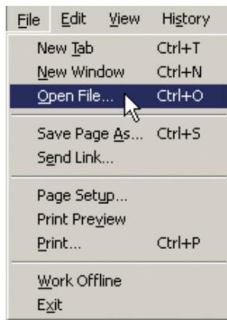
A Em um Mac, escolha Nova pasta e dê um nome à pasta. Crie uma pasta separada para cada seção do seu site.



B No Windows, na área de trabalho ou no Windows Explorer, clique com o botão direito e escolha Novo > Pasta.



C Você pode dividir a pasta em subpastas se necessário.



A No navegador desejado (este é o Firefox), escolha Arquivo > Abrir Arquivo. No Internet Explorer, chama-se Arquivo > Abrir.

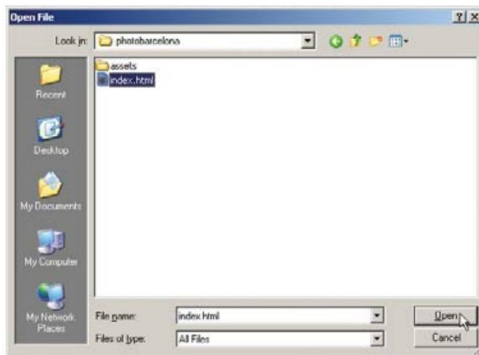
Visualizando sua página em um navegador

Depois de criar uma página, você desejará ver como ela fica em um navegador. Na verdade, como você não sabe qual navegador seus visitantes usarão (e os navegadores nem sempre renderizam as páginas exatamente da mesma maneira), é recomendável visualizar a página em vários navegadores.

Para visualizar sua página em um navegador:

1. Abra um navegador.
2. Escolha Arquivo > Abrir, > Abrir arquivo ou > Abrir página (mas não Abrir local), dependendo do navegador A.
3. Na nova caixa de diálogo exibida, navegue até a pasta em seu computador que contém a página da Web desejada, selecione a página e clique em Abrir B. A página é exibida no navegador C exatamente como aparecerá quando você realmente a publicar em seu servidor Web (consulte o Capítulo 21).

Estas etapas podem variar ligeiramente em diferentes navegadores.



B Escolha o arquivo que deseja abrir e clique em o botão Abrir.



C A página aparece no navegador. Verifique com atenção para ver se está saindo do jeito que você planejou.

TIP Você também pode (normalmente) clicar duas vezes em um Web ícone da página para visualizá-la em um navegador. Ou, se já tiver um navegador aberto, você pode arrastar o ícone ou o nome do arquivo e soltá-lo na janela do navegador. Geralmente, essa é a maneira mais fácil de visualizar uma página em um navegador, depois que você pega o jeito.

continua na próxima página

TIP Alguns navegadores modernos não possuem uma opção de menu equivalente a Arquivo > Abrir para abrir uma página. Experimente o método de arrastar e soltar descrito na dica anterior.

TIP Se a sua página da Web não aparecer na caixa de diálogo Abrir, certifique-se de salvá-la como somente texto e de ter fornecido a extensão .html ou .htm (consulte “Salvar sua página da Web”).

TIP Você não precisa fechar o documento no editor de texto antes de visualizá-lo em um navegador, mas é necessário salvá-lo. Se você fizer uma alteração na página em seu editor de texto depois de abri-la em um navegador, salve o arquivo novamente e use o botão recarregar do navegador para atualizar a página. (Você poderia seguir inicialmente as etapas usadas para visualizar a página no navegador, mas isso levaria mais tempo.)

TIP Seus visitantes não poderão visualizar seu Web site até publicá-lo em seu servidor Web (consulte o Capítulo 21).



R Todos os navegadores de desktop possuem um comando de menu que permite visualizar o código HTML de uma página. O nome varia de View Source para Page Source (no Firefox, mostrado) para nomes semelhantes. (No Chrome, é Ferramentas > Exibir código-fonte da página.)



B A maioria dos navegadores também permite que você clique com o botão direito na página e escolha o comando Exibir código-fonte (seja qual for o nome) no menu que aparece.

O Chrome é mostrado. Geralmente, essa é a maneira mais fácil de visualizar a fonte, pois pode ser difícil encontrar a opção no menu principal ou submenu.



C Os navegadores modernos exibem o código em sua própria guia ou janela (conforme mostrado), enquanto os navegadores mais antigos podem exibi-lo em um editor de texto específico.

As cores distinguem o conteúdo da página dos elementos HTML, atributos e valores de atributos. Isso é chamado de realce de sintaxe. Os números das linhas à esquerda não fazem parte do código HTML e nem todos os navegadores os mostram nos modos Visualizar código-fonte.

Eles são apenas um indicador que o Chrome incluiu em sua janela Exibir código-fonte.

A inspiração de outros

Uma das maneiras mais fáceis de expandir sua fluência em HTML é observar como outros desenvolvedores e designers criaram suas páginas. Felizmente, o código HTML é fácil de visualizar e aprender. No entanto, o conteúdo de texto, gráficos, sons, vídeos, folhas de estilo e outros arquivos externos podem estar protegidos por direitos autorais. Como regra geral, use páginas de outras pessoas como inspiração para o seu HTML e, a seguir, crie seu próprio conteúdo.

Para visualizar o código HTML de outros designers com View Source:

1. Abra uma página da Web com qualquer navegador.
2. Escolha Exibir código-fonte (ou a escolha apropriada para um navegador específico)
(A e B). O código HTML será exibido C.
3. Se desejar, salve o arquivo para estudo posterior.

Para visualizar o código HTML de outros designers com ferramentas de desenvolvedor:

Outra maneira de visualizar o código-fonte de uma página é com as ferramentas de desenvolvedor do navegador. As ferramentas são diferentes para cada fornecedor de navegador, mas todas possuem alguns recursos que se sobrepõem.

Essas ferramentas mostram uma visão mais interativa do código-fonte. Você pode inspecionar o HTML e CSS de partes específicas de uma página, editá-lo no navegador e ver as alterações refletidas na página imediatamente. E você pode usá-los em qualquer site, não apenas no seu. As alterações são temporárias – elas não sobrescrevem o HTML e CSS reais

arquiva a página carregada. Isso é valioso para o aprendizado, porque você pode ver como um determinado efeito foi alcançado ou mexer nele.

o código para ver o que acontece sem medo de danificar nada.

Consulte a barra lateral “Ferramentas de desenvolvedor de navegador” no Capítulo 20 para obter informações sobre as ferramentas de desenvolvedor de navegador para navegadores modernos e mais antigos.

TIP Não há regra sobre quem pode colocar um site na Web. Isso é o que há de tão bom nisso: é um meio aberto com uma barreira de entrada relativamente baixa. Você pode ser um novato, um especialista ou qualquer coisa entre os dois. Tenha isso em mente ao revisar o código de outros sites. Se parte do código parecer suspeito, não presuma que o autor sabe melhor do que você só porque o site dele está na Web. Há muitos sites que servem como ótimos exemplos de melhores práticas de codificação, e há muitos outros que são, digamos, menos que ideais. Portanto, mantenha um olhar crítico e verifique este livro e outros recursos quando tiver dúvidas sobre a adequação de uma técnica específica.

TIP Você também pode salvar o código-fonte copiando-o da janela Exibir código-fonte e colando-o em seu editor de texto. Então você pode salvar o arquivo.

TIP Você também pode salvar o código-fonte e normalmente muitos de seus ativos (como imagens) selecionando Arquivo > Salvar como (ou Arquivo > Salvar página como) na maioria dos navegadores. No entanto, o navegador pode reescrever partes do código ao salvar a página, portanto não será exatamente como se você o tivesse salvo usando a dica anterior.

TIP Para visualizar o CSS em uma página da Web, consulte “A inspiração de outros: CSS” no Capítulo 8.

3

HTML básico Estrutura

Este capítulo aborda os elementos HTML necessários para estabelecer a base e a estrutura de seus documentos. Ou seja, o esboço e os recipientes semânticos primários do seu conteúdo.

Você aprenderá sobre:

- n Iniciando uma página da Web
- n O esboço do documento HTML5
- n O **h1–h6**, **hgroup**, **cabeçalho**, **nav**, elementos **de artigo**, **seção**, **aparte**, **rodapé** e **div** (a maioria dos quais são novos em HTML5)
- n Como os atributos **de função** ARIA podem melhorar a acessibilidade da sua página
- n Aplicando uma **classe** ou **id** aos elementos
- n Aplicando o atributo **title** a elementos
- n Adicionando comentários ao seu código

Neste capítulo

Iniciando sua página da web	43
Criando um título	46
Criando títulos	48
Compreendendo o documento HTML5	
Contorno	50
Agrupando títulos	58
Construções de páginas comuns	60
Criando um cabeçalho	61
Marcação de navegação	64
Criando um Artigo	68
Definindo uma seção	72
Especificando um aparte	75
Criando um rodapé	80
Criando contêineres genéricos	84
Melhorando a acessibilidade com ARIA	88
Nomeando Elementos com uma Classe ou ID	92
Adicionando o atributo Title aos elementos	95
Adicionando comentários	96

Criar uma estrutura clara e consistente não apenas estabelece uma boa base semântica para sua página, mas também torna muito mais fácil aplicar estilos ao seu documento com Cascading Style Sheets (CSS) (a cobertura começa no Capítulo 7).

Se ainda não o fez, sugiro fortemente que leia o Capítulo 1 antes de continuar. Mostra uma página HTML simples e explica alguns dos conceitos básicos. Como esta é sua primeira visão de uma página da Web, repetirei algumas informações (mas não todas) aqui e presumirei que você esteja familiarizado com o restante para poder desenvolver essas ideias.

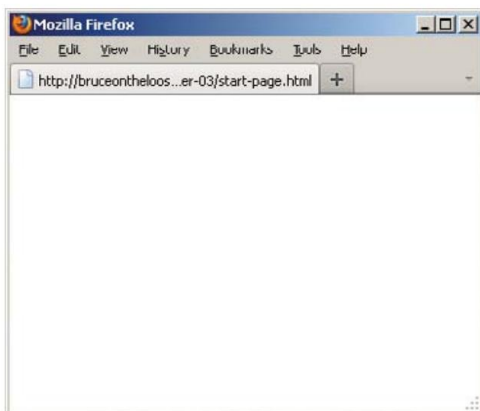
Além disso, se você leu meu livro The HTML Pocket Guide, parte deste material lhe será familiar.

R : Aqui está a base de toda página HTML.

O recuo não importa, mas a estrutura é crucial. Neste exemplo, o idioma padrão (de acordo com o atributo **lang**) é definido como **en** para inglês. A codificação de caracteres está definida como UTF-8.

```
<!DOCTYPEhtml>
<html lang="pt">
<cabeca>
  <meta charset="UTF-8" />
  <título></título>
</head>
<corpo>

</body>
</html>
```



B O código básico HTML mínimo conforme visualizado no Firefox. Como você pode ver, não há nada para ver! No entanto, você começará a adicionar conteúdo em breve.

Iniciando seu

Página da Internet

No seu nível mais básico, cada um dos seus documentos HTML deve conter os seguintes componentes, conforme mostrado em A:

n O DOCTYPE

n O elemento **html** (com o **lang** atributo, que é opcional, mas recomendado)

n O elemento **principal**

n A codificação de caracteres em um **meta** elemento

n O elemento **title** (você adicionará seu conteúdo em breve)

n O elemento **corpo**

Este é o equivalente HTML de uma folha de papel em branco, pois não possui nenhum conteúdo no **corpo** B.

Portanto, antes de adicionar qualquer conteúdo ou outra informação, você precisa configurar a base da sua página:

Para iniciar uma página HTML5:

1. Digite **<!DOCTYPE html>** para declarar sua página como um documento HTML5. (Consulte a barra lateral "DOCTYPE aprimorado do HTML5" para obter informações relativas às versões anteriores do HTML.)
2. Digite **<html lang="idioma-código">** para iniciar a parte HTML real do seu documento, onde **código do idioma** é o código do idioma que corresponde ao idioma padrão do conteúdo da sua página. Por exemplo, **<html lang="es">** para inglês ou **<html lang="fr">** para francês. Veja www.bruceontheloose.com/references/language-codes.html para obter uma lista de códigos de idiomas disponíveis.

language-codes.html para obter uma lista de códigos de idiomas disponíveis.

continua na próxima página

3. Digite **<head>** para iniciar o documento cabeçalho da sua página.
4. Digite **<meta charset="UTF-8" />** para declare a codificação de caracteres do seu documento como UTF-8. Se preferir, você também pode digitá-lo como **utf-8** em seu HTML.
Além disso, o espaço e a barra são opcionais, então **<meta charset="UTF-8">** funciona da mesma forma. (As codificações de caracteres além do UTF-8 também são válidas, mas o UTF-8 é o mais versátil, por isso é raro que você precise desviar-se.)
5. Digite **<título></título>**. Isso conterá o título da sua página. Você adicionará o texto do título na seção "Criando um título".
6. Digite **</head>** para finalizar o documento cabeçalho da sua página.
7. Digite **<body>** para iniciar o corpo do seu página. É para onde o seu conteúdo irá (eventualmente).
8. Deixe algumas linhas em branco para criar o conteúdo da sua página, o que você fará no restante deste livro.
9. Digite **</body>** para finalizar o corpo.
10. Digite **</html>** para finalizar sua página.

Esse é um número bastante saudável de etapas, mas como todas as suas páginas começarão dessa maneira, você pode usar uma única página HTML como modelo para iniciar cada página, evitando digitar. Na verdade, a maioria dos editores de código permite especificar o código inicial para cada nova página, tornando isso ainda mais fácil. Se você não encontrar o menu Configurações ou Preferências em seu editor, pesquise na seção Ajuda.

As duas seções de uma página: cabeça e corpo

Apenas como uma rápida recapitulação do que você aprendeu no Capítulo 1, as páginas HTML são divididas em duas seções: a **cabeça** e o **corpo**.

O DOCTYPE, que inicia cada página, é uma espécie de preâmbulo.

O **cabeçalho** do documento é onde você define o título da sua página, inclui informações sobre sua página para mecanismos de pesquisa como o Google, carrega folhas de estilo e ocasionalmente carrega arquivos JavaScript (embora, por motivos de desempenho, seja preferível na maioria das vezes para carregar o JavaScript logo antes da tag final **</body>** na parte inferior da sua página). Você verá exemplos disso à medida que avança no livro. Exceto pelo **título**, que você abordará mais adiante, o conteúdo do **cabeçalho** não fica visível para os usuários quando eles visitam sua página.

O elemento **body** inclui o conteúdo da sua página, incluindo texto, imagens, formulários, áudio, vídeo e outros conteúdos interativos.

Em outras palavras, o que seus visitantes veem. Existem vários capítulos dedicados aos elementos relacionados ao conteúdo do HTML, alguns dos quais você verá antecipadamente neste capítulo.

TIP O HTML5 DOCTYPE garante os navegadores são renderizados em um modo confiável e instruem os validadores HTML a julgar seu código em relação aos elementos e à sintaxe permitidos. Validadores HTML são discutidos no Capítulo 20.

TIP O DOCTYPE do HTML5 não faz distinção entre maiúsculas e minúsculas positivo. Por exemplo, alguns optam por digitá-lo como `<!doctype html>`, mas provavelmente é mais comum usar `<!DOCTYPE html>` A.

TIP O elemento `html`, que segue o DOCTYPE, deve incluir todos os outros elementos da sua página A.

TIP Certifique-se de que seu editor de código esteja configurado para salvar arquivos como UTF-8 para corresponder à codificação de caracteres especificada no código por `<meta charset="UTF-8" />` A. (Ou se você especificou um conjunto de caracteres diferente, salve seus arquivos nele.) Nem todos os editores salvarão suas páginas como UTF-8 por padrão, mas a maioria permite que você para escolher a codificação em um menu ou painel (veja “Salvar suas páginas da Web” no Capítulo 2). Sem a configuração UTF-8 em vigor, você poderá ocasionalmente ver caracteres engraçados em seu conteúdo, em vez de uma letra pretendida, como um i acentuado ou um n com um til (~).

TIP Você não precisa recuar o código que está aninhado no elemento `head` A. No entanto, a vantagem de fazer isso é que você pode ver rapidamente onde o `head` começa, o que há nele e onde termina. Não é incomum que o cabeçalho fique muito longo em algumas páginas.

DOCTYPE aprimorado do HTML5

Ah, como é mais simples iniciar sua página da Web agora que o HTML5 está aqui. O DOCTYPE do HTML5 é refrescantemente curto, especialmente quando comparado aos DOCTYPEs de antigamente.

Na época do HTML 4 e do XHTML 1.0, havia vários DOCTYPEs para escolher, cada um significando a versão do HTML e se estava no modo Transicional ou Estrito. Você invariavelmente tinha que copiá-los de algum outro lugar porque eles eram complicados demais para serem lembrados.

Por exemplo, aqui está o DOCTYPE para documentos XHTML Strict.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Gobbledygook.

Felizmente, todos os navegadores - antigos e novos - entendem o DOCTYPE do HTML5, então você pode segui-lo em todas as suas páginas e esquecer que as outras existiram. (A única ocasião em que eles podem ser relevantes é se você herdar um site antigo e o proprietário não permitir que você altere o DOCTYPE para a versão HTML5.)

Criando um título

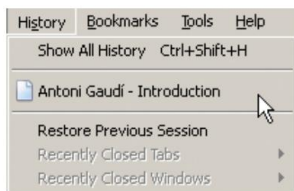
O código HTML básico na seção anterior tinha `<title></title>` como espaço reservado até a hora de discutir o **título**

avançar. Agora é a hora!

Cada página HTML deve ter um **título** elemento. Um título deve ser curto, descritivo e exclusivo para cada página A. Na maioria dos navegadores, o título aparece na barra de título da janela (o Chrome é uma exceção)
B. Ele também aparece na aba da página em navegadores que suportam navegação por abas – em outras palavras, todos os principais navegadores lançados nos últimos anos. O título também aparece nas listas de histórico do navegador e nos favoritos dos visitantes C.

Talvez ainda mais importante, o título é usado por mecanismos de busca como Google, Bing e Yahoo! tanto para ter uma ideia do conteúdo da sua página quanto, normalmente, como o link que aparece nos resultados da pesquisa D.

Resumindo, torne seu **título** exclusivo para cada página para melhorar os resultados dos mecanismos de pesquisa e melhorar a experiência dos visitantes.

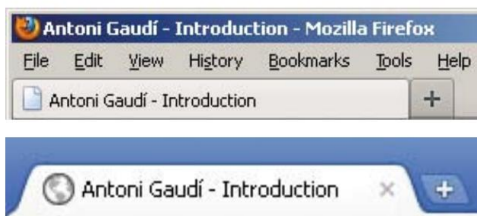


C O título também aparece no painel Histórico do visitante (mostrado), na lista de Favoritos e na lista de Favoritos.

A O elemento **title** deve ser colocado na seção **head**. Coloque-o após o **metaelemento** que especifica a codificação de caracteres.

```
<!DOCTYPEhtml>
<html lang="pt">
<cabeca>
    <meta charset="UTF-8" />
    <title>Antoni Gaudí - Introdução
</title>
</head>
<corpo>

</body>
</html>
```



B Na maioria dos navegadores, como o Firefox, o título de uma página da Web é exibido na barra de título da janela e na guia. No entanto, o Chrome (parte inferior) exibe o título apenas na guia.



D Talvez o mais importante seja que o título normalmente é usado como o texto do link que aponta para sua página nos resultados de pesquisa do Google e de outros. É também um fator importante para determinar a relevância de uma página nos resultados de pesquisa. Aqui você vê um título e uma cópia parcial do corpo aparecendo no Google.

Uma análise mais aprofundada dos títulos das páginas

Muitos desenvolvedores – mesmo os bem-intencionados e bastante experientes – dão pouca consideração ao elemento **do título**.

Eles simplesmente inserirão o nome do site e o copiarão em todas as páginas HTML.

Ou pior ainda, eles vão deixar o **título**

texto que seu editor de código pode inserir por padrão.

Se um de seus objetivos é direcionar tráfego para seu site, você estaria prestando um enorme desserviço a si mesmo e a seus leitores em potencial ao seguir o exemplo.

Os motores de busca possuem diferentes algoritmos que determinam a classificação de uma página e como seu conteúdo é indexado. Universalmente, porém, o **título** desempenha um papel fundamental. Os mecanismos de pesquisa podem olhar para o **título** em busca de uma indicação do assunto de uma página e indexar o conteúdo de uma página em busca de texto relacionado. Um **título** eficaz concentra-se em alguns

de palavras-chave que são centrais para o conteúdo de uma página contente.

Como prática recomendada, escolha um texto **de título** que resuma brevemente o conteúdo de um documento para beneficiar tanto o leitor de tela usuários e as classificações do seu mecanismo de pesquisa. Secundariamente, e opcionalmente, indique o nome do seu site no **título**. É comum

veja o nome de um site no início do **título**, mas é melhor colocar o texto **do título** exclusivo e específico da página no início.

Eu recomendo que você coloque a mensagem central **do seu título** nos primeiros 60 caracteres, incluindo espaços, porque os mecanismos de pesquisa geralmente os cortam em seus resultados em torno desse número (como linha de base).

Os navegadores exibem um número variável de caracteres, mas não mais que 60, na barra de título na parte superior do navegador antes de cortar o texto. As guias do navegador cortam o título ainda mais cedo porque há menos imóveis.

Para criar um título:

1. Coloque o cursor entre `<título>` e `</title>` no **cabeçalho do documento**.
2. Digite o título da sua página da Web.

TIP O elemento **title** é obrigatório.

TIP Um título não pode conter qualquer formatação, HTML, imagens ou links para outras páginas.

TIP Alguns editores de código preenchem previamente o título com texto padrão quando você inicia uma nova página, a menos que você tenha instruído a usar um código inicial específico, conforme descrito em “Iniciando sua página da Web”. Portanto, fique atento a isso e certifique-se de substituir qualquer texto padrão por um título de sua própria criação.

TIP Se o seu título contém caracteres especiais como acentos ou alguns símbolos, você terá que torná-los parte da sua codificação (o que normalmente não será um problema se você estiver usando UTF-8) ou escrevê-los com referências (veja a lista de referências de entidades de personagem disponíveis em www.elizabethcastro.com/html/extras/entidades.html). Além disso, não se esqueça de configurar seu editor de código para salvar suas páginas com a codificação adequada, como UTF-8, para que os caracteres especiais sejam salvos corretamente (veja “Salvando suas páginas da Web” no Capítulo 2).

Criando títulos

HTML fornece seis níveis de títulos para estabelecer a hierarquia de informações em suas páginas. Marque cada título com um dos elementos h1–h6, onde **h1** é um título de nível superior, **h2** é um subtítulo de **h1**, **h3** é um subtítulo de um **h2** e assim por diante.

Os títulos estão entre os elementos HTML mais importantes em qualquer página, como você aprenderá.

Pense nos títulos h1–h6 como semelhantes aos títulos de um documento não HTML que você pode escrever, como um relatório de vendas, trabalho de conclusão de curso, manual de produto, artigo de notícias – essa é a ideia. Ao escrever esses tipos de documentos, você identifica cada seção principal do conteúdo com um título e qualquer número de subtítulos (e subsubtítulos, e assim por diante), conforme apropriado. Coletivamente, esses títulos representam o esboço do documento. O mesmo se aplica às suas páginas da Web. A. Discuto isso com mais profundidade na próxima seção, “Compreendendo o esboço do documento HTML5”.

Para organizar sua página da Web com títulos:

1. Na seção **corporal** do seu documento HTML, digite **<h*n*>**, onde ***n*** é um número de 1 a 6, dependendo do nível de importância do título que você deseja criar. **h1** é o mais importante e **h6** é o menos importante.
2. Digite o conteúdo do cabeçalho.
3. Digite **</h*n*>** onde ***n*** é o mesmo número usado na etapa 1.

R Use títulos para definir a estrutura do seu documento, assim como um esboço. Aqui, “La Casa Milà” e “La Sagrada Família” – marcados como elementos **h2** – são subtítulos do título de nível superior, “Antoni Gaudí”, porque é um **h1**. (A parte **lang="es"** indica que o conteúdo está em espanhol; não afeta o contorno.) Se “La Sagrada Família” fosse um **h3**, então seria um subtítulo de “La Casa Milà” (e um sub-subtítulo de “Antoni Gaudí”). A linha em branco entre cada título é totalmente opcional e não afeta a exibição do conteúdo. Se eu estivesse codificando o resto da página agora, o conteúdo relacionado (parágrafos, imagens, vídeo e assim por diante) seguiria cada título. Você verá exemplos disso nas páginas subsequentes.

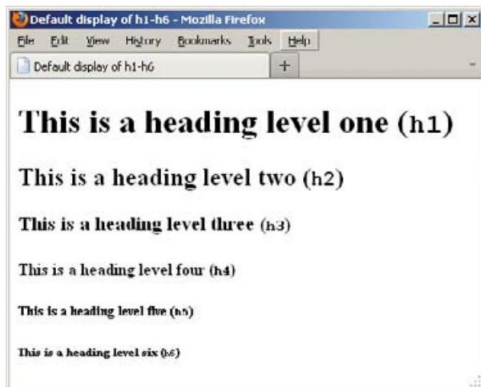
```
<!DOCTYPEhtml>
<html lang="pt">
  <cabeca>
    <meta charset="UTF-8" />
    <title>Antoni Gaudí - Introdução
  </title>
</head>
<corpo>

  <h1>Antoni Gaudí</h1>

  <h2 lang="es">La Casa Milà</h2>

  <h2 lang="es">A Sagrada Família</h2>

</body>
</html>
```



B Embora todos os títulos sejam exibidos em negrito por padrão, **h1** está em uma fonte maior que **h2**, que é maior que **h3**, e assim por diante. Mas como você sabe, a aparência não é relevante ao decidir qual nível de título usar. Você pode alterar a apresentação com CSS.

TIP Seus títulos h1–h6 são especialmente importantes devido ao seu impacto na definição do contorno da sua página. Por padrão, os navegadores exibem títulos progressivamente menores, passando de h1 para h6. Mas não se esqueça de escolher os níveis de título apenas com base na hierarquia apropriada para o seu conteúdo, e não no tamanho que você deseja que o texto apareça. Isso torna sua página semanticamente mais forte, o que, por sua vez, melhora o SEO e a acessibilidade. Você pode estilizar os títulos como desejar com uma fonte, tamanho, cor específicos e muito mais. Para obter detalhes sobre como conseguir isso com CSS, consulte o Capítulo 10.

TIP Os mecanismos de pesquisa pesam muito em seus títulos, especialmente aqueles como h1 (o que não quer dizer que carregue sua página com h1s; os mecanismos de pesquisa são sábios quanto a isso). Enquanto isso, os usuários de leitores de tela geralmente navegam em uma página pelos títulos por meio do teclado, pois isso lhes permite avaliar rapidamente o conteúdo de uma página e encontrar o que mais lhes interessa sem ter que ouvir a página inteira. Mais uma razão para ter uma hierarquia lógica de títulos.

TIP Use os níveis de título de forma consistente em todo o site para uma melhor experiência do usuário.

TIP Você pode adicionar um id a um título se quiser criar um link diretamente para ele (veja “Criando Âncoras” no Capítulo 6).

TIP Como observação lateral, em [A](#) usei a linguagem atributo em cada h2 para indicar que seu conteúdo está em um idioma diferente (espanhol, representado pelo código de idioma es) do padrão da página (inglês) conforme declarado por `<html lang="en">`.

Entendimento Documento HTML5

Contorno

Na seção anterior, você aprendeu que os elementos de títulos, **h1–h6**, contribuem para o esboço da sua página HTML. Você vai se aprofundar mais nesta seção para aprender como alguns elementos exclusivos do HTML5 também afetam o contorno.

Uma nota sintática antes de continuar.

Na discussão e nas páginas a seguir, usarei frequentemente “seção” como um termo genérico para significar uma parte distinta de uma página, em oposição ao elemento **de seção** (sobre o qual você aprenderá) especificamente. Quando me refiro ao elemento **de seção**, a palavra terá o estilo de código, assim como nesta frase.

OK, continuando.

Então, você sabe que cada documento HTML tem um esboço subjacente, que é como um índice analítico, conforme definido pelos elementos do título. Agora, o esboço não é algo exibido explicitamente em sua página - embora os navegadores possam um dia fornecer um meio de vê-lo - mas, como acontece com toda semântica, é significativo para mecanismos de pesquisa e leitores de tela, que usam o esboço para coletar a estrutura da sua página e forneça as informações aos usuários.

Um esboço do documento da Versão 1

```
...
<corpo>
  <h1>Guia do usuário do produto</h1>
  <h2>Configurando</h2>
  <h2>Recursos básicos</h2>
  <h3>Reprodução de vídeo</h3>
  <h2>Recursos Avançados</h2>
</body>
</html>
```

Esboço do documento B Versão 2 (mesmo esboço do Versão 1, mas com marcação mais significativa)

```
...
<corpo>
  <h1>Guia do usuário do produto</h1>
  <seção>
    <h1>Configurando</h1>
  </seção>
  <seção>
    <h1>Recursos básicos</h1>
    <section> <!-- aninhado, então é um
    ↳ subseção de seu pai ->
    <h1>Reprodução de vídeo</h1>
  </seção>
  <seção>
    <h1>Recursos avançados</h1>
  </seção>
</body>
</html>
```

Nas versões de HTML e XHTML que

precedeu o HTML5, os elementos de título h1–h6 eram tudo o que você precisava para estruturar o esboço. O HTML5, por outro lado, inclui quatro elementos de conteúdo de seção – **artigo**, **aparte**, **navegação** e **seção** – que demarcam seções distintas.

dentro de um documento e defina o escopo dos elementos h1–h6 (bem como **cabeçalho** e **rodapé**) dentro deles.

Isso significa que cada elemento de seccionamento tem sua própria hierarquia h1–h6, o que é uma grande mudança em relação às versões anteriores da linguagem. Além disso, não só há mais de um **h1** em uma página OK, geralmente é recomendado pela especificação HTML5 (no entanto, mantenha esse pensamento; em breve explicarei por que você deve limitar seus h1s).

Tudo isso afeta o contorno. Vamos comparar dois esboços equivalentes para ver como isso funciona. Para ambos, imagine que cada título é seguido por uma série de parágrafos e outros conteúdos que representam as informações da seção.

O primeiro esboço, que é HTML5 perfeitamente válido e será familiar para aqueles com experiência em HTML e XHTML, usa apenas elementos de título A.

A segunda versão B usa títulos e elementos **de seção** do HTML5, incluindo uma **seção aninhada**.

(Observação: o recuo do código não é importante e não afeta o contorno, mas deixa claro quais elementos estão contidos em outros.)

Anteriormente, mencionei que os navegadores ainda não expõem o esboço para você. No entanto, você pode verificar isso com o HTML 5 Outliner de Geoffrey Sneddon (<http://gsnedders.html5.org/outliner/>), uma ferramenta simples, mas excelente, que apresenta uma representação visual do esboço do seu documento. Usando-o para gerar contornos para as versões 1 e 2 (A e B) mostra que, embora seus níveis de cabeçalho h1–h6 sejam diferentes, ambos resultam neste esboço:

1. Guia do usuário do produto

1. Configurando

2. Recursos básicos

1. Reprodução de vídeo

3. Recursos avançados

Como você pode ver, cada elemento **de seção** na Versão 2 B se torna uma subseção de seu h1–h6 mais próximo ou ancestral de conteúdo de seção (que também é **seção**, neste caso). O mesmo comportamento se aplica a todos os quatro elementos de conteúdo de seccionamento HTML5 que mencionei anteriormente (**artigo, aparte, navegação e seção**), mesmo quando eles estão misturados.

Em comparação, se a Versão 2 B não tivesse **seções – vamos** chamá-la de Versão 3 C – seu esboço seria muito diferente.

Ou seja, cada título estaria no mesmo nível de importância, **h1**, o que significa que não haveria subtítulos (ou subtítulos):

1. Guia do usuário do produto

2. Configurando

3. Recursos básicos

4. Reprodução de vídeo

5. Recursos avançados

Esboço do documento C Versão 3 (não o mesmo esboço das Versões 1 e 2)

```
...
<corpo>
  <h1>Guia do usuário do produto</h1>
  <h1>Configurando</h1>
  <h1>Recursos básicos</h1>
  <h1>Reprodução de vídeo</h1>
  <h1>Recursos avançados</h1>
</body>
</html>
```

D Exemplo de esboço com semântica explícita

```

...
<corpo>
  <artigo>
    <h1>Guia do usuário do produto</h1>
    <seção>
      <h1>Configurando</h1>
    </seção>

    <seção>
      <h1>Recursos básicos</h1>
      <seção>
        <h1>Reprodução de vídeo</h1>
      </seção>
    </seção>

    <seção>
      <h1>Recursos avançados</h1>
    </seção>
  </artigo>
</body>
</html>

```

Dos esboços com o mesmo significado (ou seja, Versões 1 e 2), ambos são válidos HTML5, mas o segundo é preferível porque os elementos da **seção** são mais

explícito semanticamente. Na prática, você poderia envolver um elemento **de artigo** em todo o conteúdo da Versão 2, já que isso é ainda mais apropriado neste contexto (embora o esboço resultante seja um pouco diferente).

A Figura D mostra um exemplo.

Não se esqueça também de que cada título será seguido por textos, imagens e outros conteúdos relacionados, sobre os quais você aprenderá à medida que avança. Mas, por enquanto, deixe tudo isso de fora para que você possa se concentrar em aprender sobre títulos e contornos.

Fazendo o que puder no ecossistema atual

Mas espere! Há outro ajuste que você deve fazer no código. Lembre-se de quando

Eu disse para “manter esse pensamento; em breve vou explicar porque você deve limitar seus h1s”?

Embora seja verdade que cada elemento de conteúdo da seção (**artigo, aparte, navegação e seção**) possa começar sua hierarquia de títulos com **h1**, isso não é obrigatório. Na verdade, em um futuro próximo, é melhor começar cada um com um **h2** ou menos se eles representarem um subtítulo de um **h1** já existente para um pedaço de conteúdo relacionado.

Aqui está o porquê.

Existem muitas partes móveis no mundo em constante evolução da Web. Novas especificações como HTML5 mudam diariamente até serem finais (o que leva anos e ainda não aconteceu com HTML5). Novas versões de navegadores são lançadas. Novas versões da tela

leitores e outras tecnologias assistivas são desenvolvidas e lançadas. Nada disso acontece em perfeita sincronidade.

Em vez disso, cada navegador tende a adicionar recursos de forma incremental (em geral, uma coisa muito boa), e não necessariamente os mesmos que seus concorrentes (não tão bons), e certamente não no mesmo cronograma que seus concorrentes. O mesmo vale para leitores de tela. Portanto, embora os navegadores modernos suportem muitos recursos do HTML5, nenhum deles, no momento em que este livro foi escrito, expunha o esboço do documento HTML5 aos leitores de tela, e os leitores de tela não o expunham aos usuários.

Resumindo, isso significa que os leitores de tela e outras tecnologias assistivas ainda não distinguem entre um **h1** que está diretamente no **corpo** e um **h1** que fica dentro de um **artigo**, **à parte**, **navegação** ou **seção**. Eles são todos **h1s** de nível superior em sua perspectiva. Bruce Lawson, notável evangelista da Web do Opera, foi a primeira pessoa que conheço a apontar isso (www.brucelawson.co.uk/2009/titulos-em-html-5-e-acessibilidade/; esteja ciente de que algumas outras informações em este URL estão desatualizadas porque as especificações mudaram desde então. Veja também seu mais recente *Introducing HTML5*, que ele escreveu com Remy Sharp).

Enquanto isso, os usuários de leitores de tela não podem se dar ao luxo de esperar que a Web atenda às suas necessidades. Eles continuarão a aproveitar os títulos para obter uma visão geral do conteúdo de uma página e para navegar nas páginas como fazem regularmente. Uma hierarquia significativa de títulos torna isso mais fácil e proporciona uma melhor experiência para eles em seu site.

E Versão 4 (a abordagem recomendada para títulos de todas as quatro versões)

```
...
<corpo>
<artigo>
  <h1>Guia do usuário do produto</h1>
  <seção>
    <h2>Configurando </h2>
  </seção>

  <seção>
    <h2>Recursos básicos</h2>
    <seção>
      <h3>Reprodução de vídeo</h3>
    </seção>
  </seção>

  <seção>
    <h2>Recursos Avançados</h2>
  </seção>
</artigo>
</body>
</html>
```

Portanto, até que o ecossistema se atualize um pouco, você e seus usuários estarão em melhor situação se usarem títulos que indiquem explicitamente a hierarquia com **h1–h6**, assim como fariam se os elementos de seccionamento não estivessem presentes. Law-son e outros na área recomendam isso, e eu também recomendo essa abordagem, inclusive eu.

Vamos ver como fazer E.

O que costumava ser **h1s** no primeiro nível dos elementos **da seção** agora são h2s, e o cabeçalho de reprodução de vídeo que está em uma **seção** aninhada em outra **seção** é agora um **h3** em vez de um **h1**. O esboço do documento não mudou, apenas os níveis de título.

Este exemplo demonstra apenas **h1–h3**, mas use h4–h6 também em suas páginas se seu conteúdo exigir isso. Por exemplo, um subtítulo de Reprodução de Vídeo seria um **h4** (com ou sem um elemento de seccionamento pai, conforme achar adequado) e assim por diante.

E lembre-se, esta recomendação vale para todos os elementos de seccionamento – artigo, **aparte**, **navegação** e **seção** – **não** apenas aqueles mostrados no exemplo.

Embrulhando-o

Sugiro reler esta discussão sobre o esboço do documento HTML5 se alguma coisa não for bem compreendida. Não é tão complicado quanto pode parecer. Eu recomendo fortemente que você crie uma variedade de páginas de teste e compare os resultados no HTML5 Outliner para ter uma ideia melhor de como o algoritmo de estrutura de tópicos funciona. Use o Outliner também durante o trabalho do projeto, para garantir que sua estrutura esteja conforme planejado. Primeiro, certifique-se de validar suas páginas HTML5 em <http://validator.nu/> ou <http://validator.w3.org/> para verificar se há erros de codificação (consulte “Validando seu código” no Capítulo 20).

TIP Por favor, não fique com a impressão de que você sempre deve usar um artigo, ou que uma seção deve sempre (e somente pode) ser aninhada em um artigo. O exemplo que discutimos era apenas uma forma de usar esses elementos e, na verdade, esse mesmo conteúdo poderia ser marcado de algumas maneiras diferentes e ainda seria HTML5 válido. Explicarei o artigo e a seção mais adiante neste capítulo e, como você verá, eles têm algumas aplicações, dependendo do seu conteúdo.

Como o algoritmo Outline do HTML5 ajuda no conteúdo distribuído

Se você acompanhou até agora, viu que - em virtude de estar seccionando o conteúdo e as regras do algoritmo de esboço do HTML5 - cada **artigo**, **aparte**, **navegação** e **seção** tem seu próprio esboço que pode começar com **h1** e continue até **h6**.

Além de permitir muita flexibilidade com os títulos do seu documento, isso tem um benefício não tão óbvio: permite que seu conteúdo apareça em outras páginas, até mesmo em outros sites, sem causar estragos no esboço do documento pai. E o seu contorno também permanece intacto.

Hoje em dia, o conteúdo é compartilhado entre sites mais do que nunca. Você tem sites de agregação de notícias, blogs com feeds RSS, feeds do Twitter e assim por diante. Como você aprenderá em “Criando um Artigo”, o **artigo**

O elemento representa uma composição independente que pode ser distribuída (não deve ser, apenas que seria apropriado fazê-lo).

Imagine que o seguinte **artigo** de um site seja exibido em outro site:

...

<h2>Notícias de toda a Web</h2>

<artigo>

<h1>Adolescente local prefere vinil ao digital</h1>

<p>Uma adolescente local substituiu todas as suas faixas digitais por vinil. “É y legal”, disse ela, oficialmente.</p>

<h2>Fisgado pelo primeiro álbum</h2>

...

</artigo>

...

Verificando o código com o HTML5 Outliner, você verá isto:

1. Notícias de toda a web

1. Adolescente local prefere vinil em vez de digital

1. Viciado após o primeiro álbum

Portanto, mesmo que o título Local Teen tenha uma classificação mais alta (**h1**) do que o **h2** abaixo, é um subtítulo do **h2** porque está contido em um **artigo** sob esse título. E o Hooked **h2** é um sub-subtítulo do News **h2**, não em pé de igualdade.

O título Notícias pode ser **h3**, **h4** ou qualquer nível de título, e o contorno seria exatamente o mesmo. O mesmo se aplica a Local Teen e Hooked, desde que Local Teen tenha a classificação mais alta.

Agrupando títulos

Às vezes, um título tem vários níveis consecutivos, como subtítulos, títulos alternativos ou slogans.

Agrupá-los em um elemento **hgroup** indica que eles estão relacionados. Cada **hgroup** pode conter apenas dois ou mais títulos h1–h6 ; nenhum outro elemento é permitido.

Apenas a primeira instância do título com classificação mais alta em um **hgroup** aparece no esboço do documento (consulte “Compreendendo o esboço do documento HTML5”). Então esse pode ser outro fator decisivo para você ao escolher usar o **hgroup**. Para ficar claro, porém, todos os títulos em um **hgroup** são exibidos no navegador B.

A Dois títulos relacionados são agrupados.

Neste exemplo, o **h2** é um subtítulo do título **h1** do artigo . Por estar marcado com o título de classificação mais alta, apenas “Giraffe Escapes from Zoo” aparece no esboço do documento, mas ambos são exibidos no navegador, como esperado B. Da mesma forma, se um segundo **h1** aparecesse depois dele no **hgroup**, ele seria omitido do contorno, assim como **h2**. Além disso, como **h2** não aparece no esboço, o próximo título do artigo poderia ser **h2** (em vez de **h3**) e deve ser entendido como um subtítulo direto de **h1**, “Girafa escapa do zoológico”.

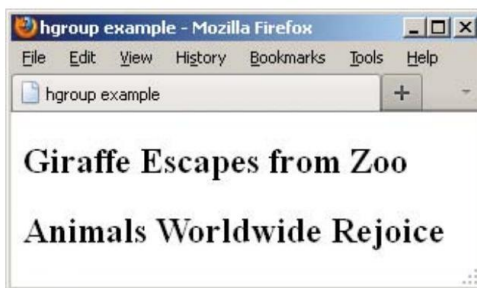
```
...
<corpo>

<artigo>

  <hgrupo>
    <h1>Girafa escapa do zoológico</h1>
    <h2>Animais em todo o mundo se alegram</h2>
  </hgrupo>

  <p>... [conteúdo do artigo] ...</p>
</artigo>

</body>
</html>
```



B Ambos os títulos são exibidos no navegador exatamente como seriam se não estivessem contidos em um **hgroup**.

Para agrupar dois ou mais títulos:

1. Digite `<hgrupo>`.
2. Digite `<hn>`, onde *n* é um número de 1 a 6, dependendo do nível de importância do título que se deseja criar.
3. Digite o conteúdo do cabeçalho.
4. Digite `</hn>` onde *n* é o mesmo número usado na etapa 2.
5. Repita as etapas 2 a 4 para quantos títulos deseja que façam parte do **hgroup**. Normalmente, o nível de título para cada título subsequente aumentaria em um (por exemplo, de **h1** para **h2** e assim por diante).
6. Digite `</hgroup>`.

TIP Não use hgroup em torno de apenas um título. Destina-se a pelo menos dois.

TIP Como mencionado, apenas a primeira instância de o título de classificação mais alta em um hgroup aparece no esboço do documento. A ordem dos títulos é irrelevante. Então, se o seu hgroup tivesse um h3 seguido por um h2, o h2 estaria no contorno. Normalmente, você ordenará os títulos por nível de prioridade, portanto, um de classificação inferior (como h3) não precederia um de classificação mais alta (como h2). Você pode encontrar exceções ocasionais.

Página Comum

Construções

Sem dúvida você já visitou dezenas de sites organizados como o mostrado em A. Retirando o conteúdo, você pode ver que existem quatro componentes principais: um cabeçalho com navegação, um artigo na área de conteúdo principal, um barra lateral com informações tangenciais e rodapé B.

Agora, você não pode estilizar uma página como esta A ou organizá-la conforme mostrado (A e B) sem CSS. Você começará a aprender CSS no Capítulo 7, verá como formatar texto e adicionar cores começando no Capítulo 10, e faça um layout de múltiplas colunas no Capítulo 11.

No entanto, a semântica que se aplica a essas construções de páginas comuns é bastante semelhante, independentemente do layout. Você os explorará na maior parte das páginas restantes deste capítulo. Trabalhando do topo da página para baixo, você verá como usar os elementos **de cabeçalho, navegação, artigo, seção, lado** e **rodapé** para definir a estrutura

de suas páginas e como usar **div** como um contêiner genérico para estilos adicionais e outros fins. Exceto o **div**, nenhum desses elementos existia até o HTML5.

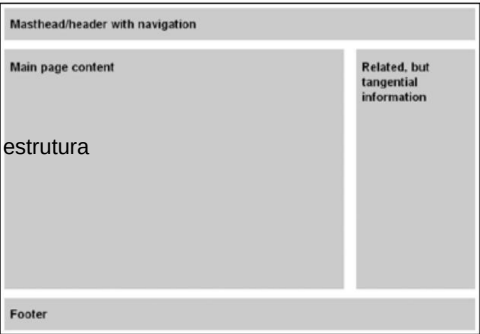
Você já teve uma ideia de alguns deles em exemplos de código e discussões anteriores.

À medida que você aprende sobre esses elementos, não se apegue muito ao local onde eles são exibidos nos layouts de amostra e, em vez disso, concentre-se em seu significado semântico.

Nas páginas seguintes, você também verá alguns outros elementos, como **ul** (lista não ordenada) e **a** (para links). Isso será explicado adequadamente em capítulos posteriores.



A Um layout comum com navegação principal na parte superior, conteúdo principal à esquerda, uma barra lateral à direita e rodapé na parte inferior. CSS é necessário para deixar a página assim.



B Os tipos de informações comumente encontrados em uma página. Este é apenas um tipo de arranjo, embora um comum.

A Este **cabeçalho** representa o cabeçalho de toda a página. Ele contém uma lista de links em um **navegador** elemento para indicar que é um conjunto primário de navegação na página. Consulte **C** para obter um exemplo de aplicação do opcional **role="banner"** a um **cabeçalho** de nível de página para fins de acessibilidade. (Consulte "Marcando Navegação" para obter o valor da **função** específico do elemento **nav**.)

```

...
<corpo>
<b>cabeçalho</b>
  <nenhuma>
    <ul>
      <li><a href="#gaudi">Arquiteto & de
        Barcelona </a></li>
      <li lang="es"><a href="#sagrada"> &
        familia">A Sagrada Família</a> & </li>
      <li><a href="#park-guell">Parque &
        Guell</a></li>
    </ul>
  </nenhuma>
</b>cabeçalho</b>
</body>
</html>

```



B O cabeçalho no nível da página que contém a navegação.

Criando um cabeçalho

Se uma seção da sua página tiver um grupo de conteúdo introdutório ou de navegação, marque-o com o elemento **de cabeçalho**.

Uma página pode ter qualquer número de **cabeçalhos** elementos, e seu significado pode variar dependendo do contexto. Por exemplo, um **cabeçalho** no topo de uma página ou próximo a ele pode representar o cabeçalho (às vezes chamado de masthead) de toda a página A. Muitas vezes, o cabeçalho da página inclui o logotipo do site, a navegação principal B, outros links globais e até mesmo uma caixa de pesquisa. Sem dúvida, este é o uso mais comum do elemento **de cabeçalho**, mas não o confunda com o único.

Um **cabeçalho** também seria apropriado para marcar um grupo de conteúdo introdutório ou de navegação mais profundamente em uma página. Um exemplo é o índice da seção **C** (na próxima página).

O elemento **de cabeçalho** é um dos quatro seccionando os elementos de conteúdo que mencionei em "Compreendendo o esboço do documento HTML5". Isso significa que qualquer cabeçalho h1–h6 dentro de um **cabeçalho** é considerado dentro do contexto do **cabeçalho** – e não da página como um todo – no que diz respeito ao esboço do documento. Portanto, um **cabeçalho** geralmente inclui o título de sua seção (h1–h6 ou **hgroup**), mas isso não é obrigatório. Por exemplo, você vê títulos em **C**, mas não em A.

Para criar um cabeçalho:

1. Coloque o cursor dentro do elemento para qual você deseja criar um cabeçalho.
2. Digite **<cabeçalho>**.
3. Digite o conteúdo do cabeçalho, que pode incluir uma variedade de tipos de conteúdo marcados com seus respectivos elementos HTML (a maioria dos quais você aprenderá

C Esta página tem dois **cabeçalhos**: um que serve como cabeçalho de toda a página e outro como cabeçalho para o elemento principal do **artigo** Perguntas frequentes . Observe que o primeiro não tem títulos h1–h6 , mas o segundo tem. Consulte a última dica desta seção para obter informações sobre o atributo **role** opcional mostrado no primeiro **cabeçalho**.

```
...
<body>
<header role="banner"> ...
    [logotipo do site, navegação, etc.] ... </header>

<artigo>
    <cabeçalho>
        <h1>Perguntas frequentes</h1> <nav>

            <ul>
                <li><a href="#answer1">Qual é a sua política de devolução?</a></li>
                <li><a href="#answer2">Como encontro um local?</li>
                ...
            </ul>
        </nav>
    </header>

    <!-- os links do cabeçalho apontam para estes -->
    <article id="answer1">
        <h2>Qual é a sua política de devolução?
        </h2> <p> ... [resposta] ...
    </p> </article>

    <article id="answer2">
        <h2>Como encontro um local?</h2>
        <p> ... [resposta] ... </p> </article>
    ...
</article> <!-- fim do artigo pai -->

</body>
</html>
```

sobre no resto do livro). Por exemplo, um **cabeçalho** pode conter h1–h6 títulos, um logotipo ou série de logotipos, navegação, uma caixa de pesquisa e muito mais.

4. Digite `</header>`.

TIP Não use o cabeçalho desnecessariamente.

Se tudo o que você tem é um h1–h6 ou um hgroup e nenhum conteúdo complementar digno de ser agrupado com ele, não há necessidade de envolvê-lo em um cabeçalho na maioria dos casos.

TIP Um cabeçalho não é intercambiável com um título, como nos elementos h1–h6 (consulte “Criando Títulos”). Cada um tem seu próprio propósito semântico.

TIP Você não pode aninhar um elemento de rodapé ou outro cabeçalho dentro de um cabeçalho, nem pode aninhar um cabeçalho dentro de um rodapé ou endereço elemento.

TIP Um cabeçalho nem sempre precisa conter um elemento `nav` como os exemplos (A e C), mas na maioria dos casos, provavelmente acontecerá se o cabeçalho contiver links de navegação. No caso de C, `nav` é apropriado em torno da lista de links de Perguntas Frequentes, já que é um grupo importante de navegação dentro da página, conforme discutido em “Marcando Navegação”.

TIP Consulte “Criando contêineres genéricos” para saber como o cabeçalho substituiu uma das funções do elemento `div` desde seus dias anteriores ao HTML5.

TIP Consulte “Melhorando a acessibilidade com ARIA” para saber como você pode usar `role="banner"` com cabeçalho.

Marcação de navegação

Versões anteriores do HTML não tinham um elemento que representa explicitamente uma seção dos principais links de navegação, mas o HTML5 representa: o elemento **nav**. Os links em uma **navegação** podem apontar para o conteúdo da página A, para outras páginas ou recursos, ou ambos. Seja qual for o caso, use **navegação** apenas para os grupos de links mais importantes do seu documento, não para todos eles.

Se você examinou atentamente o código da seção anterior, deu uma olhada na **navegação** elemento em ação. Eu carreguei esse código amostra para esta página, destacando **nav** A. O elemento **nav** não impõe nenhuma formatação padrão em seu conteúdo B.

R Esses links (os elementos **a**) representam um conjunto importante de navegação, então aninhei-os em um elemento **nav**. Normalmente, você marcará uma lista de links com o elemento **ul** (lista não ordenada), a menos que seus links sejam links de localização atual. Nesse caso, use uma **ol** (lista ordenada). Consulte o Capítulo 15 para obter mais informações sobre listas. O atributo **role** não é obrigatório, mas pode melhorar a acessibilidade. Veja a última dica desta seção para obter informações sobre como aplicar **role="navigation"** à **navegação**.

```
...
<corpo>
<cabecalho>

    <nav role="navegação">
        <ul>
            <li><a href="#gaudi">Arquiteto y de
                Barcelona </a></li>

            <li lang="es"><a href="#sagrada-y
                familia">A Sagrada Família</a> y </li>

            <li><a href="#park-guell">Parque y
                Guell</a></li>
        </ul>
    </nenum>
</header>
</body>
</html>
```



B Nossa navegação parece bastante simples por padrão. Os marcadores não são um produto do elemento `nav`, que não possui nenhum estilo padrão além de começar em sua própria linha. Os marcadores são exibidos porque cada link está em um elemento `li` (um item de lista). Com CSS, você pode desativar os marcadores ou mostrar outros diferentes, bem como dispor os links horizontalmente, alterar sua cor, torná-los parecidos com botões e muito mais. Você começará a aprender sobre CSS no Capítulo 7.

Para designar um grupo de links como navegação importante:

1. Digite `<nav>`.
2. Digite sua lista de links estruturados como `ul` (lista não ordenada), a menos que a ordem dos links seja significativa (como navegação estrutural); nesse caso, você deve estruturá-los como uma `ol` (lista ordenada). (Veja os Capítulos 6 e 15 para aprender sobre links e listas, respectivamente.)
3. Digite `</nav>`.

TIP Se você tem alguma experiência com HTML ou XHTML, você provavelmente está acostumado a estruturar seus links em um elemento `ul` ou `ol`, conforme apropriado. No HTML5, `nav` não substitui essa prática recomendada; continue a usar esses elementos e simplesmente envolva-os com uma navegação A.

TIP Embora os leitores de tela em geral ainda estejam se atualizando com a nova semântica do HTML5, o elemento `nav` pode ajudá-los a identificar a navegação importante da sua página e permitir que os usuários acessem-na por meio do teclado. Isso torna sua página mais acessível, melhorando a experiência dos visitantes.

TIP A especificação HTML5 recomenda não agrupar. Faça ping de links de rodapé de páginas auxiliares, como “Termos de Uso” e “Política de Privacidade” em uma navegação, o que faz sentido. Às vezes, porém, o rodapé da sua página reitera a navegação global de nível superior ou inclui outros links importantes como “Localizador de lojas” e “Carreiras”. Na maioria dos casos, recomendo colocar esses tipos de links de rodapé em um arquivo de navegação.

TIP HTML5 não permite aninhar uma navegação dentro de um elemento de endereço.

TIP Consulte “Melhorando a acessibilidade com ARIA” para aprender como usar `role="navigation"` com navegação A.

Uma análise mais aprofundada da navegação

Como mencionei anteriormente, só porque você tem um grupo de links em sua página não significa que ele deva estar contido em uma **navegação**.

O exemplo de página de notícias a seguir inclui quatro listas de links, dos quais apenas dois são considerados importantes o suficiente para justificar serem agrupados em uma **navegação**. (Como você verá, abreviei partes do código.)

...

<corpo>

<cabeçalho>

<!-- o logotipo do site pode ir aqui --> <!--

navegação global do site -->

<nenhum>

** ... **

</nenhum>

</header>

<div id="principal">

<h1>Artes e artes Entretenimento</h1>

<artigo>

<h1>A abertura da galeria apresenta o inspirado, inspirador</h1> <p>...

[conteúdo da história] ... </p>

<à parte>

<h1>Outras histórias</h1>

<!-- não incluído em navegação -->

** ... [links de histórias] ... **

</aside>

</artigo>

</div>

Uma análise mais aprofundada da navegação (continuação)

```
<aside id="barra lateral">
    <nav><!-- navegação secundária -->
        <ul>
            <li><a href="/arts/movies/">Filmes</a></li>
            <li><a href="/arts/music/">Música</a></li>
            ...
        </ul>
    </nenhum>
</aside>

<rodapé>

    <!-- Links auxiliares não agrupados em navegação. -->
    <ul> ... </ul>

</footer>

</body>
</html>
```

A navegação secundária no **aparte** (veja "Especificando um aparte") permite ao usuário navegar para outras páginas no diretório Arts & Entertainment, constituindo assim uma importante seção de navegação da página. Porém, as Outras Histórias **além** dos links não.

Então, como você decide quando um grupo de links merece **navegação**? Em última análise, é um julgamento baseado na organização do seu conteúdo. No mínimo, marque a navegação global do seu site (ou seja, o que permite aos usuários acessar as seções do site) com **nav**. Frequentemente, mas nem sempre, essa **navegação** específica aparece dentro de um elemento **de cabeçalho** no nível da página (consulte "Criando um cabeçalho").

Criando um Artigo

Outro elemento novo graças ao HTML5 é o **artigo** A.

Você já viu alguns exemplos dele em jogo. Agora vamos aprender mais sobre o que o motiva.

Com base no nome, você adivinharia, com razão, que pode usar **artigo** para conter conteúdo como um artigo de jornal. No entanto, não se limita a isso. No HTML5, “artigo” é mais parecido com “item”.

Veja como o HTML5 o define:

O elemento **article** representa uma composição independente em um documento, página, aplicativo ou site e é, em princípio, distribuível ou reutilizável de forma independente, por exemplo, em distribuição. Pode ser uma postagem em um fórum, um artigo de revista ou jornal, uma entrada de blog, um comentário enviado por um usuário, um widget ou gadget interativo ou qualquer outro item de conteúdo independente.

Outros exemplos **de artigos** podem incluir uma crítica de filme ou música, um estudo de caso, uma descrição de produto e muito mais. Você pode ter ficado surpreso ao saber que também pode ser um widget ou gadget interativo, mas esses também são itens de conteúdo independentes e redistribuíveis.

A Abreviei o conteúdo do artigo e a navegação

código da seção anterior para mantê-lo simples.

Você pode ver a versão completa do código da página no site do livro em www.bruceontheloose.com/htmlcss/examples/. Embora este exemplo inclua apenas parágrafos e imagens, um **artigo** pode conter uma variedade de tipos de conteúdo, como vídeo, figuras, listas e muito mais.

```
...
<corpo>
<cabecalho>

    <nav role="navegação">
        ... [ul com links] ...
    </nenhum>
</header>
<artigo>
    <h1 id="gaudi">Arquiteto de Barcelona</h1>

    <p>Os incríveis edifícios de Antoni Gaudí
    trazem milhões de turistas para
    Barcelona todos os anos.</p>

    <p>O inconformismo de Gaudí, já
    visível na sua adolescência, aliado
    à sua devoção silenciosa mas firme
    à igreja, constituiu uma base única
    para os seus pensamentos e ideias.
    Sua busca pela simplicidade, baseada
    em suas observações cuidadosas da
    natureza, são bastante evidentes
    em seu trabalho, desde o
    <a href="#park-guell">Parque Guell</a>
    e suas incríveis esculturas e
    mosaicos, à Igreja da
    <a href="#sagrada-familia">Sagrada Família</a>
    e suas torres orgânicas e bulbosas.</p>

    <h2 id="sagrada-familia" lang="es">A
    Sagrada Família</h2>

    ... [imagem e parágrafos] ...

    <h2 id="park-guell">Parque Guell</h2>

    ... [imagem e parágrafos] ...
</artigo>

</body>
</html>
```



B Agora a página tem **cabeçalho, navegação e artigo** elementos, bem como seu conteúdo. O **artigo** os títulos podem ter um tamanho diferente por padrão, dependendo do navegador. Você pode padronizar sua aparência nos navegadores com CSS (veja o Capítulo 10).

Para criar um artigo:

1. Digite `<artigo>`.
2. Digite o conteúdo do artigo, que pode incluir qualquer número de elementos, como parágrafos, listas, áudio, vídeo, imagens, figuras e muito mais.
3. Digite `</artigo>`.

TIP Como você aprendeu em “Compreendendo o esboço do documento HTML5”, o artigo é um dos quatro elementos de conteúdo de seção, juntamente com cabeçalho, seção e lado.

TIP Você pode aninhar um artigo dentro de outro, desde que o artigo interno esteja relacionado ao artigo como um todo. Porém, você não pode aninhar um artigo dentro de um elemento de endereço.

TIP Uma página pode conter vários artigos elementos (ou nenhum). Por exemplo, a página inicial de um blog normalmente inclui algumas das postagens mais recentes; cada um poderia ser seu próprio artigo.

TIP Não é obrigatório que um artigo tenha um ou mais elementos de seção. É perfeitamente válido deixar que os elementos h1–h6 dentro de um artigo sejam independentes, embora ao definir seções você esteja tornando a semântica do artigo mais explícita. E cada seção pode ter sua própria hierarquia de níveis de títulos, conforme discutido em “Compreendendo o esboço do documento HTML5”.

TIP Os elementos do artigo e da seção são facilmente (e com razão) confundidos entre si, e é por isso que citei diretamente as definições do HTML5; Eu não queria que você aprendesse sobre eles através de um filtro. Eu discuto a seção e escolher entre os dois em “Definindo uma Seção”.

TIP Consulte “Melhorando a acessibilidade com ARIA” para saber como você pode usar `role="main"` com o artigo sob uma determinada circunstância. Seria apropriado incluí-lo no artigo em `A` porque é o contêiner do conteúdo principal da página, mas o omiti para evitar dar a impressão de que `role="main"`

é adequado para todos os elementos do artigo.

Mais exemplos de artigos

O exemplo anterior **A** é apenas uma maneira de usar **o artigo**. Vamos dar uma olhada em mais algumas possibilidades.

Exemplo 1 (artigo básico):

<artigo>

<h1>A Diversidade de Papua Nova Guiné</h1>

<p>Papua Nova Guiné é o lar de mais de 800 tribos e línguas...</p>

... [resto do conteúdo da história] ...

<footer> <!-- o rodapé do artigo, não o da página -->

<p>Leandra Allen é jornalista freelancer formada em  antropologia pela Universidade de Copenhague.</p>

<endereço>

Você pode contatá-la em  leandra@therunningwriter.com.

</endereço>

</footer>

</artigo>

Observe o uso dos elementos **rodapé** e **endereço** (veja as discussões sobre eles neste capítulo e no Capítulo 4, respectivamente). Aqui, **o endereço** se aplica apenas ao **artigo** pai (aquele mostrado), não à página ou a quaisquer **artigos** aninhados nesse **artigo**, como os comentários do leitor no Exemplo 2.

O Exemplo 2 demonstra elementos **de artigo** aninhados na forma de comentários enviados pelo usuário ao **artigo pai**, exatamente como você vê na seção de comentários de blogs ou sites de notícias. Também mostra um uso para o elemento **de seção** (veja “Definindo uma Seção”) e o elemento **de tempo**, abordado no Capítulo 4.

Mais exemplos de artigos (continuação)

Exemplo 2 (artigos aninhados):

<artigo>

<h1>A Diversidade de Papua Nova Guiné</h1>

... [conteúdo do artigo principal] ...

<rodapé>

... [rodapé do artigo principal] ...

</footer>

<seção>

<h2>Comentários do leitor</h2>

<artigo>

<footer>travelgal escreveu em <time datetime="2011-11-17" &#x27; pubdate>17 de novembro de 2011</time>;</footer>

<p>Ótimo artigo! Sempre tive curiosidade sobre Papua Nova &#x27; Guiné.</p>

</artigo>

<artigo>

... [próximo comentário do leitor] ...

</artigo>

</seção>

</artigo>

Estas são apenas algumas maneiras mais comuns de aproveitar **o artigo** e seus elementos complementares.

Definindo uma seção

O elemento **article** tem um primo menos semanticamente específico, **a seção**, que é outro dos elementos exclusivos do HTML5.

Em parte, o HTML5 define **a seção** da seguinte forma:

O elemento de seção representa uma seção genérica de um documento ou aplicativo. Uma seção, neste contexto, é um agrupamento temático de conteúdo, normalmente com o título A.

Exemplos de seções seriam os capítulos, as diversas páginas com guias em uma caixa de diálogo com guias ou as seções numeradas de uma tese. Um site a página inicial pode ser dividida em seções para introdução, notícias e informações de contato.

Os elementos **do artigo** e **da seção** são bastante semelhantes. Se você não tiver certeza de como diferenciar os dois, consulte a barra lateral “Como você decide entre **o artigo** e **seção**?”

Para definir uma seção:

1. Digite **<seção>**.
2. Digite o conteúdo da seção, que pode incluir qualquer número de elementos, como parágrafos, listas, áudio, vídeo, imagens, figuras e muito mais.
3. Digite **</seção>**.

R O código é o mesmo de antes, exceto que envolvi uma **seção** em torno de cada uma das duas seções do **artigo** que segue a introdução. Simplifiquei o código novamente por questões de brevidade.

```
...
<corpo>
<cabecalho>
    <nav role="navegação">
        ... [ul com links] ...
    </nenhum>
</header>

<artigo>
    <h1 id="gaudi">Arquiteto de Barcelona</h1>

    <p>Os incríveis edifícios de Antoni Gaudí
    trazem milhões de turistas para
    Barcelona todos os anos.</p>

    ... [outro parágrafo introdutório] ...

    <seção>
        <h2 id="sagrada-familia" lang="es">A
        Sagrada Família</h2>

        <p> A
        obra-prima inacabada e de nome
        complicado e curiosamente
        que é o Templo Expiatório da
        Sagrada Família é o edifício
        mais visitado de Barcelona.
        Nele, Gaudí combina a sua
        visão da natureza e da arquitetura
        com a sua devoção à sua fé. A
        Sagrada Família atrai até mesmo
        os não religiosos para suas portas,
        em grande parte devido à sua
        história trágica e ao seu estado
        ainda inacabado, dos quais os
        sempre presentes andaimes e
        guindastes são lembranças
        permanentes.</p>
    </seção>

    <seção>
        <h2 id="park-guell">Parque Guell</h2>

        ... [outra imagem e parágrafos] ...
    </seção>
</artigo>
</body>
</html>
```



B Agora a página possui elementos de **cabeçalho**, **navegação**, **artigo** e **seção**, bem como seus conteúdos. A renderização padrão é a mesma de antes de você adicionar os elementos da **seção**.

TIP Como você aprendeu em “Compreendendo o esboço do documento HTML5”, **seção** é um dos quatro elementos de conteúdo de seção junto com navegação, artigo e lateral.

TIP Por padrão, você pode não ver diferença ao visualizar uma página com seções (ou artigos, nesse caso), mas o importante é que você fortaleça a semântica do seu documento usando-as. **seção de estilo** e elementos de **artigo** como você quiser com CSS.

TIP Tenha em mente que a **seção** não é um contêiner genérico como **div**, porque a **seção** transmite significado e **div** não tem absolutamente nenhum significado semântico (veja “Criando Containers Genéricos”).

TIP Há vários exemplos ao longo deste capítulo para ajudá-lo a ter uma ideia de como usar o **artigo** e a **seção** (de diversas maneiras).

TIP Consulte “Melhorando a acessibilidade com ARIA” para saber como você pode usar `role="main"` com a **seção** sob uma determinada circunstância.

Como você decide entre artigo e seção?

Citei deliberadamente as definições de **seção** e **artigo** do HTML5 (consulte “Criando um artigo”) para ajudá-lo a entender a distinção, porque às vezes ela é sutil. Pense neles desta forma. O seu conteúdo é um conteúdo independente ou um widget que seria apropriado para distribuição? Se sim, use o **artigo**. (Caso contrário, na maioria dos casos, use a **seção**, mas consulte “Criando contêineres genéricos” para saber quando usar **div**.) Isso não significa que você precisa distribuir ou distribuir o conteúdo **do artigo**, apenas que o conteúdo é adequado para isto.

Se você ainda acha que o **artigo** e a **seção** às vezes parecem muito semelhantes, não se preocupe, você não está sozinho. Mesmo desenvolvedores experientes às vezes aplicam esses dois elementos de maneira diferente.

Conforme mencionado no Capítulo 1, nem sempre há uma escolha certa e uma escolha errada quando se trata de marcar seu conteúdo, apenas na maioria das vezes. As outras vezes se resumem a decisões pessoais sobre quais elementos HTML você acha que melhor descrevem seu conteúdo.

Portanto, pense cuidadosamente ao decidir entre **artigo** e **seção**, mas não se preocupe se você acertará sempre. Às vezes é um pouco subjetivo e, de qualquer forma, sua página continuará funcionando. Além disso, ninguém vai bater à sua porta no meio da noite.

Bem, eu poderia, mas isso é só porque está escuro e assustador lá fora.

Um exemplo de seção sem artigo Até agora você viu

exemplos de **seção** aninhada em um **artigo** A. Esse é apenas um uso do elemento.

No seguinte exemplo ligeiramente modificado da especificação HTML5, você vê **a seção** usada sem **artigo**. (Você também terá uma visão antecipada das listas ordenadas em ação. Saiba mais sobre **ol** e outros elementos da lista no Capítulo 15.)

...

<body>

<h1>Programa de Pós-Graduação</h1>

<seção>

<h2>Cerimônia</h2>

Procissão de Abertura

Discurso do orador da turma

Discurso do Presidente da Turma

li> Apresentação dos Diplomas

Discurso de encerramento do Diretor

</seção>

<seção>

<h2>Graduados (em ordem alfabética)</h2>

Molly Carpenter

Anastasia Lúcio

Ebenazar McCoy

Adição de Murphy

Thomas Raith

Susan Rodríguez

</seção>

</body>

</html>

R Este **aparte**, que apresenta informações sobre as maravilhas arquitetônicas de Barcelona, está tangencialmente relacionado ao conteúdo de Antoni Gaudí que é o foco da página, mas também pode ser independente. Eu poderia ter aninhado dentro do **artigo** já que estão relacionados, mas resolvi colocá-lo depois do **artigo** para tratá-lo visualmente posteriormente como uma barra lateral C. O **role="complementar"** ao **lado** é opcional, mas pode melhorar a acessibilidade. Veja a última dica para mais informações.

Especificando um aparte

Às vezes, você tem uma seção de conteúdo que está tangencialmente relacionada ao conteúdo principal da sua página, mas que poderia ser independente A. Como você indicaria isso semanticamente?

```
...
<corpo>

<cabecalho>
  <nav role="navegação">
    ... [ul com links] ...
  </nenhum>
</header>

<artigo>
  <h1 id="gaudi">Arquiteto de Barcelona</h1>
  ... [parágrafos introdutórios] ...

  <seção>
    <h2 id="sagrada-familia" lang="es">A Sagrada Família</h2>
    ... [imagem e parágrafo] ...
  </seção>

  <seção>
    <h2 id="park-guell">Parque Guell</h2>
    ... [outra imagem e parágrafos] ...
  </seção>
</artigo>

<aside role="complementar">
  <h1>Maravilhas Arquitetônicas de Barcelona</h1>

  <p>Barcelona é o lar de muitas maravilhas arquitetônicas além da obra de Gaudí. Alguns deles y incluem:</p>

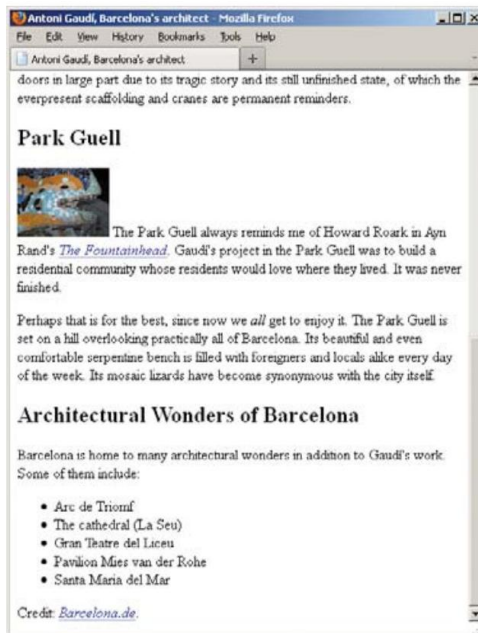
  <ul>
    <li lang="es">Arco do Triunfo</li>
    <li>A catedral <span lang="es">(La Seu)</li>
    <li lang="es">Grande Teatro do Liceu</li>
    <li lang="es">Pavilhão Mies van der Rohe</li>
    <li lang="es">Santa Maria do Mar</li>
  </ul>

  <p>Crédito: <a href="http://www.barcelona.de/en/barcelona-architecture-buildings.html" y rel="external">
    <cite>Barcelona.de</cite></a>.</p>
</aside>

</body>
</html>
```

Até o HTML5, não havia como fazer isso explicitamente. Agora, você tem o elemento **lateral B**.

É comum pensar em um **aparte** como um barra lateral C, mas você pode colocar um **lado** elemento em vários locais da sua página, dependendo do contexto. Pode ser uma caixa (conceitual ou literalmente) dentro do próprio conteúdo principal, na mesma coluna, mas não aninhada no conteúdo principal, ou em (ou como) uma coluna secundária como uma barra lateral. Exemplos de **aparte** incluem uma citação, uma barra lateral, uma caixa de links para artigos relacionados em uma notícia site, publicidade, grupos de elementos **de navegação** (por exemplo, um blog roll), um feed do Twitter e uma lista de produtos relacionados em um site de comércio.



B O **aparte** aparece abaixo do artigo porque o segue no próprio HTML A. Como você pode ver, os navegadores não aplicam nenhuma formatação especial a um **aparte** por padrão (exceto iniciá-los em sua própria linha). No entanto, você tem controle total sobre sua aparência com CSS C.



C Ao aplicar CSS à página finalizada, você pode fazer com que o **lado** (que começa com "Maravilhas Arquitetônicas de Barcelona") apareça ao lado do conteúdo principal, em vez de abaixo dele. Portanto, neste caso, você tratou o **lado** como uma barra lateral. (Você aprenderá como fazer um layout CSS de duas colunas no Capítulo 11.)

Para especificar um aparte:

1. Digite `<à parte>`.
2. Digite o conteúdo **do aparte**, que pode incluir qualquer número de elementos, como parágrafos, listas, áudio, vídeo, imagens, figuras e muito mais.
3. Digite `</aside>`.

TIP Embora um modo de lado seja usado para marcar o conteúdo em uma barra lateral C, o lado o elemento em si não afeta o layout da página B.

TIP Se você usar um ou mais apartes na barra lateral ou como barra lateral, coloque o conteúdo da barra lateral após o conteúdo principal da sua página no HTML A. É melhor para fins de SEO e acessibilidade colocar o conteúdo mais importante primeiro. Você pode alterar a ordem em que eles são exibidos no navegador com CSS.

TIP Use o elemento **figura** (consulte o Capítulo 4), sem deixar de lado, para marcar figuras relacionadas ao seu conteúdo, como um quadro, um gráfico ou uma foto inserida com uma legenda.

TIP HTML5 não permite aninhar um aparte dentro de um elemento de endereço.

TIP Consulte "Melhorando a acessibilidade com ARIA" para saber como você pode usar `role="complementary"` com aparte.

Outros exemplos à parte

Conforme mencionado, o **lado** pode aparecer na mesma coluna do conteúdo principal, aninhado no conteúdo principal ou em uma barra lateral.

O exemplo 1 mostra um **aparte** aninhado em seu conteúdo relacionado.

Exemplo 1 (aninhado no conteúdo principal):

```
...
<corpo>
<artigo>
  <h1>A Diversidade de Papua Nova Guiné</h1>
  ... [conteúdo do artigo] ...
  <à parte>
    <h1>Fatos rápidos sobre Papua Nova Guiné</h1>
    <ul>
      <li>O país tem 38 das 43 aves do paraíso conhecidas</li>
      <li>Embora bastante tropical em algumas regiões, outras ocasionalmente
        ỳ experimente nevascas.</li>
      ...
    </ul>
  </aside>
  ... [mais conteúdo do artigo] ...
</artigo>
</body>
</html>
```

Esse mesmo **artigo** pode incluir uma citação extraída do texto do artigo. Isso também seria um **aparte**. Ou poderia ter **à parte** "Histórias Relacionadas" contendo uma lista de links para outros ensaios sobre o país ou região circundante (Indonésia, Austrália e assim por diante). Alternativamente, esse **lado** poderia estar em uma coluna de página diferente, em vez de aninhado no **artigo**.

Você já viu um exemplo de **aparte** em uma barra lateral (A e C). Agora, vamos considerar um exemplo de portfólio de design ou conjunto de estudos de caso, em que cada página HTML se concentra em um único projeto e você fornece links (aninhados em uma **navegação**) para as outras páginas do projeto em uma coluna adjacente (controlada por CSS). , não simplesmente em virtude de organizar o código conforme mostrado no Exemplo 2).

Outros exemplos à parte (continuação)

Exemplo 2 (além de não aninhado no conteúdo principal e contendo uma navegação):

```
...

<corpo>
<!-- conteúdo principal da página -->
<artigo>

    <h1>... [nome do projeto] ...</h1>
    <figure>... [foto do projeto] ...</figure> <p>...
    [redação do projeto] ... </p>
</artigo>

<!-- este aparte não está aninhado no artigo -->
<à parte>
    <h1>Outros Projetos</h1>
    <nenhum>
    <ul>
        <li><a href="habitat-for-humanity.html">Habitat para a Humanidade &
folheto</a></li>
        <li><a href="royal-philharmonic.html">Royal Philharmonic Orchestra &
site</a></li>
        ...
    </ul>
    </nenhum>
</aside>
</body>
</html>
```

Seria perfeitamente adequado aninhar isso **também** no **artigo** do projeto , uma vez que eles estão relacionados.

Criando um rodapé

Quando você pensa em rodapé, provavelmente pensa em rodapé de página. **Rodapé** do HTML5

O elemento é apropriado para isso, mas como o **cabeçalho**, você também pode usá-lo em outro lugar.

O elemento **footer** representa um rodapé para o **artigo mais próximo, aparte, blockquote, corpo, detalhes, fieldset, figura, nav, seção** ou elemento **td** no qual está aninhado. É o rodapé do

página inteira somente quando seu ancestral mais próximo for o **corpo** (A e B). E se um **rodapé** envolve todo o conteúdo de sua seção (um **artigo**, por exemplo), representa algo como um apêndice, índice, colofão longo ou contrato de licença longo, dependendo de seu conteúdo.

A Este **rodapé** representa o rodapé de toda a página, já que seu ancestral mais próximo é o **corpo** elemento. Nossa página agora possui elementos **de cabeçalho, navegação, artigo, seção, lado e rodapé**. Nem todas as páginas exigem todos eles, mas eles representam as construções de páginas primárias disponíveis em HTML.

```
...
<corpo>
<cabeçalho>
    <nav role="navegação">
        ... [ul com links] ...
    </nenhum>
</header>

<artigo>
    <h1 id="gaudi">Arquiteto de Barcelona</h1>
    ... [parágrafos introdutórios] ...

    <seção>
        <h2 id="sagrada-familia" lang="es">A y Sagrada
        Família</h2>
        ... [imagem e parágrafo] ...
    </seção>

    <seção>
        <h2 id="park-guell">Parque Guell</h2>
        ... [outra imagem e parágrafos] ...
    </seção>
</artigo>

<aside role="complementar">
    <h1>Maravilhas arquitetônicas de Barcelona y </
    h1>
    ... [resto à parte] ...
</aside>

<rodapé>
    <p><small>&copy; Direitos autorais 2011</small>
    y </p>
</footer>

</body>
</html>
```



B O próprio elemento **de rodapé** não impõe nenhuma formatação ao texto por padrão. Aqui, o aviso de copyright é menor que o texto normal porque está aninhado em um **pequeno** elemento para representar semanticamente a impressão legal (consulte o Capítulo 4). Como tudo mais, você pode alterar o tamanho da fonte com CSS.

Para criar um rodapé:

1. Coloque o cursor dentro do elemento para qual você deseja criar um rodapé.
2. Digite `<rodapé>`.
3. Digite o conteúdo do rodapé.
4. Digite `</footer>`.

TIP Um rodapé normalmente inclui informações sobre sua seção, como links para documentos relacionados, informações sobre direitos autorais, seu autor e itens semelhantes. Consulte os Exemplos 1 e 2 na barra lateral “Outros exemplos de rodapé”.

TIP Um rodapé não precisa estar no final do elemento que o contém, embora geralmente esteja.

TIP É inválido aninhar um cabeçalho ou outro rodapé dentro de um rodapé. Além disso, você não pode aninhar um rodapé em um cabeçalho ou endereço elemento.

TIP Consulte “Criando contêineres genéricos” para saber como o rodapé substituiu um dos `div` funções do elemento desde seus dias pré-HTML5.

TIP Consulte “Melhorando a acessibilidade com ARIA” para saber como você pode usar `role="contentinfo"` com rodapé em uma determinada circunstância. Seria apropriado incluí-lo no rodapé em `A` porque representa o rodapé de toda a página, mas omiti para evitar dar a impressão de que `role="contentinfo"` é adequado para todos os rodapés elementos. Consulte “Outros exemplos de rodapé” para obter um exemplo que mostra a distinção e usa a função corretamente.

Outros exemplos de rodapé Você viu

um pequeno exemplo de rodapé para a página inteira (**A** e **B**). Aqui está outro rodapé de página, mas com mais conteúdo.

Exemplo 1 (como rodapé da página):

...

<corpo>

... **[cabeçalho e conteúdo da página]** ...

<!-- este é um rodapé de página porque body é seu ancestral mais próximo -->

<footer role="contentinfo">

<p><small>© Copyright 2011 The Corporation, Inc.</small></p>

Termos de Uso

Política de Privacidade

</footer>

</body>

</html>

O próximo exemplo demonstra um **rodapé** no contexto de uma seção de página (neste caso, um **artigo**) e um segundo **rodapé** para toda a página. (Veja "Mais exemplos **de artigos** " para uma explicação do escopo do elemento de **endereço** aqui.)

Exemplo 2 (como rodapé de uma seção da página e de toda a página):

...

<corpo>

...

<artigo>

<h1>... [cabeçalho do artigo] ...</

h1> <p>... [conteúdo do artigo] ...</p>

Outros exemplos de rodapé (continuação)

```
<!-- o rodapé do artigo -->

<rodapé>

  <p>Leandra Allen é jornalista freelance e se formou ÿ em antropologia pela Universidade de
  Copenhagen.</p>

  <endereço>

  Você pode contatá-la em <a href="mailto: leandra@therunningwriter. ÿ com">
  leandra@therunningwriter.com</a>.

  </endereço>

</footer>

</artigo>

<!-- o rodapé da página -->

<footer id="footer-page" role="contentinfo">

  ... [direitos autorais, termos de uso, política de privacidade] ...

</footer>

</body>

</html>
```

O **id="footer-page"** (você pode especificar qualquer **id válido**) no rodapé da página é opcional e serve apenas para diferenciá-lo do outro **rodapé** para controle de estilo. Observe que apenas o **rodapé** da página recebe o opcional **role="contentinfo"**. Consulte "Melhorando a acessibilidade com ARIA" para saber mais sobre esta função.

Criando Genérico Containers

Às vezes você precisa envolver um contêiner em torno de um segmento de conteúdo porque deseja aplicar algum estilo com CSS ou talvez um efeito com JavaScript. Sua página simplesmente não seria a mesma sem ele A. Mas, ao avaliar o conteúdo, você determina que usar artigos como **artigo**, **seção**, **aparte**, **navegação** ou outros elementos não seria semanticamente apropriado. O que você realmente precisa é de um contêiner genérico, sem nenhum significado semântico. Esse contêiner é o elemento **div** (pense em uma “divisão”) B. Com uma **div** instalada, você pode aplicar o estilo desejado C ou efeito JavaScript para ele. Não deixe de ler a barra lateral para saber mais sobre quando use **div** em suas páginas.



A Consegui esse design sem nenhum **div** elementos da página. Mas ao adicionar uma **div** ao redor de todo o conteúdoB da página, agora tenho um contêiner genérico ao qual posso aplicar mais alguns estilos (veja os resultados emC).

B Agora um **div** envolve todo o conteúdo. A semântica da página permanece inalterada, mas agora tenho um contêiner genérico ao qual posso conectar alguns estilos com CSS C.

```
...
<corpo>

<div>

  <cabecalho>

    <nav role="navegação">
      ... [ul com links] ...
    </nenhum>
  </header>

  <artigo>

    <h1 id="gaudi">Arquiteto de Barcelona & </h1>

    ... [parágrafos introdutórios] ...

    <seção>
      ... [título, imagem e &
        parágrafo] ...
    </seção>

    <seção>
      ... [título, outra imagem e & parágrafos] ...
    </seção>
  </artigo>

  <aside role="complementar">
    <h1>Maravilhas arquitetônicas de &
      Barcelona</h1>
    ... [resto à parte] ...
  </aside>

  <rodapé>
    ... [direito autor] ...
  </footer>
</div>

</body>
</html>
```



C Um elemento **div** não tem nenhum estilo próprio por padrão, exceto que ele começa em uma nova linha D. No entanto, você pode aplicar estilos a **div** para implementar seus designs. Aqui, adicionei o fundo azul claro e uma sombra de caixa ao **div**. Isso me permitiu alterar o plano de fundo do elemento **body** para roxo para que o conteúdo aparecesse. Também adicionei uma borda fina ao **lado**.
Você pode ver como consegui isso no HTML e CSS da página (www.bruceconterloose.com/htmlcss/examples/).



D A mesma página sem CSS aplicado ao **div**, aos títulos, aos parágrafos ou a qualquer outro elemento. Como você pode ver, o **div** não faz nada parecer sofisticado por si só.

Para criar um contêiner genérico:

1. Digite `<div>`.
2. Crie o conteúdo do contêiner, que pode incluir qualquer número de elementos.
3. No final do contêiner, digite `</div>`.

TIP Assim como cabeçalho, rodapé, artigo, seção, aparte, nav, h1–h6, p e muitos outros, **div** é exibido automaticamente em uma nova linha por padrão.

TIP **div** também é útil ao implementar certas interações ou efeitos com JavaScript. Por exemplo, exibir uma foto ou caixa de diálogo em uma sobreposição semitransparente que cubra a página (a sobreposição normalmente é um **div**).

TIP Apesar de enfatizar que o HTML descreve o significado do seu conteúdo, **div** não é o único elemento que não tem valor semântico. O elemento **span** é a contraparte do **div**. Enquanto **div** é um contêiner sem semântica para blocos de conteúdo, **span** (escrito como `<span>`o conteúdo vai aqui) é um contêiner para frases, como dentro de um elemento p para parágrafos. Veja mais sobre **span** no Capítulo 4.

TIP Consulte “Melhorando a acessibilidade com ARIA” para saber como você pode usar funções de referência com **div**.

Um pouco da história sobre div e quando usá-lo em HTML5

Dos elementos estruturais apresentados neste capítulo, **div** é o único além de h1–h6 que é anterior ao HTML5. Até o HTML5, **div** era a escolha de fato para agrupar partes de conteúdo, como cabeçalho, rodapé, conteúdo principal, inserções e barras laterais de uma página, para que você pudesse estilizá-los com CSS. Mas **div** não tinha significado semântico naquela época, e ainda não tem hoje.

É por isso que o HTML5 introduziu **cabeçalho**, **rodapé**, **artigo**, **seção**, **lado** e **navegação**. Esses tipos de blocos de construção eram tão predominantes nas páginas da Web que mereciam seus próprios elementos com significado. **div** não desaparece no HTML5, você terá menos ocasiões para usá-lo do que no passado.

Vejamos alguns casos comuns em que **div** é a escolha certa.

Você já viu um: envolver uma página inteira com um contêiner para fins de estilo (**B** e **C**).

Como consegui o layout de duas colunas com **div**? Apliquei um pouco de CSS ao elemento **article** para exibi-lo como coluna um e ao elemento **side** para exibi-lo como coluna dois. (Veja o Capítulo 7 para começar a aprender CSS, e veja o Capítulo 11 para layouts com CSS.)

Entretanto, na maior parte (se não na maior parte) do tempo, cada uma de suas colunas tem mais de uma seção de conteúdo. Por exemplo, talvez você queira outro **artigo** (ou **seção**, ou **à parte**, e assim por diante) na área de conteúdo principal abaixo do primeiro **artigo**. E talvez você queira um **aparte** adicional na segunda coluna, digamos, com uma lista de links para outros sites sobre Gaudí. Ou talvez você queira outro tipo de elemento nessa coluna.

Você precisaria agrupar o conteúdo que deseja representar cada coluna em um **div** **E** e depois estilizar esse **div** de acordo. (Se você estava pensando que **a seção** seria uma opção, ela não se destina a ser um contêiner genérico para estilo.) Fornecei um diagrama **F** para ajudá-lo a visualizar a relação entre o código **E** e um possível layout CSS. Tenha em mente que é apenas uma possibilidade de layout para este HTML; CSS é bastante poderoso.

Portanto, é muito comum ter uma **div** em torno de cada grupo de conteúdo que você deseja estilizar como uma coluna (claro, você pode fazer mais de duas). Em termos do que há neles, isso pode variar muito, com base no conteúdo que você deseja em suas páginas. Não se esqueça disso, pois seus contêineres semânticos primários para seções de conteúdo, **artigo**, **seção**, **aparte** e **navegação** podem ir a praticamente qualquer lugar. Assim como **o cabeçalho** e **o rodapé**, como você aprendeu neste capítulo. Não dê muita importância ao fato de que o exemplo (**E** e **F**) mostra apenas **artigos** na área de conteúdo principal e **apartes** na barra lateral.

Porém, para ter certeza, **div** deve ser seu último recurso como contêiner porque não tem valor semântico.

Na maioria das vezes, será correto usar **cabeçalho**, **rodapé**, **artigo**, **seção**, **lado** e, possivelmente, **navegação**. No entanto, não use um desses apenas para evitar **div** se não for semanticamente apropriado fazê-lo. **div** tem seu lugar, você só quer limitar seu uso.

Dito isto, há uma situação válida em que não há problema em usar **div** para todos (ou a maioria, depende de você) contêineres em uma página, em vez dos novos elementos HTML 5. Consulte “Estilizando elementos HTML5 em navegadores mais antigos” no Capítulo 11 para obter mais informações.

E Esta página possui a **div** que contém a página inteira, mais duas novas. Uma **div** com **id="content"** agrupa o conteúdo principal para que possa ser estilizado como a coluna um. Outra **div** com **id="sidebar"** envolve o conteúdo que você deseja exibir na coluna dois. Então você pode usar o **id** em seu CSS para direcionar cada **div** específico para estilo.

```
...
<corpo>

<!-- Contêiner da página inicial -->
<div id="contêiner">

  <cabecalho>
    ...
  </header>

  <!-- Coluna Um quando CSS aplicado -->
  <div id="conteúdo">
    <artigo>
      ...
    </artigo>

    <artigo>
      ...
    </artigo>

    ... [mais seções conforme desejado] ...
  </div>
  <!-- fim da coluna um -->

  <!-- Coluna dois quando CSS aplicado -->
  <div id="barra lateral">
    <à parte>
      ...
    </aside>

    <à parte>
      ...
    </aside>

    ... [mais seções conforme desejado] ...
  </div>
  <!-- fim da coluna dois -->

  <rodapé>
    ...
  </footer>
</div>
<!-- contêiner da página final -->

</body>
</html>
```



F Este diagrama ilustra como o código em **E** poderia mapear conceitualmente para um layout CSS. É um arranjo muito comum, mas apenas uma das muitas possibilidades que o CSS oferece com o mesmo HTML. Não deixe de ler a próxima seção, "Melhorando a acessibilidade com ARIA", para aprender como aprimorar a semântica e a acessibilidade de suas páginas ainda mais.

Melhorando Acessibilidade com ARIA

Como você aprendeu na seção “Por que Semantics Matter” no Capítulo 1, a acessibilidade melhora simplesmente marcando seu conteúdo com o HTML que melhor descreve isto. Então, se você já está fazendo isso, está indo muito bem. Nesta seção, direi como adicionar alguns atributos simples ao seu HTML pode ajudar ainda mais seus visitantes.

WAI-ARIA (Web Accessibility Initiative's Accessible Rich Internet Applications), ou ARIA, é uma especificação que se declara “uma tecnologia de ponte”. Ou seja, preenche lacunas semânticas com atributos até que linguagens como HTML forneçam sua própria semântica equivalente.

Por exemplo, qual marcação HTML você usaria para permitir que um leitor de tela soubesse como ir (ou passar) do conteúdo principal da sua página? Ou para uma caixa de pesquisa? Como você aprenderá, há alguma sobreposição entre ARIA e HTML5 (que também tentou preencher algumas lacunas), mas nem mesmo o HTML5 tem soluções para esses dois. Os papéis marcantes da ARIA fazer; eles identificam um conjunto de regiões de página parcialmente para esse propósito: **aplicativo, banner, complementar, contentinfo, formulário, principal, navegação e pesquisa.**

Onde há sobreposição entre funções de referência e elementos HTML5, o suporte ao leitor de tela atualmente está mais avançado para ARIA. Assim, você pode continuar a criar HTML como sempre fez e adicionar funções ARIA para melhorar a acessibilidade de suas páginas.

Em A, adicionei funções de referência ARIA e um elemento **de navegação** ao exemplo de “Criando contêineres genéricos”. Embora eu tenha colocado um papel **complementar** em cada elemento **lateral**, seria apenas

A O exemplo de “Criando contêineres genéricos” com a adição de um elemento **de navegação** e cinco funções de referência diferentes.

```
...
<corpo>

<!-- Contêiner da página inicial -->
<div id="contêiner">
  <header role="banner">
    ...
    <nav role="navegação">
      ... [ul com links] ...
    </nav>
  </header>

  <!-- Coluna Um quando CSS aplicado -->
  <div id="content" role="main">
    <artigo>
      ...
    </artigo>

    <artigo>
      ...
    </artigo>

    ... [mais seções conforme desejado] ...
  </div>
  <!-- fim da coluna um -->

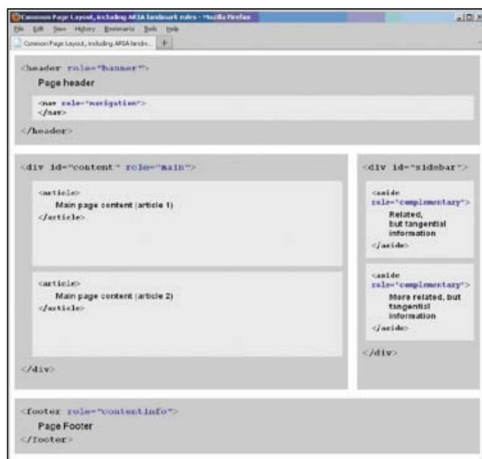
  <!-- Coluna dois quando CSS aplicado -->
  <div id="barra lateral">
    <aside role="complementar">
      ...
    </aside>

    <aside role="complementar">
      ...
    </aside>

    ... [mais seções conforme desejado] ...
  </div>
  <!-- fim da coluna dois -->

  <footer role="contentinfo">
    ...
  </footer>
</div>
<!-- contêiner da página final -->

</body>
</html>
```



B Este é o diagrama de layout de “Criando Generic Containers”, mas agora inclui as funções ARIA. Conforme observado, a **div** da barra lateral poderia ter **role="complementary"** em vez de **aparte** elementos.

como válido para o código **<div id="sidebar"**

role="complementary">, marcando toda a barra lateral. Antes de fazer isso em suas páginas, certifique-se de que todo o seu conteúdo **div** seja qualificado como conteúdo **complementar**.

Abaixo estão alguns dos papéis de referência definidos nições encontradas na especificação ARIA, seguidas pelo meu uso recomendado. Eles são demonstrados em A e em um diagrama **B** semelhante ao de “Criando contêineres genéricos”.

n papel = "banner"

Uma região que contém principalmente conteúdo orientado ao site, em vez de conteúdo específico da página.

O conteúdo orientado ao site normalmente inclui itens como o logotipo ou a identidade do patrocinador do site e uma ferramenta de pesquisa específica do site. Um banner geralmente aparece na parte superior da página e normalmente ocupa toda a largura.

Uso: adicione-o ao cabeçalho do nível da página (normalmente um elemento **de cabeçalho**) e use-o apenas uma vez em cada página.

n role="navegação"

Uma coleção de elementos de navegação (geralmente links) para navegar no documento ou documentos relacionados.

Uso: Isso espelha o **elemento nav** do HTML5, então adicione-o a cada elemento **nav** ou, se nenhum estiver presente, adicione-o ao contêiner ao redor dos seus links. Você pode usar essa **função** mais de uma vez em cada página.

n papel = "principal"

O conteúdo principal de um documento.

Uso: Adicione-o ao contêiner da sua seção principal de conteúdo. Muitas vezes isso irá ser um elemento **div**, mas também pode ser um **artigo** ou **seção**. Exceto em raras circunstâncias, sua página deve ter apenas uma área marcada com **main**.

n role="complementar"

Uma seção de apoio do documento, concebida para complementar o conteúdo principal. . . mas isso continua significando útil quando separado do principal conteúdo.

O papel complementar indica que o conteúdo contido é relevante para o conteúdo principal.

Uso: Isso reflete o **lado** do HTML5 elemento, então adicione-o a um **aparte** ou **div** que contém todo o conteúdo complementar. Você pode incluir mais de um função **complementar** em cada página.

n role="contentinfo"

Uma grande região perceptível que contém informações sobre o documento pai.

Exemplos de informações incluídas nesta região da página são direitos autorais e links para declarações de privacidade.

Uso: adicione-o uma vez em uma página ao rodapé no nível da página (normalmente um **rodapé** elemento).

Em resumo, geralmente é uma boa ideia adicionar funções de referência ARIA ao seu HTML. Eu os incluí em alguns outros exemplos ao longo do livro, bem como no site do livro. Para ser claro, suas páginas funcionarão sem eles, mas incluí-los pode melhorar a experiência de alguns usuários. Você pode achar os resultados do teste do leitor de tela listados nas dicas úteis para decidir se deve usá-los você mesmo (o suporte é sólido fora do leitor de tela Window-Eyes 7.5).

TIP A função de formulário é semanticamente redundante com o elemento de formulário, a pesquisa marca um formulário de pesquisa (BBC, Yahoo! e Google usam isso, bem como algumas outras funções de referência em alguns casos), e o aplicativo é para uso avançado.

TIP As funções de referência são apenas um dos muitos recursos da especificação ARIA (www.w3.org/TR/wai-aria/). Você também pode estar interessado neste guia de implementação: [www.w3.org/WAI/PF/práticas de ária/](http://www.w3.org/WAI/PF/práticas_de_ária/).

TIP Defensores da acessibilidade Steve Faulkner e Jason Kiss publicaram testes separados de suporte à função de referência do leitor de tela em www.html5accessibility.com/tests/landmarks.html e www.accessibleculture.org/research/html5-aria-2011/, respectivamente. Veja as discussões relacionadas de Faulkner em www.paciellogroup.com/blog/2011/11/latest-aria-landmark-support-data/ e em www.paciellogroup.com/blog/2011/07/html5-accessibility-chops-aria-markter-support/.

TIP NVDA (Windows, download gratuito em www.nvda-project.org/), VoiceOver (gratuito como parte do Mac OSX e iOS 4+) e JAWS (Windows, teste gratuito disponível em www.freedomscientific.com/) estão entre os leitores de tela mais avançados disponíveis. Não posso recomendar fortemente que você experimente pelo menos um deles para avaliar melhor como suas escolhas semânticas de HTML influenciam a experiência do usuário do leitor de tela. Melhor ainda, teste suas páginas em um leitor de tela como parte do seu desenvolvimento normal processo.

TIP Você pode usar atributos de função ARIA em seus seletores CSS. Na verdade, usando os papéis de referência adequados, você poderia omitir os atributos `id="content"` e `id="sidebar"` do exemplo de código A. Consulte o Capítulo 11 para obter detalhes.

Nomeando Elementos com uma Classe ou ID

Embora não seja obrigatório, você pode dar aos seus elementos HTML um identificador exclusivo, atribuir-lhes uma classe (ou classes) específica, ou ambos. Depois de fazer isso, você pode aplicar estilos a todos os elementos com um determinado **ID** ou nome **de classe**. Esse é certamente o uso mais popular, mas não o único (veja as dicas nesta seção).

Para nomear um

elemento com um ID exclusivo:

Na tag inicial do elemento, digite `id="nome"`, onde **nome** identifica exclusivamente o elemento A. **nome** pode ser quase qualquer coisa, desde que não comece com um número ou contenha espaços.

Para atribuir uma classe a um elemento:

Dentro da tag inicial do elemento, digite `class="nome"`, onde **nome** é o nome identificador da classe A. Caso queira atribuir mais de uma classe, separe cada uma delas com um espaço, como em `class="nome outronome"`. (Você pode atribuir mais de dois nomes de classe.)

TIP Cada id em um documento HTML deve ser exclusivo. Em outras palavras, dois elementos na mesma página não podem ser nomeados com o mesmo id e cada elemento pode ter apenas um id. O mesmo id pode aparecer em múltiplas páginas e não precisa ser atribuído ao mesmo elemento todas as vezes, embora seja habitual fazê-lo.

TIP Por outro lado, um nome de classe específico pode ser atribuído a qualquer número de elementos em uma página e um elemento pode ter mais de uma classe.

O atributo da classe e Microformatos

Há um equívoco comum de que o atributo **class** foi criado exclusivamente para aplicar CSS a grupos de elementos.

Esse não é o caso. Também foi projetado para enriquecer a semântica do HTML sem adicionando mais elementos à linguagem de marcação.

Os microformatos fazem exatamente isso. Eles usam nomes **de classe** acordados para identificar uma parte do HTML como, digamos, um evento ou entrada de calendário (o microformato **hCalendar**); identificar pessoas, organizações e empresas (**hCard**); ou para descrever a relação entre as pessoas (XFN). Esses são apenas alguns dos muitos microformatos definidos hoje, e mais estão sempre em desenvolvimento.

Aplicativos, bots de pesquisa e outros softwares podem ler e fazer uso do microformatos em seu HTML. Por exemplo, Operator, um complemento do Firefox, expõe os microformatos em qualquer página.

Você pode aprender mais sobre a implementação de microformatos em <http://microformats.org>.

continua na página 94

A Adicione um atributo **id** exclusivo a um elemento para identificá-lo para formatação, links ou comportamento JavaScript posterior. Adicione um atributo **de classe** a um ou mais elementos para poder formatá-los todos de uma só vez. Por exemplo, as classes **arquiteto** e **projeto** poderiam ser aplicadas ao conteúdo sobre outros arquitetos para uma formatação consistente. Os links na **navegação** apontam para os **ids** em **h1** e **h2s** (veja o Capítulo 6 para obter mais informações sobre links). Os outros **ids** são para formatação. Consulte "Criando Containers Genéricos" para obter mais informações sobre ids, bem como outro exemplo que os utiliza. Os atributos **id** e **class** não afetam a aparência de um elemento, a menos que o CSS faça referência a eles.

```
...
<corpo>

<div id="container">
  <header>
    <nav role="navigation">
      <ul id="toc">
        <li><a href="#gaudi">Arquiteto de Barcelona</a></li>
        <li><a href="#sagrada-familia " lang="es">A Sagrada Família</a></li>
        <li><a href="#park-guell">Parque Guell</a></li>
      </ul>
    </
  nav> </header>

  <artigo class="arquiteto" role="principal">
    <h1 id="gaudi">Arquiteto de Barcelona</h1>

    <p>Os incríveis edifícios de Antoni Gaudí trazem milhões de turistas a Barcelona todos os anos.</p>
    ...

    <section class="project">
      <h2 id="sagrada-familia" lang="es">A Sagrada Família</h2>
      ...
    </seção>

    <section class="project">
      <h2 id="park-guell">Parque Guell</h2>
      ...
    </seção> </
  artigo>
  ...
</div>
</body>
</html>
```

TIP Os atributos `class` e `id` podem ser adicionados a qualquer elemento HTML. Um elemento pode ter um `id` e qualquer número de classes.

TIP Para obter informações sobre como aplicar estilos a um elemento com um `ID` ou classe específico, consulte “Selecionando Elementos por Classe ou `ID`” no Capítulo 9.

TIP Escolha nomes significativos (isto é, semânticos) para seus `ids` e classes, independentemente de como você pretende usá-los. Por exemplo, se você usar uma classe para estilizar, evite nomes que descrevam a apresentação, como `class="red"` - isso é um pecado capital (entendeu, vermelho, capital?). Falando sério, `class="red"` é uma escolha ruim porque você pode decidir na próxima semana mudar o esquema de cores do seu site para azul. Alterar a cor atribuída a uma classe em CSS é incrivelmente simples, mas seu HTML teria uma classe chamada `red` que realmente seria renderizada em uma cor diferente. Alterar todos os nomes de classes em seu HTML geralmente não é trivial.

TIP Ao escolher entre aplicar uma classe ou um `id` a um elemento para fins de estilo, geralmente é preferível usar uma classe porque você pode reutilizar seus estilos associados em outros elementos da mesma classe. No entanto, certamente haverá momentos em que você desejará direcionar seus estilos para um elemento (e possivelmente para qualquer um de seus descendentes) através de seu `id`.

TIP O atributo `id` transforma automaticamente o elemento em uma âncora nomeada, para a qual você pode direcionar um link. Para obter mais detalhes, consulte “Criando Âncoras” no Capítulo 6.

TIP Você pode usar o atributo `class` para implementar microformatos (veja a barra lateral para mais detalhes).

TIP Você pode usar JavaScript para acessar os atributos `id` e `class` para aplicar comportamento a elementos específicos.

R Você pode adicionar títulos a qualquer elemento que desejar, embora seja mais comum usá-los em links.

```
...
<corpo>
  <header role="banner">
    <nav role="navegação">
      <ul id="toc" title="Tabela de
        Conteúdo">
        <li><a href="#gaudi" title=
          "Saiba mais sobre Antoni
          Gaudí"> Gaudí </a></li>
        <li> <a href="#sagrada-familia" >
          A Sagrada Família
          </a></li>
        <li><a href="#park-guell">Parque
          Guell</a></li>
      </ul>
    </header>
  </nenhum>
</body>
</html>
...
```



B Quando seus visitantes apontarem para o elemento rotulado, o título aparecerá. Se você estivesse apontando para o link do Arquiteto de Barcelona, veria "Saiba mais sobre Antoni Gaudí", já que tem **título** próprio atributo.

Adicionando o título Atributo aos Elementos

Você pode usar o atributo **title** - para não ser confundido com o elemento **title** - para adicionar um rótulo de dica de ferramenta para praticamente qualquer parte do seu site (**A** e **B**). No entanto, eles não servem apenas para dicas de ferramentas. Os leitores de tela podem ler o texto do título para os usuários, melhorando a acessibilidade.

Para adicionar um título aos elementos de uma página da web:

No elemento HTML do item que você deseja rotular com título, adicione **title="label"**, onde **rótulo** é o breve texto descritivo que deve aparecer na dica de ferramenta quando um visitante apontar para o elemento ou que será lido em voz alta por um leitor de tela.

TIP Versões antigas do Internet Explorer (IE7 e anteriores) também criam dicas de ferramentas a partir do atributo **alt** usado em elementos **img** (veja o Capítulo 5). No entanto, se os atributos **title** e **alt** estiverem presentes em um elemento **img**, a dica de ferramenta será definida para o conteúdo do atributo **title**, não para o atributo **alt**.

Adicionando comentários

Você pode adicionar comentários aos seus documentos HTML para observar onde as seções começam ou terminam, para comentar para você mesmo (ou para futuros editores) a finalidade de um determinado trecho de código, para impedir a exibição do conteúdo no navegador, caso contrário B.

A Este exemplo inclui quatro comentários. Dois se combinam para marcar o início e o fim do artigo. Outro “comenta” o primeiro parágrafo para que não seja exibido na página (se você deseja que o parágrafo seja removido a longo prazo, seria melhor excluí-lo do HTML). O último comentário é um lembrete para adicionar mais conteúdo antes de colocar a página no site ativo. Apenas certifique-se de remover quaisquer comentários temporários como “tarefas” antes de colocar sua página no ar, caso os visitantes vejam seu código. “Criando Containers Genéricos” tem mais exemplos de comentários.

```
...
<corpo>
  ...

  <!-- ===== INICIAR ARTIGO ===== -->
  <artigo class="arquiteto">
    <h1 id="gaudi">Arquiteto de Barcelona</h1>

    <!-- Este parágrafo não é exibido porque está comentado.
    <p>Os incríveis edifícios de Antoni Gaudí trazem milhões de turistas a Barcelona todos os anos
    &y .</p>
    -->

    <p>O inconformismo de Gaudí, já visível na sua adolescência, aliado à sua tranquila &y mas firme
    devoção à Igreja, constituiu uma base única para os seus pensamentos e ideias. Sua &y busca pela
    simplicidade...</p>

    <seção class="projeto">
      <h2 id="sagrada-familia" lang="es">A Sagrada Família</h2>
      ...
    </seção>

    <seção class="projeto">
      <h2 id="park-guell">Parque Guell</h2>
      ...
    </seção>
  </artigo>
  <!-- fim do artigo -->

  <!--
  O QUE FAZER: Adicione outro artigo aqui sobre outros edifícios famosos antes de colocar a página no ar.
  -->

  ...
</body>
</html>
```



B Os comentários são invisíveis (embora apareçam prontamente quando o código-fonte é exibido).

Da mesma forma, se você colocar um comentário em torno de parte do seu conteúdo, ele não exibirá A. Aqui, o primeiro parágrafo do código não é exibido.

Para adicionar um comentário à sua página HTML:

1. No seu documento HTML, onde deseja inserir um comentário, digite `<!--`.
2. Digite os comentários.
3. Digite `-->` para completar o comentário texto.

TIP Um bom uso para comentários é lembrar a si mesmo (ou aos futuros editores) de incluir, remover ou atualizar certas seções A.

TIP Outro uso para comentários é anotar um número de revisão.

TIP É comum comentar o início e o fim das principais seções do código para facilitar a modificação posterior por você ou outros programadores (as páginas podem ficar longas). Gosto de usar um formato diferente e mais proeminente para um comentário inicial do que para um que significa o final de um bloco, para que meus olhos possam distinguir facilmente entre os dois pontos enquanto leio o código A.

TIP Você deve ver sua página comentada com um navegador antes de publicar. Isso o ajudará a evitar a exibição de seus (possivelmente) comentários privados ao público porque você acidentalmente formatou um comentário errado.

TIP Cuidado, entretanto, com comentários muito privados. Embora invisíveis ao visitar sua página normalmente no navegador, eles podem ser vistos por meio do recurso Visualizar código-fonte do navegador ou se o usuário salvar a página como código HTML (fonte).

TIP Os comentários não podem ser aninhados dentro outros comentários.

TIP A sintaxe mostrada é para comentários HTML apenas. CSS e JavaScript têm uma sintaxe de comentários diferente. CSS e JavaScript usam `/*` O comentário vai aqui `*/` para um comentário que cobre uma ou mais linhas, enquanto o JavaScript também usa `//` O comentário vai aqui para comentários de linha única.

Esta página foi intencionalmente deixada em branco